HTBL Krems

Höhere Lehranstalt für Informationstechnologie

DIPLOMARBEIT

DropIT - strukturierte digitale Dokumentenablage

Ausgeführt im Schuljahr 2025/26 von:

Jacob Toifl	5AHIT-01	Winkler Norbert, MSc
Michael Schaider	5AHIT-02	Winkler Norbert, MSc

Betreuer:

Krems, am 01.04.2026

EIDESSTATTLICHE ERKLÄRUNG

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt und die den benutzten Quellen wörtlich und inhaltlich entnommenen Stellen als solche erkenntlich gemacht habe.

Krems, (Datum)

Verfasser/Verfasserinnen:

Jacob Toifl

Michael Schaider

DIPLOMARBEIT

Bestätigung der Abgabe

Abgabebestätigung

Datum	Name	Unterschrift

Genehmigung der Diplomarbeit

Approbation

Datum	Prüfer*in	Abteilungsleiter*in Direktor*in

DIPLOMARBEIT

Dokumentation

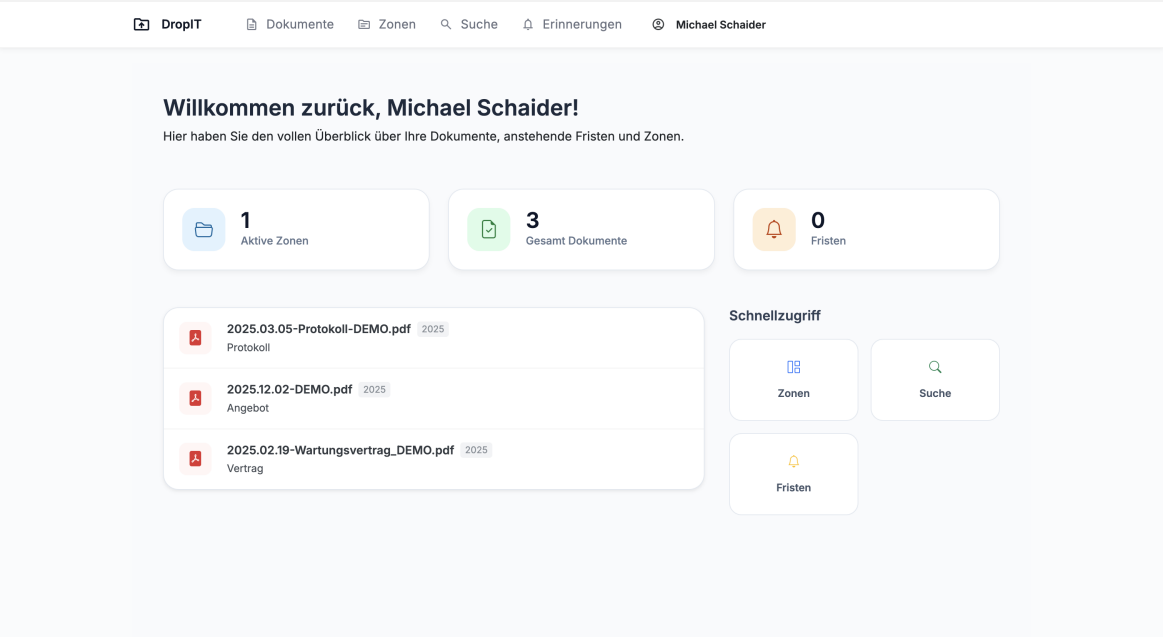
Verfasser*innen
Jacob Toifl, 5AHIT
Michael Schaider, 5AHIT
Abteilung
Informationstechnologie
Ausbildungsschwerpunkt:
Schuljahr
2025/2026
Thema der Diplomarbeit
DropIT - strukturierte digitale Dokumentenablage
Kooperationspartner
MBIT Solutions GmbH

Aufgabenstellung
In vielen Unternehmen fehlt eine einheitliche und strukturierte digitale Dokumentenablage. Verträge und Unterlagen werden unsystematisch gespeichert, wodurch klare Zugriffsrechte fehlen und wichtige Informationen schwer auffindbar sind. Dies führt zu Fristversäumnissen, erhöhtem Suchaufwand, Fehlern und Sicherheitsrisiken. Ziel dieser Diplomarbeit ist die Entwicklung von DropIT – einer zentralen, benutzerfreundlichen Dokumentenablage mit KI-gestützter Klassifizierung. Das System soll Dokumente automatisch erkennen und kategorisieren, eine schnelle Suche ermöglichen und über ein Erinnerungssystem vor Fristversäumnissen schützen. Die Integration erfolgt über Microsoft SharePoint und Entra ID.

Realisierung
Das System wurde als moderne Webanwendung entwickelt, bei der sich Benutzer mit ihrem bestehenden Microsoft-Firmenkonto anmelden können. Dokumente werden automatisch in der vorhandenen SharePoint-Umgebung des Unternehmens gespeichert, sodass keine separate Datenhaltung erforderlich ist. Ein zentrales Element ist die intelligente Dokumentenerkennung: Beim Hochladen analysiert das System automatisch den Inhalt und erkennt, um welchen Dokumententyp es sich handelt – etwa einen Vertrag, eine Rechnung oder ein Angebot. Relevante Informationen wie Ablaufdaten oder Kundennamen werden dabei automatisch ausgelesen. Die Anwendung ist so aufgebaut, dass sie problemlos mit steigenden Anforderungen wachsen kann und sich flexibel erweitern lässt.

Ergebnisse

Im Rahmen dieser Diplomarbeit wurde eine voll funktionsfähige Dokumentenverwaltung entwickelt. Benutzer können sich über ihr bestehendes Microsoft-Konto anmelden und erhalten automatisch die entsprechenden Berechtigungen. Das Herzstück der Anwendung ist die KI-gestützte Dokumentenerkennung, die hochgeladene Dateien automatisch klassifiziert und relevante Metadaten wie Ablaufdaten oder Vertragspartner extrahiert. Dokumente können per Drag & Drop in frei definierbare Upload-Zonen abgelegt werden, wobei die Ablage direkt in SharePoint erfolgt. Die implementierte Volltextsuche ermöglicht ein schnelles Auffinden von Dokumenten nach Schlagworten, Dokumententyp oder Datum. Zusätzlich können Dokumente direkt in der Anwendung in einer Vorschau angezeigt und heruntergeladen werden. Das integrierte Erinnerungssystem benachrichtigt Benutzer rechtzeitig über ablaufende Fristen und anstehende Aufgaben, wodurch Fristversäumnisse vermieden werden.



DIPLOMA THESIS

Documentation

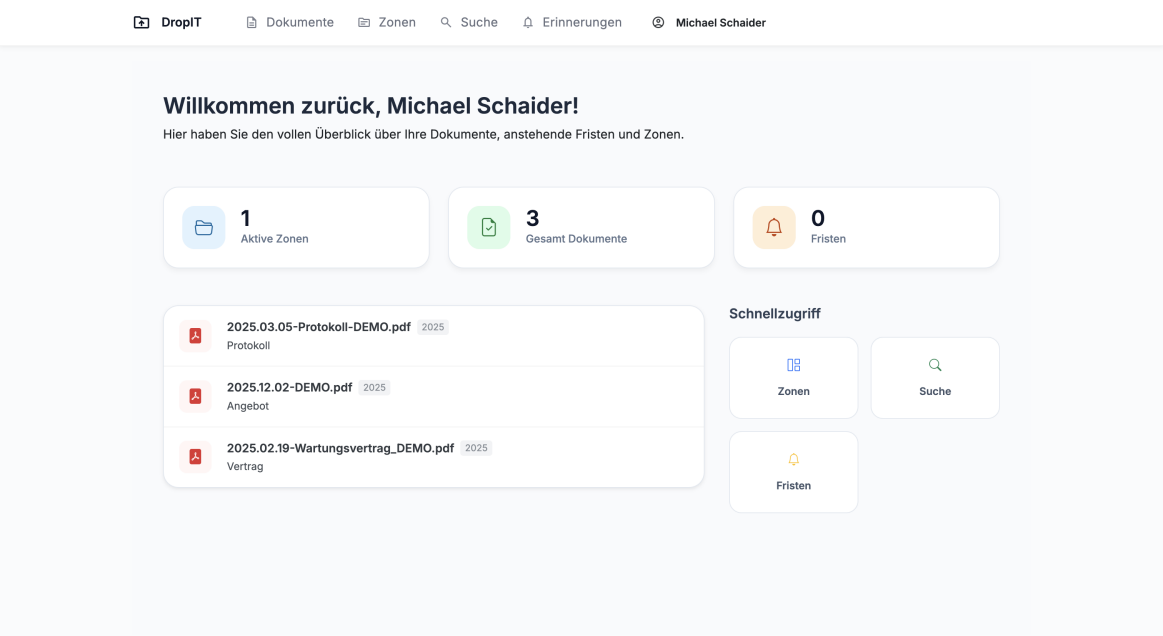
Authors
Jacob Toifl, 5AHIT
Michael Schaider, 5AHIT
Department
Information Technology
Specialization:
Academic year
2025/2026
Thesis Topic
DropIT - structured digital document storage
Cooperation Partner
MBIT Solutions GmbH

Task Description
Many companies lack a unified and structured digital document storage system. Contracts and documents are stored unsystematically, resulting in unclear access rights and important information being difficult to find. This leads to missed deadlines, increased search efforts, errors, and security risks. The goal of this diploma thesis is the development of DropIT – a centralized, user-friendly document storage system with AI-powered classification. The system should automatically recognize and categorize documents, enable fast searching, and protect against missed deadlines through a reminder system. Integration is achieved through Microsoft SharePoint and Entra ID.

Implementation
The system was developed as a modern web application where users can log in with their existing Microsoft company account. Documents are automatically stored in the company's existing SharePoint environment, eliminating the need for separate data storage. A key feature is the intelligent document recognition: when uploading, the system automatically analyzes the content and identifies the document type – such as a contract, invoice, or proposal. Relevant information like expiration dates or customer names is extracted automatically. The application is designed to scale easily with growing requirements and can be flexibly extended.

Results

As part of this diploma thesis, a fully functional document management system was developed. Users can log in with their existing Microsoft account and automatically receive the appropriate permissions. The core of the application is the AI-powered document recognition, which automatically classifies uploaded files and extracts relevant metadata such as expiration dates or contract partners. Documents can be uploaded via drag & drop into freely definable upload zones, with storage directly in SharePoint. The implemented full-text search enables quick retrieval of documents by keywords, document type, or date. Additionally, documents can be previewed directly in the application and downloaded. The integrated reminder system notifies users in advance about expiring deadlines and upcoming tasks, preventing missed deadlines.



Inhaltsverzeichnis

1. Präambel 10
1.1. Kurzfassung . . . 10
1.2. Abstract . . . 10
1.3. Team . . . 10
1.4. Danksagung . . . 10
1.5. Gendererklärung . . . 11

2. Einleitung 12
2.1. Ausgangslage . . . 12
2.2. Forschungsfrage . . . 12
2.2.1. Dynamische Zugriffskontrolle und Skalierbarkeit von Dokumentensystemen . . . 12
2.2.2. Automatisierte Dokumentenanalyse und Suchoptimierung . . . 13

3. Theoretische Grundlagen 14
3.1. Künstliche Intelligenz zur Dokumentenverarbeitung . . . 14
3.1.1. KI-gestützte Dokumentenklassifikation . . . 14
3.1.2. Architekturen zur Integration externer KI-Dienstleister . . . 16
3.1.3. Merkmalsextraktion und Embedding . . . 19
3.1.4. Modellfamilien zur Dokumenttypenerkennung . . . 23
3.1.5. Datenaufbereitung und Preprocessing für Dokumente . . . 25
3.2. Architekturen für Dokumentensysteme . . . 27
3.2.1. Skalierbare plattformunabhängige Systemarchitekturen . . . 27
3.2.2. SharePoint . . . 32
3.2.3. Rollen- und Berechtigungssysteme . . . 34
3.2.4. Microsoft Entra ID . . . 37
3.2.5. Suche und Filter-Architekturen . . . 40

4. Implementierung 44
4.1. Grundlagen . . . 44
4.1.1. Systemarchitektur . . . 44
4.1.2. Technologien . . . 44
4.2. Funktionen . . . 46
4.2.1. Dokumenten-Upload . . . 46
4.2.2. Dokumenten-Klassifikation . . . 54
4.2.3. Dokumenten-Suche . . . 58
4.2.4. Benutzer- und Rollenverwaltung . . . 62
4.2.5. API-Endpunkte . . . 64

5. Beurteilung 67
5.1. Bewertung der Implementierung . . . 67
5.2. Erfüllung der Forschungsfragen . . . 67
5.3. Kritische Reflexion und Grenzen . . . 68

6. Installation 69
6.1. Registrierung im Microsoft Entra Admin Center . . . 69
6.1.1. DropIT.Core . . . 69
6.1.2. Backend-APIs . . . 69
6.2. Azure-Ressourcen einrichten . . . 70
6.2.1. Ressourcengruppe . . . 70
6.2.2. Azure AI Search . . . 70
6.3. Start mit Docker Compose . . . 70

7. Zusammenfassung und Ausblick 72
7.1. Zusammenfassung . . . 72
7.2. Ausblick . . . 72

I. Literaturverzeichnis 73

II. Abbildungsverzeichnis 79

III. Tabellenverzeichnis 80

IV. Quellcodeverzeichnis 81

A. Anhang 82
A.1. Arbeitsteilung . . . 82
A.2. Kapitelverzeichnis . . . 82
A.3. Projekttagebücher . . . 83
A.3.1. Projekttagebuch Jacob Toifl . . . 83
A.3.2. Projekttagebuch Michael Schaider . . . 84
A.4. Besprechungsprotokolle . . . 86
A.5. Pflichtenheft . . . 88
A.6. Datenträgerbeschreibung . . . 103

1. Präambel

1.1. Kurzfassung

Die vorliegende Diplomarbeit beschäftigt sich mit der Entwicklung und Implementierung des Systems DropIT, einer modernen Lösung zur strukturierten, sicheren und effizienten Verwaltung von Dokumenten innerhalb einer Organisation. Ziel des Projekts ist es, eine nutzerfreundliche Anwendung zu schaffen, welche den Upload, die Klassifizierung, die Suche sowie die Organisation von Dokumenten zentralisiert und vereinfacht. DropIT integriert sich nahtlos in bestehende Microsoft-Dienste wie SharePoint und Entra ID, wodurch sowohl private als auch unternehmensinterne Anwender von einer verbesserten Übersichtlichkeit, Automatisierung und Datensicherheit profitieren.

1.2. Abstract

DropIT is a user-friendly document management system developed to make it easier to store, organize, and find digital files. The system works together with Microsoft services such as SharePoint and Entra ID, allowing secure login and central storage of documents. With AI-supported classification and metadata extraction, DropIT can automatically recognize document types and important information like contract periods or expiration dates. Features such as full-text search, filters, and a built-in reminder system help users manage documents faster and more efficiently. This thesis describes the idea, design, development, and evaluation of the system. The goal of DropIT is to offer a clear, reliable, and scalable solution that improves document handling for both private users and organizations.

1.3. Team

Das Projektteam besteht aus:

- **Jacob Toifl** – Projektleiter
- **Michael Schaider** – Projektmitglied

1.4. Danksagung

Wir möchten uns an dieser Stelle herzlich bei allen Personen bedanken, die uns während der Erstellung dieser Diplomarbeit unterstützt haben. Besonderer Dank gilt unserem Betreuer **Winkler Norbert, MSc**, für seine fachliche Beratung, seine Unterstützung im Entwicklungsprozess und seine wertvollen Rückmeldungen. Ebenso bedanken wir uns bei der Firma **MBIT Solutions GmbH** für die tolle Zusammenarbeit und die Bereitstellung der notwendigen Ressourcen und Infrastruktur, die maßgeblich zum Erfolg dieses Projekts beigetragen haben.

1.5. Gendererklärung

Zur besseren Lesbarkeit der Diplomarbeit wurde ausschließlich die männliche Form verwendet. Da Begriffe wie „Benutzerinnen und Benutzer“ den Text unleserlich machen, wurde es schlicht auf „Benutzer“ gekürzt, dies soll jedoch keine Geschlechterdiskriminierung zum Ausdruck bringen.

2. Einleitung

2.1. Ausgangslage

Die Ausgangslage dieser Arbeit ist durch eine fehlende einheitliche digitale Dokumentenablage geprägt. Dokumente werden in unterschiedlichen Strukturen gespeichert, sind nur mit hohem Aufwand auffindbar und unterliegen keiner durchgängig zentral verwalteten Zugriffskontrolle. Dadurch entstehen organisatorische und technische Probleme, die sowohl die tägliche Arbeit als auch die Sicherheit und Nachvollziehbarkeit der Dokumentenverwaltung beeinträchtigen.

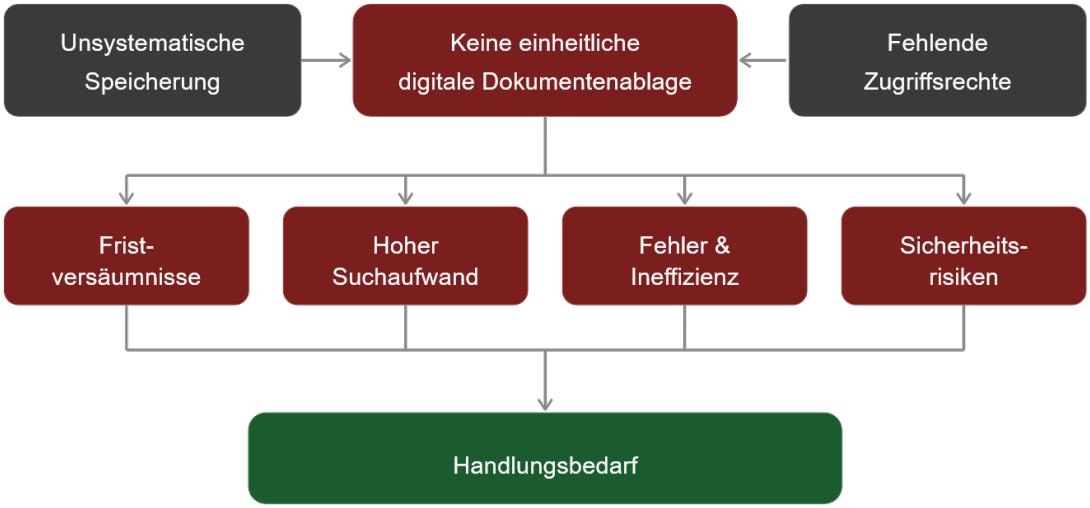


Abbildung 2.1.: Schematische Darstellung der Ausgangslage

Abbildung 2.1 zeigt diese Situation in vereinfachter Form. Eine unsystematische Speicherung von Dokumenten und fehlende Zugriffsrechte führen dazu, dass keine einheitliche digitale Dokumentenablage vorhanden ist. Daraus ergeben sich in der Praxis ein hoher Suchaufwand, Fristversäumnisse, Fehler und Ineffizienz in Arbeitsabläufen sowie erhöhte Sicherheitsrisiken. Insgesamt entsteht daraus ein klarer Handlungsbedarf, die Dokumentenverwaltung strukturiert, sicher und effizient neu zu gestalten.

2.2. Forschungsfrage

2.2.1. Dynamische Zugriffskontrolle und Skalierbarkeit von Dokumentensystemen

Im architektonischen Teil dieser Arbeit steht die Frage im Mittelpunkt, wie ein Dokumentensystem so entworfen werden kann, dass es sowohl sicher als auch langfristig erweiterbar bleibt. Dabei geht es insbesondere um die Verbindung von Zugriffskontrolle, Administrierbarkeit und technischer Skalierbarkeit in einer webbasierten Umgebung.

Aus diesem Themenfeld ergeben sich folgende Forschungsfragen:

1. Wie kann ein flexibles Rollen- und Berechtigungssystem entworfen werden, das sich dynamisch an unterschiedliche Benutzergruppen anpasst und dennoch leicht administrierbar bleibt?
2. Wie lässt sich eine skalierbare und plattformunabhängige Architektur für eine webbasierte Dokumentenablage entwerfen, die große Datenmengen performant verarbeitet und gleichzeitig hohe Sicherheitsanforderungen erfüllt?

Mit diesen Fragen soll analysiert werden, welche Architektur- und Sicherheitsprinzipien für ein modernes Dokumentensystem geeignet sind. Im Fokus stehen dabei die strukturierte Vergabe von Berechtigungen, die zentrale Verwaltung von Identitäten und Zugriffsrechten sowie der Aufbau einer technischen Architektur, die Lastspitzen, wachsende Dokumentbestände und unterschiedliche Zielplattformen zuverlässig bewältigen kann.

2.2.2. Automatisierte Dokumentenanalyse und Suchoptimierung

Im zweiten Schwerpunkt dieser Arbeit wird untersucht, wie Dokumente automatisiert inhaltlich verarbeitet und anschließend effizient wieder aufgefunden werden können. Dabei stehen sowohl die KI-gestützte Erkennung des Dokumenttyps als auch die Ausgestaltung einer geeigneten Such- und Filterarchitektur im Vordergrund.

Aus diesem Themenfeld ergeben sich folgende Forschungsfragen:

1. Wie genau können KI-Modelle anhand des Inhalts erkennen, um welchen Dokumententyp es sich handelt?
2. Welche Architektur für Suche und Filterung bietet die höchste Geschwindigkeit und Genauigkeit bei der Auffindbarkeit von Dokumenten?

Mit diesen Fragen soll erklärt werden, auf welcher technischen Grundlage Dokumente automatisiert klassifiziert und für nachgelagerte Prozesse aufbereitet werden können. Zusätzlich wird analysiert, welche Suchlogiken und Metadatenstrukturen erforderlich sind, damit Dokumente in einer wachsenden Ablage schnell, präzise und benutzerorientiert gefunden und eingegrenzt werden können.

3. Theoretische Grundlagen

3.1. Künstliche Intelligenz zur Dokumentenverarbeitung

Dieses Kapitel beschreibt die theoretischen Grundlagen für den Einsatz von künstlicher Intelligenz in der Dokumentenverarbeitung. Zunächst wird erläutert, wie Dokumente mit KI klassifiziert und externe KI-Dienste in Systeme eingebunden werden können. Anschließend werden zentrale Konzepte der Merkmalsextraktion und von Embeddings erklärt, bevor darauf aufbauend verschiedene Modellfamilien zur Dokumenttypenerkennung sowie Anforderungen an die Datenaufbereitung dargestellt werden.

3.1.1. KI-gestützte Dokumentenklassifikation

Dieses Unterkapitel gibt einen Überblick über die Grundlagen der KI-gestützten Dokumentenklassifikation. Dabei werden die Definition, die Abgrenzung zur Textklassifikation sowie der typische Ablauf eines KI-basierten Klassifikationssystems beschrieben.

3.1.1.1. Definition von Dokumentenklassifikation

Bei der Dokumentenklassifikation werden eingehende Unterlagen vordefinierten Klassen zugeordnet, damit sie anschließend passend gespeichert, weiterverarbeitet oder an nachgelagerte Workflows übergeben werden können. Allgemein beschreibt Klassifikation die Aufgabe, für ein Objekt aus einer Menge definierter Klassen die passende Klasse zu bestimmen. Im Unterschied zur reinen Textklassifikation berücksichtigt Dokumentenklassifikation jedoch nicht nur den sprachlichen Inhalt, sondern zusätzlich Layout, Seitenstruktur und bei gescannten Unterlagen auch visuelle Merkmale. [1, 2, 3, 4]

Für digitale oder gescannte Dokumente ist daher häufig eine vorgelagerte Analyse notwendig. OCR- und Layout-Verfahren lesen gedruckten oder handschriftlichen Text aus, erkennen Tabellen, Formularelemente und Seitenstrukturen und stellen diese Informationen für die eigentliche Klassifikation bereit. Offizielle Plattformdokumentationen von Microsoft, Amazon und Google beschreiben Dokumentenklassifikation entsprechend nicht als reine Textanalyse, sondern als Kombination aus Texterkennung, Strukturverständnis und anschließender Zuordnung zu Dokumenttypen. [5, 6, 7, 8]

Die Begriffe Dokumentklassifizierung und Textklassifizierung werden häufig synonym verwendet, weisen jedoch einige Unterschiede auf, wie in Tabelle 3.1 ersichtlich.

Im Allgemeinen lässt sich sagen, dass Textklassifizierung eine Teilmenge der Dokumentenklassifizierung ist, die sich ausschließlich auf den Textinhalt konzentriert, während Dokumentenklassifizierung den Merkmalsraum um Layout- und Bildinformationen erweitert. Dieser multimodale Ansatz wird auch in der Forschung zu „visually rich documents“ als zentral beschrieben, weil sich viele Dokumenttypen nicht allein über ihren Wortlaut, sondern zusätzlich über Positionen, Tabellen, Überschriften und Formularstrukturen unterscheiden. [4, 9, 10]

Aspekt	**Textklassifizierung**	**Dokumentenklassifizierung**
Geltungsbereich	Analysiert nur Textinhalt.	Analysiert Text sowie Layout- und Bildelemente.
Eingabedaten	Rein textliche Daten (Sätze, Absätze).	Gesamtes Dokument inkl. Bilder und Tabellen.
Anwendungsfälle	Sentiment, Themenzuordnung, Spam-Erkennung.	Rechnungen, Verträge, Formulare.
Techniken	NLP-Methoden.	Kombination aus NLP, Computer Vision und OCR.

Tabelle 3.1.: Textklassifizierung vs. Dokumentenklassifizierung [1, 2, 3]

3.1.1.2. Funktion der Dokumentenklassifizierung

Die Dokumentenklassifizierung kann manuell, regelbasiert oder automatisiert erfolgen. Manning et al. beschreiben manuelle Zuordnung und handkodierte Regeln als frühe, inhaltlich nachvollziehbare Ansätze, weisen jedoch darauf hin, dass diese Verfahren mit wachsendem Dokumentvolumen schwer skalierbar werden. In heutigen Systemen werden daher überwiegend Klassifikatoren mit gelabelten Beispieldokumenten trainiert, um neue Unterlagen anschließend ohne menschliches Eingreifen systematisch zuzuordnen. [1, 11, 8]

Strukturierte Dokumente weisen klar definierte, wiederkehrende Muster auf, etwa konsistente Nummerierungen, feste Felder oder stabile Layouts. Solche Unterlagen eignen sich besonders gut für layoutgestützte Klassifikation, weil Modelle neben dem Text auch Positionen und Strukturelemente lernen können. Amazon unterscheidet hierfür explizit zwischen Plain-Text-Modellen und Native-Document-Modellen; letztere nutzen zusätzlich Signale aus dem originären Dokumentlayout. Microsoft beschreibt seine Custom-Classifier analog als Deep-Learning-Modelle, die Sprach- und Layoutmerkmale kombinieren. [2, 3]

Unstrukturierte Dokumente liegen dagegen in freieren und weniger standardisierten Formaten vor, etwa als Schreiben, Verträge oder Bestellungen mit variierendem Aufbau und sprachlicher Gestaltung. Dadurch steigt der Aufwand für Datenaufbereitung, Trennschärfe zwischen Klassen und Nachtraining. Google empfiehlt für ähnliche oder schwer unterscheidbare Klassen deshalb zusätzliche Trainings- und Testdokumente, während Microsoft Konfidenzschwellen und eine „other“-Klasse als praktische Absicherung gegen unbekannte Dokumenttypen nennt. [8, 3]

3.1.1.3. Funktion der KI-basierten Dokumentenklassifizierung

Die automatisierte Klassifizierung von Dokumenten mit KI erfolgt typischerweise in mehreren aufeinanderfolgenden Schritten:

1. **Datensammlung und Beschriftung**
   Die Basis sind qualitativ hochwertige, gelabelte Trainingsdaten. Dafür werden Dokumente je Klasse gesammelt, mit Labels versehen und in Trainings- und Testmengen

aufgeteilt. Microsoft nennt mindestens zwei Klassen und fünf Beispieldokumente pro Klasse als Untergrenze, Google empfiehlt pro Label getrennte Trainings- und Testdokumente und verweist darauf, dass sich ähnliche Klassen durch zusätzliche Beispiele robuster unterscheiden lassen. [3, 8]

2. **Vorverarbeitung und Feature-Erzeugung**
   Liegt ein Dokument als Scan, Bild oder PDF vor, wird der enthaltene Text zunächst ausgelesen. Amazon Comprehend führt dafür je nach Dokumenttyp automatische Textverarbeitung durch und greift bei gescannten Dokumenten auf Textract zurück. Azure Document Intelligence ergänzt diese Vorverarbeitung um Read- und Layout-Modelle, die Text, Tabellen und Seitenstruktur extrahieren. Auf dieser Basis entstehen Text-, Layout- und Bildmerkmale für die Klassifikation. [6, 7, 5]
3. **Training des Klassifikationsmodells**
   Anschließend werden überwachte Modelle mit den gelabelten Beispielen trainiert. In der Praxis reicht das Spektrum von textbasierten Klassifikatoren bis zu multimodalen Modellen, die Text, Layout und visuelle Informationen gemeinsam verarbeiten. Die Forschungslinie um LayoutLM und LayoutLMv2 zeigt, dass diese gemeinsame Modellierung die Leistung auf Aufgaben der Dokumentklassifikation und des Dokumentverständnisses messbar verbessert. [2, 3, 9, 10]
4. **Evaluation und Feintuning**
   Im Anschluss wird das Modell mit bislang unbekannten Testdaten geprüft. In der Praxis werden dabei Kennzahlen wie Präzision, Recall, F1-Score und Konfidenzwerte betrachtet, um die Trennschärfe zwischen Klassen zu bewerten. Google stellt diese Metriken direkt in der Evaluationsansicht bereit; Microsoft empfiehlt zusätzlich die Nutzung von Konfidenzschwellen, wenn unbekannte Dokumenttypen im Produktivbetrieb auftreten können. [8, 3]
5. **Produktivbetrieb und laufende Anpassung**
   Nach der Bereitstellung ordnet das Modell neue Dokumente automatisch zu und kann über neue Beispiele fortlaufend verbessert werden. Google unterstützt dafür Auto-Labeling und erneutes Training, Microsoft zusätzlich inkrementelles Training auf Basis eines bestehenden Klassifikators. Dadurch lassen sich neue Klassen oder veränderte Dokumentmuster schrittweise in den laufenden Betrieb übernehmen. [8, 3]

3.1.2. Architekturen zur Integration externer KI-Dienstleister

Dieses Unterkapitel gibt einen Überblick über zentrale Architekturansätze zur Einbindung externer KI-Dienstleister. Dabei werden typische Integrationsmuster sowie Anforderungen an Sicherheit, Resilienz und Performance beschrieben.

3.1.2.1. Grundidee der Integration externer KI-Dienste

Externe KI-Dienstleister stellen ihre Modelle typischerweise als verwaltete Cloud- oder API-Dienste bereit. Für Unternehmen reduziert das den Aufwand für den Eigenbetrieb komplexer Modelle, beschleunigt die Einführung neuer KI-Funktionen und ermöglicht einen schnellen

Zugriff auf aktuelle Modellgenerationen. Gleichzeitig verlagert sich ein Teil der Systemarchitektur von der Modellimplementierung hin zur sauberen Integration externer Abhängigkeiten, etwa für Authentifizierung, Quotensteuerung, Fehlertoleranz und Kostenkontrolle. [12, 13]

3.1.2.2. Direkte Kopplung und Vermittlungsschicht

Die einfachste Integrationsform ist die direkte Kopplung einer Anwendung an den Endpunkt eines externen KI-Anbieters. Dieses Muster eignet sich vor allem für Prototypen oder kleine Lösungen mit nur wenigen Konsumenten. Mit zunehmender Systemgröße entstehen jedoch Nachteile: Modellwechsel, neue Endpunkte oder geänderte Authentifizierungsverfahren müssen direkt in den konsumierenden Anwendungen umgesetzt werden. Eine vorgeschaltete Vermittlungsschicht in Form eines Gateways oder eines internen KI-Adapters entkoppelt daher die Clients vom konkreten Anbieter und erlaubt serverseitige Routing-Entscheidungen, ohne Client-Code oder Konfiguration neu ausrollen zu müssen. [14]

Ein zentrales Gateway übernimmt in diesem Zusammenhang nicht nur die Weiterleitung von Requests, sondern auch Governance-Aufgaben. Laut Microsoft Learn kann ein AI-Gateway Authentifizierung und Autorisierung bündeln, Anfragen über mehrere KI-Endpunkte verteilen, Interaktionen überwachen, Tokenverbrauch begrenzen und verschiedene Anbieter über eine gemeinsame Verwaltungsschicht anbinden. Dadurch entsteht eine providernahe, aber fachlich zentralisierte Integrationsarchitektur, die besonders in größeren Organisationen die Standardisierung vereinfacht. [13]

3.1.2.3. Asynchrone Integrationsmuster

Für dokumentenbezogene KI-Anwendungsfälle ist eine rein synchrone Kommunikation oft nicht ausreichend. OCR, Klassifikation, Extraktion oder mehrstufige Prompt-Ketten können zu längeren Laufzeiten führen, die eine direkte Request-Response-Kommunikation unnötig blockieren würden. Das Architekturpattern „Asynchronous Request-Reply“ entkoppelt deshalb die Annahme einer Anfrage von ihrer eigentlichen Verarbeitung: Der Client erhält zunächst eine Bestätigung, während die Verarbeitung in einer Queue oder einer nachgelagerten Komponente stattfindet und der Status später über Polling, Redirects oder Benachrichtigungsmechanismen abgefragt wird. Dadurch steigen Reaktionsfähigkeit, Parallelität und Skalierbarkeit der Gesamtlösung. [15]

3.1.2.4. Sicherheit, Governance und Datenhoheit

Sobald externe KI-Dienstleister in produktive Systeme eingebunden werden, wird die Sicherheitsarchitektur zu einem eigenständigen Entwurfsbereich. Ein Gateway erlaubt es, Zugangsdaten an zentraler Stelle zu terminieren und gegenüber dem externen Dienst mit einer eigenen Identität aufzutreten. Microsoft empfiehlt hierfür insbesondere Managed Identities, damit API-Schlüssel nicht in Anwendungen, Konfigurationsdateien oder Quellcode verteilt werden müssen. Darüber hinaus lassen sich über die Vermittlungsschicht Protokollierung, Content-Safety-Policies und feinere Zugriffskontrollen umsetzen. [13, 14]

Hinzu kommt die Frage der Datenhoheit. Bei KI-Services, die über mehrere Instanzen, Regionen oder Anbieter verteilt werden, müssen Architekturentscheidungen auch unter Compliance- und Residency-Gesichtspunkten betrachtet werden. Microsoft weist ausdrücklich darauf hin, dass active-active- oder active-passive-Strategien nur innerhalb der zulässigen geopolitischen Grenzen angewendet werden sollten, wenn Datenresidenz eine Anforderung darstellt. Für sensible Dokumentensysteme bedeutet das, dass Routing- und Failover-Logik nicht nur nach Verfügbarkeit, sondern auch nach regulatorischen Vorgaben entworfen werden muss. [14]

3.1.2.5. Resilienz gegenüber Anbieterfehlern

Externe KI-Dienste sind aus Sicht der eigenen Anwendung entfernte Ressourcen und damit grundsätzlich anfällig für temporäre Fehler wie Timeouts, Überlastung oder kurzzeitige Nichterreichbarkeit. Das Retry-Pattern adressiert solche vorübergehenden Störungen, indem fehlgeschlagene Aufrufe kontrolliert wiederholt werden. Die Dokumentation des Azure Architecture Center empfiehlt dabei insbesondere verzögerte und bei Bedarf exponentiell ansteigende Wiederholungen, um ausgelastete Dienste nicht zusätzlich zu belasten. Für Integrationen mit KI-Dienstleistern ist diese Logik zentral, weil Lastspitzen und Quotenengpässe in der Praxis regelmäßig auftreten können. [16]

Nicht jeder Fehler ist jedoch nur kurzfristig. Dauert eine Störung länger an, reicht Wiederholen allein nicht aus. Das Circuit-Breaker-Pattern verhindert in diesem Fall weitere Aufrufe an eine wahrscheinlich fehlerhafte Abhängigkeit und ermöglicht so eine sanfte Degradation des Gesamtsystems. Gerade bei der Einbindung externer KI-APIs schützt dieses Muster davor, dass ein gestörter Anbieter die gesamte Anwendung blockiert oder nachgelagerte Komponenten überlastet. Microsoft beschreibt explizit die Kombination von Retry und Circuit Breaker als robusten Ansatz zur Fehlerbehandlung in verteilten Systemen. [17, 16]

3.1.2.6. Performance-, Kosten- und Routingoptimierung

Neben Sicherheit und Resilienz spielt bei externen KI-Dienstleistern die effiziente Nutzung von Tokens, Durchsatz und Latenz eine zentrale Rolle. Ein Gateway kann hierfür Tokenlimits pro Konsument durchsetzen, Last über mehrere Backends verteilen und semantisches Caching einsetzen, um wiederkehrende Anfragen nicht erneut vollständig an den Provider zu senden. Auf Anbieter-Seite beschreibt OpenAI ergänzend, dass sich Latenzen architektonisch unter anderem durch weniger Requests, Parallelisierung unabhängiger Schritte, kleinere Modelle für Teilaufgaben und klassische Optimierungstechniken reduzieren lassen. Damit wird deutlich, dass die Integrationsarchitektur nicht nur eine Verbindungsschicht ist, sondern aktiv die Wirtschaftlichkeit und Antwortzeit von KI-Funktionen beeinflusst. [13, 18]

Ein weiterer Optimierungshebel ist Caching auf Prompt-Ebene. OpenAI dokumentiert, dass Prompt Caching bei langen, wiederkehrenden Präfixen Latenz und Eingabekosten deutlich senken kann, wenn statische Instruktionen am Anfang und variable Inhalte am Ende eines Prompts angeordnet werden. In Verbindung mit einem zentralen Gateway oder einem internen KI-Service entsteht daraus ein zweistufiges Optimierungsmodell: Einerseits kann die

Integrationsschicht Anfragen deduplizieren oder semantisch cachen, andererseits nutzt der externe Anbieter dieselben oder ähnliche Prompt-Präfixe effizienter. [19, 13]

3.1.2.7. Einordnung für Dokumentenverarbeitungssysteme

Für Systeme zur Dokumentenverarbeitung ergibt sich daraus ein klares Zielbild: Externe KI-Dienstleister sollten nicht direkt und unkontrolliert aus jeder Fachkomponente heraus aufgerufen werden, sondern über eine definierte Integrationsarchitektur mit zentralem Routing, Sicherheitskontrollen, Quotenmanagement und Beobachtbarkeit. Ergänzt um asynchrone Verarbeitungsmuster, Retry-/Circuit-Breaker-Mechanismen sowie providernahe Caching- und Routingstrategien entsteht eine Architektur, die sowohl technisch robust als auch organisatorisch beherrschbar bleibt. Genau diese Entkopplung ist entscheidend, wenn KI-Funktionen langfristig in produktive Dokumenten- und Unternehmensprozesse eingebettet werden sollen. [12, 13, 15, 17]

3.1.3. Merkmalsextraktion und Embedding

Dieses Unterkapitel gibt einen Überblick über die Grundlagen der Merkmalsextraktion und von Embeddings. Dabei werden die Bedeutung dieser Konzepte für KI-Systeme sowie ihre Unterschiede und Einsatzmöglichkeiten in der Dokumentenverarbeitung beschrieben.

3.1.3.1. Definition von Merkmalsextraktion

Merkmalsextraktion beschreibt die Umwandlung roher Daten in eine Form, die von Lernverfahren direkt verarbeitet werden kann. Das scikit-learn-Handbuch grenzt diesen Schritt explizit von der Merkmalsauswahl ab: Bei der Extraktion werden Texte, Bilder oder andere Eingaben zunächst in numerische bzw. strukturierte Merkmale transformiert, während die Merkmalsauswahl erst auf diesen bereits erzeugten Merkmalen arbeitet. Für Dokumentensysteme bedeutet das, dass Textinhalt, Layoutsignale oder Metadaten zunächst in eine modellierbare Repräsentation überführt werden müssen. [20, 5]

Je nach Datentyp kommen dafür unterschiedliche Repräsentationen zum Einsatz. In der Textverarbeitung zählen Bag-of-Words-, TF-IDF- oder Hashing-Verfahren zu den klassischen Ansätzen; neuere neuronale Verfahren lernen Merkmale dagegen direkt aus großen Datenbeständen. Unabhängig vom konkreten Verfahren besteht das Ziel darin, die für eine Aufgabe relevanten Informationen kompakt abzubilden und störende Varianz zu reduzieren. [1, 21, 20]

Wie in Tabelle 3.2 ersichtlich, werden im Folgenden wichtige Begriffe der Merkmalsextraktion mit kurzer Beschreibung dargestellt.

Begriff	**Beschreibung**
Merkmal	Eine quantitativ oder qualitativ erfassbare Eigenschaft, die ein Datenobjekt beschreibt.
Feature Extraction	Verfahren, bei dem aus Rohdaten gezielt die aussagekräftigen Eigenschaften herausgelöst bzw. abgeleitet werden.
Vektor	Geordnete Sammlung von Merkmalwerten, die ein Objekt in strukturierter Form repräsentiert.
Dimensionalität	Anzahl der enthaltenen Merkmale bzw. Einträge eines Vektors.

Tabelle 3.2.: Wichtige Begriffe der Merkmalsextraktion [20, 22]

3.1.3.2. Funktion der KI-gestützten Merkmalsextraktion

KI-gestützte Merkmalsextraktion ist ein zentraler Verarbeitungsschritt, weil Modelle nicht auf Rohdaten, sondern auf geeigneten Repräsentationen trainiert werden. Bei Text- und Dokumentdaten reicht das Spektrum von explizit konstruierten Merkmalen wie Wortzählungen und Gewichtungen bis zu automatisch gelernten Repräsentationen aus neuronalen Netzen. Klassische Verfahren sind transparent und effizient, neuronale Verfahren erfassen zusätzlich Kontext und semantische Zusammenhänge. [20, 1, 23]

Die in Tabelle 3.3 aufgeführten Verfahren sind typische Methoden der Merkmalsextraktion.

Verfahren	**Beschreibung**
Bag-of-Words / Count-Vektoren	Zählt das Auftreten von Begriffen und bildet daraus eine meist sparse Merkmalsmatrix.
TF-IDF	Gewichtet Begriffe nach Häufigkeit im Dokument und Seltenheit im Gesamtkorpus, um trennschärfere Textmerkmale zu erhalten.
Feature Hashing	Projiziert Merkmale speichereffizient in einen festen Vektorraum und eignet sich besonders für große, dynamische Merkmalsmengen.
Kontextuelle Sprachrepräsentationen	Lernen Merkmale aus dem Kontext eines Tokens oder Satzes und können dadurch Mehrdeutigkeit und semantische Beziehungen besser abbilden.

Tabelle 3.3.: Relevante Verfahren zur Merkmalsextraktion [20, 21, 23]

Maschinelles Lernen unterstützt die Merkmalsextraktion, indem Modelle aus Trainingsdaten lernen, welche Signale für eine Aufgabe tatsächlich relevant sind. Mikolov et al. zeigen dies bereits für Wortvektoren, indem kontinuierliche Repräsentationen semantische und syntaktische Beziehungen zwischen Wörtern erfassen. Mit BERT wurde dieser Gedanke auf kontextabhängige Sprachrepräsentationen ausgeweitet, bei denen dieselbe Zeichenfolge je nach Umgebung unterschiedlich eingebettet werden kann. [24, 23]

Text Mining ist ein praktischer Anwendungsbereich dieser Verfahren auf unstrukturierten Textbeständen. Hier werden Texte tokenisiert, normalisiert, gewichtet und gegebenenfalls in dichte Repräsentationen überführt, damit Aufgaben wie Klassifikation, semantische Suche oder Clustering möglich werden. Für Dokumentensysteme ist das besonders relevant, weil OCR-Texte, Metadaten und Layoutsignale gemeinsam in eine maschinenlesbare Form gebracht werden müssen. [1, 20, 9]

3.1.3.3. Definition von Embedding

Ein Embedding ist eine dichte Vektorrepräsentation eines Objekts in einem kontinuierlichen Raum. Google beschreibt Embedding-Vektoren als trainierte Arrays aus Fließkommazahlen, bei denen ähnliche Objekte ähnliche Zahlenmuster aufweisen. Damit unterscheiden sich Embeddings von sparsamen Repräsentationen wie One-Hot- oder Count-Vektoren, die zwar interpretierbar sind, semantische Nähe aber nur eingeschränkt abbilden. [22, 20]

Embeddings werden in vielen Anwendungsfeldern genutzt, etwa für semantische Suche, Ähnlichkeitsvergleich, Clustering oder Klassifikation. Die Grundidee ist, dass nicht einzelne Schlüsselwörter, sondern Bedeutungsbeziehungen gelernt werden. Bereits Word2Vec erzeugt kontinuierliche Wortvektoren aus großen Textmengen; neuere Transformer-Modelle wie BERT oder Sentence-BERT erweitern dieses Prinzip auf kontextabhängige Token- und Satzrepräsentationen. [24, 23, 25]

3.1.3.4. Prinzip und Funktionsweise von Embeddings

Bei der Bildung von Embeddings werden Objekte zunächst kodiert und anschließend so in einen Vektorraum projiziert, dass geometrische Nähe inhaltliche Nähe widerspiegelt. Für semantische Textähnlichkeit erzeugen moderne Embedding-Modelle Vektoren für einzelne Texte und vergleichen diese typischerweise über Cosinus-Ähnlichkeit. Die Dokumentation von Sentence Transformers beschreibt dieses Vorgehen explizit: Texte mit dem höchsten Similarity-Score gelten als semantisch am ähnlichsten. [26, 25]

In Abbildung 3.1 ist dieses Prinzip schematisch dargestellt: semantisch ähnliche Begriffe liegen im Vektorraum näher beieinander (z. B. Cat, Dog, Wolf), während thematisch andere Begriffe weiter entfernt positioniert sind (z. B. Apple, Banana). Die räumliche Distanz dient damit als Maß für Ähnlichkeit.

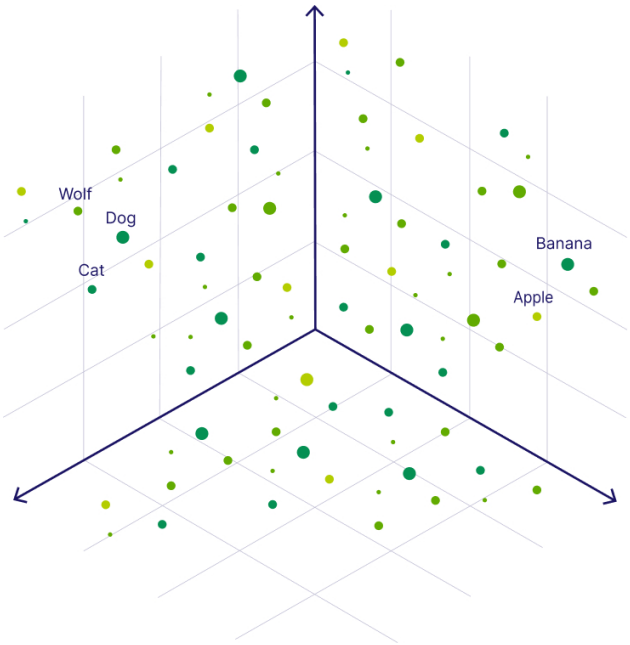


Abbildung 3.1.: Übersicht der Vektordarstellung von Embeddings [27]

Für die Dokumentenverarbeitung ist ein besonders anschauliches Beispiel die semantische Suche: Ein Modell erzeugt Embeddings für Dokumenttitel, Absätze oder ganze Seiten, und eine Suchanfrage wird in denselben Vektorraum projiziert. Dokumente mit hoher Cosinus-Ähnlichkeit zur Anfrage können anschließend priorisiert, gruppiert oder einer tieferen Extraktion zugeführt werden. Genau dafür wurden Satz-Embeddings wie Sentence-BERT entwickelt, weil klassische BERT-Paarvergleiche für große Vergleichsmengen zu teuer wären. [25, 26]

3.1.3.5. Häufige Objekte für Embeddings

Embeddings können auf verschiedenen Granularitätsstufen gebildet werden. Für dokumentenbezogene Systeme sind vor allem die in Tabelle 3.4 dargestellten Objekte relevant.

Objekt	**Beschreibung**
Wörter	Dichte Vektorrepräsentationen einzelner Wörter, die semantische und syntaktische Beziehungen im Korpus erfassen.
Sätze oder Absätze	Vektoren, die die Bedeutung kurzer Textsegmente für Ähnlichkeitssuche, Clustering oder Retrieval verdichten.
Ganze Dokumente	Semantische Repräsentationen kompletter Dokumente oder Dokumentteile, etwa für Routing, Dublettenerkennung oder thematische Suche.
Seitenbilder bzw. Layoutblöcke	Multimodale Repräsentationen, die Textpositionen, visuelle Merkmale und Layout gemeinsam kodieren.

Tabelle 3.4.: Relevante Objekte für Embeddings in Dokumentensystemen [24, 25, 9]

3.1.3.6. Abgrenzung: Merkmalsextraktion vs. Embedding

Merkmalsextraktion ist der Oberbegriff für alle Verfahren, die aus Rohdaten nutzbare Merkmale ableiten. Das Ergebnis können einfache statistische Kennwerte, sparse Vektoren wie TF-IDF oder auch multimodale Layoutmerkmale sein. Der Begriff beschreibt also den Prozess der Repräsentationsbildung insgesamt. [20, 21]

Ein Embedding ist dagegen eine spezielle, gelernte Form dieser Repräsentation. Embeddings sind typischerweise dichte Vektoren in einem kontinuierlichen Raum, in dem semantisch ähnliche Objekte nahe beieinanderliegen. Damit ist jedes Embedding ein Merkmal bzw. ein Merkmalsvektor, aber nicht jede Merkmalsextraktion erzeugt automatisch Embeddings. Genau diese Unterscheidung ist für Dokumentensysteme wichtig: TF-IDF, Layoutmerkmale und OCR-Metadaten sind extrahierte Merkmale, während Satz- oder Dokumentembeddings zusätzlich semantische Nähe direkt im Vektorraum modellieren. [22, 24, 25]

3.1.4. Modellfamilien zur Dokumenttypenerkennung

Die Dokumenttypenerkennung ist ein zentraler Baustein in der KI-gestützten Dokumentenverarbeitung. Ziel ist es, eingehende Dokumente – etwa Rechnungen, Angebote, Verträge oder Formulare – automatisch einem definierten Dokumenttyp zuzuordnen, um nachgelagerte Verarbeitungsschritte wie Routing, Extraktion oder Validierung zu steuern. Je nach Anwendungsfall stehen dafür unterschiedliche Modellansätze zur Verfügung, die sich vor allem darin unterscheiden, welche Informationsquellen eines Dokuments sie nutzen: reinen Textinhalt, visuelle Merkmale einer Seite, Layout- und Positionsinformationen oder Kombinationen daraus. In der Literatur und in praktischen Systemen haben sich daher mehrere „Modellfamilien“ etabliert.

3.1.4.1. Textbasierte Modellfamilie

Textbasierte Verfahren formulieren die Dokumenttypenerkennung als Textklassifikationsproblem: Ein Dokument wird zunächst in eine rein textuelle Darstellung überführt (z. B. durch direkt extrahierbaren PDF-Text oder durch OCR bei gescannten Dokumenten). Anschließend wird dieser Text einer vordefinierten Klasse zugeordnet. Methodisch entspricht dies dem allgemeinen Vorgehen der Textklassifikation, bei dem Textinhalte analysiert und mittels gelabelter Beispiele in Kategorien eingeteilt werden. [28]

Innerhalb dieser Modellfamilie werden sowohl klassische Feature-basierte Ansätze (z. B. Bag-of-Words/gewichtete Termrepräsentationen) als auch neuere neuronale Modelle eingesetzt. Moderne Systeme nutzen häufig kontextuelle Sprachmodelle (Transformer), um neben Schlüsselbegriffen auch semantische Zusammenhänge zu berücksichtigen. Die Leistungsfähigkeit textbasierter Ansätze hängt in der Dokumentenverarbeitung jedoch stark von der Qualität der Texterfassung ab: OCR-Fehler, fehlende Sonderzeichen oder fragmentierter Text können die Klassifikation erschweren. Außerdem werden Layout- und Strukturinformationen eines Dokuments (z. B. Positionen von Feldern oder tabellarische Anordnungen) in einem rein textuellen Ansatz nur indirekt oder gar nicht abgebildet. [28]

3.1.4.2. Bildbasierte Modellfamilie

Bildbasierte Verfahren behandeln Dokumenttypenerkennung als Bildklassifikation: Seiten werden als gerenderte PDF-Seiten oder Scans (Seitenbilder) verarbeitet und auf Basis visueller Merkmale klassifiziert. Solche Ansätze nutzen typischerweise Convolutional Neural Networks (CNNs), die darauf ausgelegt sind, Muster in visuellen Daten zu erkennen und sich besonders für Klassifikationsaufgaben in der Computer Vision eignen. [29]

Ein wichtiges Argument für bildbasierte Ansätze ist, dass Dokumenttypen oft durch wiederkehrende visuelle Strukturen unterscheidbar sind (z. B. Formularcharakter, Briefkopf, typische Anordnung von Blöcken). In der Forschung wird die Dokumentbildklassifikation häufig auf etablierten Datensätzen evaluiert; als verbreitetes Beispiel gilt RVL-CDIP mit 16 Dokumentklassen und insgesamt 400.000 Dokumentbildern. Neben CNNs kommen zunehmend Vision Transformer (ViT) zum Einsatz, die Bilder in Patches zerlegen und diese ähnlich wie Token-Sequenzen verarbeiten, um globale Zusammenhänge besser abzubilden. [30] [31]

3.1.4.3. Layout-aware / dokumentenspezifische Transformer

Layout-aware Modelle erweitern textbasierte Ansätze um räumliche Layoutinformationen. Statt Text nur als Sequenz zu betrachten, wird zusätzlich modelliert, wo ein Token auf der Seite steht (z. B. über Bounding Boxes). Damit lässt sich die Dokumenttypenerkennung als Aufgabe formulieren, bei der Inhalt und Struktur gemeinsam berücksichtigt werden (z. B. typische Positionen von Überschriften, Adressblöcken, Tabellenbereichen). [32]

Ein prominentes Beispiel für diese Modellfamilie ist die LayoutLM-Klasse: In der Implementierung (z. B. in verbreiteten Frameworks) wird explizit ein zusätzliches Eingabesignal "bbox" erwartet, das die Bounding Boxes der Tokens enthält und damit die 2D-Positionen abbildet. [32]

3.1.4.4. Multimodale Foundation-Model-Familie (VLM/LLM)

Multimodale Foundation-Model-Ansätze nutzen große Modelle, die visuelle und häufig auch sprachliche Informationen gemeinsam verarbeiten können. In der Dokumenttypenerkennung werden Dokumentseiten (als Bild bzw. gerendertes PDF) typischerweise zusammen mit einer natürlichsprachlichen Aufgabenbeschreibung (Prompt) an ein Vision-Language-Modell übergeben, das den Dokumenttyp direkt ausgibt. Solche Verfahren sind besonders dann attraktiv, wenn eine schnelle Erweiterung auf neue Klassen erforderlich ist oder wenn man Zero-/Few-shot-Strategien erproben möchte. [33]

Dass diese Vorgehensweise praktisch umgesetzt wird, zeigen Anbieterbeispiele: Ein öffentliches Azure-Sample demonstriert explizit die Klassifikation eines Dokuments mithilfe eines Vision-fähigen GPT-4o-Modells (Prompt + Dokumentbild), also genau das Grundprinzip dieser Modellfamilie. [33]

3.1.5. Datenaufbereitung und Preprocessing für Dokumente

Die Datenaufbereitung ist ein zentraler Erfolgsfaktor in der KI-gestützten Dokumentenverarbeitung, weil Klassifikationsmodelle nur so gut sind wie die Qualität ihrer Eingabedaten. Bereits kleine Inkonsistenzen in Scanqualität, Dokumentstruktur oder Labeling wirken sich direkt auf Modellgüte, Generalisierbarkeit und spätere Produktionsstabilität aus. [34]

Gerade bei heterogenen Dokumentbeständen (z. B. gescannte Belege, digital erzeugte PDFs und Formularbilder) ist Preprocessing daher kein optionaler Zwischenschritt, sondern eine eigene Prozessphase. Sie verbindet die Rohdatenerfassung mit der modellgerechten Repräsentation und reduziert Informationsverluste über die gesamte Pipeline hinweg. [34]

3.1.5.1. Zielsetzung und Qualitätskriterien

Das Ziel der Dokumentaufbereitung besteht darin, Eingaben technisch zu standardisieren, semantisch verwertbar zu machen und gleichzeitig den Informationsgehalt für nachgelagerte Modelle zu erhalten. In der Praxis bedeutet das vor allem: robuste OCR-Ergebnisse, stabile Layoutsignale und konsistente Trainingslabels. [34]

Aus OCR-Sicht hängt die Lesegenauigkeit stark von Eigenschaften wie Auflösung, Schräglage, Rauschen, Binarisierung und Randartefakten ab. Die Tesseract-Dokumentation beschreibt dafür konkrete Vorverarbeitungsschritte wie Rescaling (typisch mindestens 300 dpi), Binarisierung, Rauschreduktion und Deskewing, um die Erkennungsqualität zu erhöhen. [35]

Für dokumentenspezifische Transformer reicht reine Textqualität jedoch nicht aus, weil zusätzlich Layoutinformationen benötigt werden. Modelle wie LayoutLM verarbeiten deshalb nicht nur Tokens, sondern auch räumliche Positionsinformationen (Bounding Boxes), damit Inhalte im Seitenkontext interpretiert werden können. [9] [32]

Die wichtigsten Qualitätsziele der Aufbereitung sind in Tabelle 3.5 zusammengefasst.

Qualitätsziel	**Typische Maßnahme**	**Nutzen für KI-Modelle**
Lesbarkeit des Dokuments	Rescaling, Binarisierung, Rauschfilterung	Stabilere OCR-Texte und weniger fehlerhafte Tokens
Räumliche Konsistenz	Deskewing, Randkorrektur, Box-Normalisierung	Bessere Nutzung von Layoutmerkmalen für Dokumentverständnis
Semantische Konsistenz	Einheitliches Labeling und Klassenregeln	Weniger Labelrauschen, höhere Klassifikationsgüte

Tabelle 3.5.: Qualitätsziele in der Dokumentaufbereitung [35] [9]

3.1.5.2. Technische Preprocessing-Pipeline

Eine belastbare Pipeline folgt typischerweise einem klaren Ablauf von Standardisierung, visueller Korrektur, Textextraktion, Feature-Aufbereitung und Datensplitting. Damit wird

sichergestellt, dass sowohl klassische textbasierte Modelle als auch multimodale Architekturen mit konsistenten Eingaben trainiert werden. [34] [10]

1. **Format- und Eingangsstandardisierung**
   Zu Beginn werden Dateiformate vereinheitlicht (z. B. Scanbilder und PDFs in eine verarbeitbare Seitenrepräsentation), Seitenmetriken geprüft und ungeeignete Dokumente ausgeschlossen. Für OCR-basierte Verarbeitung ist insbesondere eine ausreichende Bildauflösung relevant; Tesseract nennt hier mindestens 300 dpi als sinnvollen Richtwert. [35]
2. **Bildvorverarbeitung**
   Anschließend folgt die visuelle Bereinigung mit Binarisierung, Entfernung von Bildrauschen, Korrektur von Schräglagen (Deskewing) und Behandlung von Randartefakten. Diese Schritte verbessern die Zeichen- und Zeilensegmentierung und senken damit die Wahrscheinlichkeit systematischer OCR-Fehler. [35]
3. **OCR und Strukturerfassung**
   Nach der Bildbereinigung wird der Text extrahiert und gleichzeitig die Seitenstruktur erfasst (z. B. Positionen von Wörtern oder Blöcken). Für layoutbewusste Modelle ist diese Strukturinformation essenziell, da Textinhalte erst in Kombination mit Position und Kontext ihre volle Aussagekraft entfalten. [32] [9]
4. **Textnormalisierung und Feature-Aufbereitung**
   Auf OCR-Ebene werden üblicherweise Normalisierungsschritte wie Bereinigung von Artefaktzeichen, Vereinheitlichung von Datums-/Zahlformaten und konsistente Tokenrepräsentationen angewendet. Dadurch sinkt die Streuung technischer Schreibvarianten, und die Merkmale werden für das Klassifikationsmodell robuster. [34]
5. **Multimodale Repräsentation**
   In modernen Dokument-Pipelines werden Text-, Layout- und Bildsignale gemeinsam verarbeitet. LayoutLMv2 zeigt, dass die gemeinsame Modellierung dieser Modalitäten auf mehreren Dokumentaufgaben deutliche Leistungsgewinne gegenüber rein textuellen Ansätzen ermöglicht. [10]
6. **Training/Validierung/Test-Split**
   Vor dem Training werden die Daten reproduzierbar in Teilmengen aufgeteilt (z. B. mit fixer Zufallsinitialisierung und stratifizierter Verteilung), um Verzerrungen bei der Evaluation zu reduzieren. Für komplexe Datenstrukturen mit Gruppenzugehörigkeiten (z. B. mehrere Seiten desselben Dokuments) sind gruppenbasierte Splitter sinnvoll, damit keine Informationsleckage zwischen Train- und Testmenge entsteht. [36] [37]

3.1.5.3. Datenannotation, Balancierung und Leakage-Vermeidung

Neben der technischen Vorverarbeitung ist die semantische Qualität der Labels entscheidend. In der Dokumentklassifikation sollten Klassenregeln eindeutig dokumentiert, Label-Guidelines versioniert und Grenzfälle (z. B. Mischdokumente) konsistent behandelt werden, damit das Modell keine widersprüchlichen Lernsignale erhält. [34]

Ein weiteres Risiko ist Klassenungleichgewicht, das in realen Unternehmensdaten häufig vorkommt (z. B. viele Rechnungen, aber wenige Sonderformulare). Für robuste Evaluationen sind daher stratifizierte Splits und klassenbewusste Kennzahlen wichtig; große Referenzdatensätze wie RVL-CDIP zeigen zugleich, dass breite Klassenabdeckung ein wesentlicher Faktor für stabile Modellvergleiche ist. [30] [36]

Besondere Aufmerksamkeit erfordert Data Leakage: Wenn inhaltlich sehr ähnliche Dokumente oder Seiten desselben Falls versehentlich in verschiedenen Splits landen, werden Ergebnisse künstlich verbessert. Methodisch lässt sich das über gruppenbasierte Aufteilungen (z. B. GroupKFold/GroupShuffleSplit) vermeiden, bei denen zusammengehörige Dokumente nur einem Split zugeordnet werden. [37]

3.1.5.4. Best Practices für die Umsetzung

Für die praktische Umsetzung hat sich ein iteratives Vorgehen bewährt: Zunächst wird ein reproduzierbarer Baseline-Preprocessing-Stand definiert, danach werden einzelne Verarbeitungsschritte (z. B. Deskewing, Binarisierung, Layout-Normalisierung) kontrolliert variiert und ihr Einfluss auf Qualitätsmetriken gemessen. So wird sichtbar, welche Maßnahmen tatsächlich Modellnutzen erzeugen und welche lediglich zusätzliche Komplexität verursachen. [35] [10]

Zusammenfassend ist Datenaufbereitung in Dokumentprojekten als eigenständige Engineering-Aufgabe zu behandeln. Je systematischer Preprocessing, Annotation und Evaluationsdesign umgesetzt werden, desto zuverlässiger lassen sich Dokumenttypen automatisiert erkennen und in stabile Geschäftsprozesse integrieren. [34] [37]

3.2. Architekturen für Dokumentensysteme

Dieses Kapitel gibt einen Überblick über zentrale Architekturbausteine moderner Dokumentensysteme. Dabei werden insbesondere Such- und Filterarchitekturen sowie Rollen- und Berechtigungssysteme als wesentliche Grundlagen für den Aufbau solcher Systeme beschrieben.

3.2.1. Skalierbare plattformunabhängige Systemarchitekturen

Dieses Unterkapitel gibt einen Überblick über grundlegende Architekturansätze skalierbarer und plattformunabhängiger Softwaresysteme. Dabei werden zentrale Prinzipien der Softwarearchitektur, unterschiedliche Architekturtypen sowie die Bedeutung von Skalierbarkeit und Portabilität für moderne Dokumentensysteme beschrieben.

3.2.1.1. Grundlagen von Softwarearchitektur

Softwarearchitektur beschreibt die grundlegende Struktur eines Softwaresystems. Sie legt fest, aus welchen Komponenten ein System besteht, wie diese miteinander interagieren und welche Prinzipien der Gestaltung zugrunde liegen. Ziel der Softwarearchitektur ist es, ein System

so zu gestalten, dass es die funktionalen Anforderungen erfüllt und gleichzeitig langfristig wartbar, erweiterbar und zuverlässig bleibt. [38]

Die Aufgabe der Softwarearchitektur besteht darin, frühzeitig grundlegende Entscheidungen zu treffen, die den gesamten Entwicklungsprozess beeinflussen. Dazu gehören die Wahl geeigneter Strukturmuster, die Definition von Schnittstellen zwischen Komponenten sowie die Festlegung von Qualitätszielen wie Sicherheit, Skalierbarkeit und Plattformunabhängigkeit. [38]

Zu den grundlegenden Prinzipien der Softwarearchitektur zählen:

- **Schichtenarchitektur (Layered Architecture):** Ein System wird in übereinanderliegende Schichten unterteilt, wobei jede Schicht eine klar definierte Aufgabe übernimmt und nur mit der direkt darunterliegenden Schicht kommuniziert. Typische Schichten sind Präsentation, Geschäftslogik und Datenhaltung. Dieses Prinzip fördert die Trennung von Verantwortlichkeiten und erleichtert die Wartbarkeit des Systems. [38]
- **Komponenten und Schnittstellen:** Softwaresysteme werden in eigenständige, wiederverwendbare Einheiten (Komponenten) unterteilt, die über klar definierte Schnittstellen miteinander kommunizieren. Durch diese Kapselung bleiben interne Implementierungsdetails verborgen, was die Austauschbarkeit einzelner Komponenten ermöglicht. [38]
- **Client-Server-Prinzip:** Dieses Prinzip beschreibt die Kommunikationsstruktur zwischen zwei Rollen: Ein Client stellt Anfragen, ein Server verarbeitet diese und liefert eine Antwort zurück. Das Client-Server-Modell bildet die Grundlage für nahezu alle webbasierten Anwendungen und ist unabhängig vom internen Aufbau der Serversoftware. [39, 40]
- **Separation of Concerns:** Unterschiedliche Belange eines Systems, etwa Benutzeroberfläche, Geschäftslogik und Datenzugriff, sollen klar voneinander getrennt werden. Dies reduziert die Komplexität und erleichtert die unabhängige Weiterentwicklung einzelner Systembereiche. [38]

3.2.1.2. Architekturtypen

Auf Basis der beschriebenen Grundprinzipien haben sich in der Softwareentwicklung unterschiedliche Architekturansätze etabliert, die jeweils verschiedene Anforderungen hinsichtlich Skalierbarkeit, Wartbarkeit und Betrieb adressieren. Die Wahl des geeigneten Architekturtyps hat weitreichende Konsequenzen für den gesamten Lebenszyklus eines Systems und sollte daher frühzeitig im Entwicklungsprozess auf Basis der fachlichen und technischen Anforderungen getroffen werden. [38]

Monolithische Architektur

Die monolithische Architektur stellt den klassischen Ansatz der Softwareentwicklung dar. Bei diesem Modell werden alle Funktionsbereiche einer Anwendung, also Benutzeroberfläche, Geschäftslogik und Datenzugriff, in einer einzigen, eng gekoppelten Einheit zusammengefasst und gemeinsam entwickelt, gebaut sowie als eine Einheit deployt. [41]

Intern ist ein monolithisches System typischerweise in Schichten unterteilt, beispielsweise eine Präsentationsschicht, eine Schicht für die Geschäftslogik und eine Datenzugriffsschicht.

Diese Schichten teilen sich jedoch denselben Prozessraum und kommunizieren über direkte Funktionsaufrufe innerhalb desselben Laufzeitsystems. Skalierung erfolgt bei monolithischen Anwendungen üblicherweise durch das Klonen der gesamten Anwendung auf mehrere Server. [42]

Monolithische Architekturen eignen sich besonders für kleinere bis mittelgroße Projekte, bei denen Einfachheit und eine schnelle Markteinführung im Vordergrund stehen. Etablierte Unternehmen in den Bereichen Banking, Versicherung oder öffentliche Verwaltung betreiben bis heute umfangreiche monolithische Systeme, die über Jahrzehnte gewachsen sind und missionskritische Prozesse abwickeln. Mit zunehmender Systemgröße entstehen jedoch typische Herausforderungen: Änderungen an einer Komponente erfordern häufig Anpassungen im gesamten System, die Deploymentzyklen werden länger und die technische Wartbarkeit nimmt ab. Diese Einschränkungen haben maßgeblich zur Entwicklung alternativer Architekturansätze beigetragen. Der schematische Aufbau einer monolithischen Architektur ist in Abbildung 3.2 dargestellt. [42, 43]

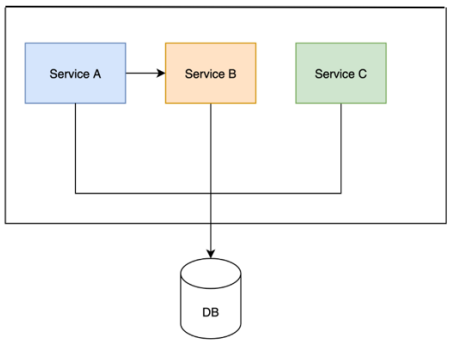


Abbildung 3.2.: Schematischer Aufbau einer monolithischen Architektur [40]

Microservices-Architektur

Die Microservices-Architektur stellt einen grundlegend anderen Ansatz dar, bei dem eine Anwendung in eine Vielzahl kleiner, unabhängiger Dienste aufgeteilt wird. Jeder dieser Dienste erfüllt eine klar abgegrenzte fachliche Aufgabe, verfügt über eine eigene Datenhaltung und kommuniziert mit anderen Diensten ausschließlich über definierte Schnittstellen, häufig in Form von REST-APIs oder nachrichtenbasierter Kommunikation. [44]

Lewis und Fowler, die den Begriff maßgeblich geprägt haben, beschreiben den Microservice-Architekturstil als einen Ansatz zur Entwicklung einer einzelnen Anwendung als Suite kleiner Dienste, von denen jeder in einem eigenen Prozess läuft und über leichtgewichtige Mechanismen kommuniziert. Jeder Dienst kann unabhängig voneinander entwickelt, getestet, deployt und skaliert werden, was Teams eine hohe Autonomie und kurze Releasezyklen ermöglicht. [45, 44]

Typische Einsatzbereiche für Microservices sind große, komplexe Systeme mit hohen Skalierungsanforderungen, wie beispielsweise E-Commerce-Plattformen, Streaming-Dienste oder Finanzanwendungen. Namhafte Unternehmen wie Amazon, Netflix und eBay haben ihre ursprünglich monolithischen Systeme auf Microservices-Architekturen migriert, um die Skalierbarkeit und Entwicklungsgeschwindigkeit zu verbessern. Dem gegenüber steht ein

erhöhter Betriebsaufwand: Die Orchestrierung, Überwachung und das Fehler-Management eines verteilten Systems stellen deutlich höhere Anforderungen an die Infrastruktur und das Betriebsteam als ein einzelner Monolith. Abbildung 3.3 zeigt den typischen Aufbau einer Microservices-Architektur. [45, 42, 46]

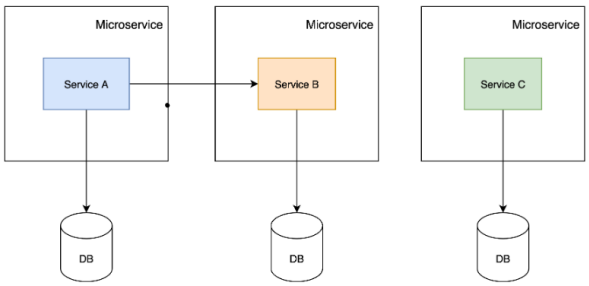


Abbildung 3.3.: Microservices-Architektur [40]

Cloud-native Architektur und SaaS

Cloud-native Anwendungen sind von Grund auf für den Betrieb in der Cloud konzipiert und nutzen deren spezifische Eigenschaften wie automatische Skalierung, verwaltete Datenbanken, globale Verfügbarkeit und integrierte Sicherheitsmechanismen. Das National Institute of Standards and Technology (NIST) definiert Cloud Computing als ein Modell, das einen allgegenwärtigen, bequemen und bedarfsgesteuerten Netzwerkzugriff auf einen gemeinsam genutzten Pool konfigurierbarer Computerressourcen ermöglicht. [47]

Eine spezifische Ausprägung dieses Ansatzes ist Software-as-a-Service (SaaS). Dabei stellt ein Anbieter eine fertige Anwendung über das Internet bereit, die von den Nutzern über einen Webbrowser oder eine Programmierschnittstelle genutzt werden kann, ohne dass diese die zugrundeliegende Infrastruktur verwalten oder kontrollieren müssen. [47] Skalierung, Wartung, Sicherheitsupdates und Verfügbarkeit werden vollständig vom Anbieter übernommen. [48, 47]

SaaS-Lösungen werden in unterschiedlichsten Bereichen eingesetzt: von Produktivitätswerkzeugen wie E-Mail- und Office-Anwendungen über CRM-Systeme und Kollaborationsplattformen bis hin zu spezialisierten Fachanwendungen in Bereichen wie Buchhaltung, Personalverwaltung oder Dokumentenmanagement. Der wesentliche Vorteil für Unternehmen liegt darin, dass keine eigene Serverinfrastruktur betrieben werden muss und die Anwendung plattformunabhängig über jeden modernen Webbrowser zugänglich ist. [47, 48]

Einordnung von SharePoint Online

Microsoft SharePoint Online ist als SaaS-Lösung in die Cloud-native Architekturkategorie einzuordnen. Die Anwendung wird vollständig von Microsoft in der Azure-Cloud betrieben und den Nutzern im Rahmen eines Microsoft-365-Abonnements bereitgestellt. Skalierung, Verfügbarkeit und Sicherheitsupdates werden automatisch durch den Anbieter gewährleistet, ohne dass eine eigene Serverinfrastruktur notwendig ist. [49]

Für den Anwendungsfall einer unternehmensweiten Dokumentenablage bietet dieser Ansatz entscheidende Vorteile: Die Plattform ist browserbasiert und damit betriebssystemunabhängig zugänglich, skaliert automatisch mit wachsendem Datenvolumen und erfüllt durch die Microsoft-Zertifizierungen hohe Sicherheitsanforderungen. [49]

3.2.1.3. Skalierbarkeit

Unter einer skalierbaren Systemarchitektur versteht man ein System, das sich problemlos an steigende Anforderungen anpassen lässt. Fehlende Skalierbarkeit führt dazu, dass Systeme bei wachsender Nutzerzahl oder zunehmendem Datenvolumen spürbare Performanceprobleme entwickeln. Während des Entwicklungsprozesses sind unterschiedliche Prinzipien zu beachten. Tabelle 3.6 fasst die zentralen Merkmale skalierbarer Softwaresysteme zusammen. [50]

Merkmal	**Beschreibung**
Modularität	Einzelne Komponenten der Software sollen unabhängig sein und sich flexibel aktualisieren oder erweitern lassen.
Flexibilität	Systeme müssen sich dynamisch an verändernde Anforderungen anpassen können.
Fehlertoleranz	Das System soll auf Fehler reagieren können und automatisch geeignete Maßnahmen ergreifen.

Tabelle 3.6.: Merkmale von Softwaresystemen [50]

Um eine skalierbare Architektur zu gewährleisten, sind Optimierungsfunktionen von Bedeutung, um die Effizienz und Leistung zu optimieren. Wichtige Optimierungsstrategien sind in Tabelle 3.7 dargestellt.

Technik	**Beschreibung**
Load Balancing	Verteilung von Anfragen auf mehrere Server, um Überlastung zu vermeiden.
Caching	Speicherung häufig abgerufener Daten, um Zugriffszeiten zu verkürzen.
Partitioning	Aufteilung von Daten in kleinere Einheiten, die parallel verarbeitet werden können.
Asynchrone Verarbeitung	Ermöglicht es Systemen, Aufgaben im Hintergrund auszuführen, was die Reaktionszeit verbessert.

Tabelle 3.7.: Techniken zur Leistungsoptimierung [50]

Ziel einer skalierbaren Architektur ist es, auch bei steigender Nutzerzahl und wachsendem Datenvolumen eine hohe Verfügbarkeit sowie eine konsistente Leistung sicherzustellen. Das System muss zudem wartbar und erweiterbar bleiben, damit zukünftige Anforderungen integriert werden können. Ohne die genannten Maßnahmen drohen bei wachsender Last messbare Einbußen in Verfügbarkeit und Antwortzeit. [50]

3.2.1.4. Plattformunabhängigkeit

Ein Computerprogramm benötigt eine Umgebung, in der es gestartet werden kann und während der gesamten Laufzeit stabil funktioniert. Ein Programm gilt als plattformunabhängig

oder plattformübergreifend, wenn es auf verschiedenen Computersystemen ausgeführt werden kann, also auf Geräten mit unterschiedlicher Hardware, verschiedenen Prozessoren oder unterschiedlichen Betriebssystemen. Der Grad dieser Unabhängigkeit wird als Portierbarkeit (oder Portabilität) bezeichnet. [51, 52]

Die Portabilität kann z. B. geschätzt werden über

$$P = 1 - \frac{U + A}{E}$$

mit

- Übertragungsaufwand U (insbesondere Neukompilierung),
- Anpassungsaufwand A (Änderung des Quellcodes, z. B. bei Austausch von Betriebsschnittstellen),
- Entwicklungsaufwand E für Neuentwicklung.

Eine Portabilität von $P = 1$ bedeutet vollständige Kompatibilität; das Programm ist also ohne Änderungen auf dem Zielsystem lauffähig, was genau dann gilt, wenn $U = A = 0$. Eine Quellcode-Portabilität liegt im Regelfall vor, wenn die Gesamt-Portabilität über 90% liegt. Dies entspricht einem Anpassungsaufwand von $A = 0$ und einem Übertragungsaufwand von $U < 0{,}1E$, da bei

$$P = 1 - \frac{U}{E} > 0{,}9$$

der Wert für U kleiner als ein Zehntel von E sein muss. Eine Portabilität nahe 0 entspricht hingegen einer nahezu vollständigen Neuentwicklung des Programms, wobei in diesem Fall $P \approx 0$ und somit $U + A \approx E$ gilt. [53, 54]

Portabilität ist kein Maß für die Lauffähigkeit eines Programms auf der Zielplattform, d. h. selbst eine Portabilität von 99 % bedeutet nicht unbedingt, dass das Programm nutzbar ist, sondern lediglich, dass eine Portierung im Vergleich zu einer Neuentwicklung deutlich weniger Aufwand erfordert. Damit ist nicht nur gemeint, dass ein Programm auf mehreren Plattformen laufen kann, sondern auch, wie viel Aufwand nötig ist, um es dafür anzupassen. Dieser Vorgang wird Portierung oder Migration genannt. [53, 51]

3.2.2. SharePoint

Dieses Unterkapitel gibt einen Überblick über SharePoint als Plattform für Dokumentenmanagement und Zusammenarbeit. Dabei werden die grundlegende Einordnung, die Architektur sowie zentrale Funktionen der Dokumentenverwaltung beschrieben.

3.2.2.1. Einordnung und Produkte

Microsoft SharePoint fungiert als webbasierte Kollaborationsplattform, die primär auf die Optimierung der teaminternen Zusammenarbeit sowie ein effizientes Informationsmanagement

ausgelegt ist. Durch die Bereitstellung zentraler Instanzen zur Inhaltsverwaltung ermöglicht die Plattform eine strukturierte Ablage und Verteilung von Daten. Ziel ist es, durch zusammenwirkende Kommunikation und medienbruchfreie Kooperation die organisatorische Produktivität zu steigern. [49, 55]

Produkt	**Kernmerkmale und Fokus**
SharePoint in Microsoft 365	Cloudbasierte Plattform (SaaS) zur zentralen Inhaltsverwaltung und teamübergreifenden Freigabe von Dokumenten. Nutzung erfolgt im Rahmen von Microsoft-365-Abonnements.
SharePoint Server	On-Premise-Lösung für den Betrieb auf eigener IT-Infrastruktur. Ermöglicht volle Kontrolle über Datenhoheit und unterstützt hybride Szenarien mit der Cloud.
SharePoint Designer 2013	Spezialisiertes Werkzeug zur Erstellung deklarativer Workflows und zur Modellierung komplexer Geschäftsprozesse innerhalb der SharePoint-Umgebung.
OneDrive	Schnittstelle zur Synchronisation von SharePoint-Bibliotheken mit lokalen Endgeräten. Ermöglicht die Offline-Bearbeitung und Dateiverwaltung im lokalen Dateisystem.

Tabelle 3.8.: Überblick über SharePoint-Produkte [49]

3.2.2.2. Architektur und Hierarchie

Die Struktur von SharePoint ist hierarchisch aufgebaut, um Inhalte logisch zu trennen und den Zugriff gezielt zu steuern. Den Kern dieser Architektur bildet die sogenannte Websitesammlung (Site Collection). [56, 55]

Eine Websitesammlung besteht immer aus einer Website auf der obersten Ebene und allen darunterliegenden Websites. Sie dient als übergeordneter Container, in dem zentrale Einstellungen für die Sicherheit, das Design und verschiedene Funktionen festgelegt werden. Technisch gesehen landen alle Daten einer solchen Sammlung in einer einzigen Inhaltsdatenbank. Zwar kann eine Datenbank mehrere Websitesammlungen speichern, aber eine Sammlung kann nicht über mehrere Datenbanken verteilt werden. [56]

Bei der Planung der Struktur gibt Microsoft die Empfehlung ab, für jede Arbeitseinheit eine eigene Websitesammlung anzulegen, anstatt eine komplexe Verschachtelung mit vielen Unterwebsites zu bauen. Das hat den Vorteil, dass die Umgebung übersichtlich bleibt und später einfacher migriert werden kann. [56]

Innerhalb einer solchen Sammlung können Ressourcen wie Bilder, Vorlagen oder bestimmte Spaltentypen gemeinsam genutzt werden. Das sorgt dafür, dass die Navigation und das Erscheinungsbild für die Nutzer einheitlich bleiben. Bei der Benennung der URLs wird meistens auf pfadbasierte Adressen (wie z. B. „/sites/Projektname") gesetzt, da diese mit Tools wie der PowerShell am einfachsten zu verwalten sind. [56]

3.2.2.3. Funktionale Elemente einer SharePoint-Website

Innerhalb einer Websitesammlung bestehen die einzelnen Websites aus verschiedenen funktionalen Elementen, die für die Organisation und Verwaltung der Daten zuständig sind:

- **Dokumentbibliotheken (Document Libraries):** Diese stellen den primären Speicherort für Dateien dar. Im Gegensatz zu herkömmlichen Dateiordnern ermöglichen sie die Nutzung von erweiterten Funktionen wie Versionierung, Check-Out-Mechanismen und die Verwendung von Metadaten. [57, 55]
- **Listen (Lists):** Listen dienen der Erfassung strukturierter Daten, ähnlich einer Tabelle. Sie werden beispielsweise für Aufgabenlisten, Kontaktverzeichnisse oder Inventarlisten verwendet. [57]
- **Seiten (Pages):** Diese bilden das visuelle Interface der Website. Informationen werden hier mithilfe von sogenannten Webparts (funktionalen Bausteinen) für den Endanwender aufbereitet. [57]
- **Metadaten (Spalten):** Anstatt Dateien ausschließlich in starren Ordnerstrukturen zu sortieren, erlauben Spalten das Hinzufügen von Attributen wie „Dokumententyp" oder „Status". Dies verbessert die Filterbarkeit und Auffindbarkeit der Dokumente innerhalb der gemeinsamen Ablage erheblich. [57]

3.2.2.4. Kernfunktionen der Dokumentenverwaltung

SharePoint bietet eine Vielzahl von Funktionen, die speziell auf die Anforderungen der Dokumentenverwaltung zugeschnitten sind:

- **Versionierung:** Jede Änderung an einem Dokument wird als separate Version gespeichert. Dadurch können Nutzer frühere Versionen wiederherstellen oder Änderungen nachverfolgen. [57, 55]
- **Auschecken und Einchecken (Check-Out/In):** Um Bearbeitungskonflikte zu vermeiden, können Dokumente exklusiv für einen Nutzer gesperrt werden. Während eine Datei ausgecheckt ist, können andere Nutzer diese zwar lesen, aber keine Änderungen vornehmen, bis sie wieder eingecheckt wird. [57]
- **Gemeinsame Dokumenterstellung (Co-Authoring):** Diese Funktion erlaubt es mehreren Anwendern, zeitgleich an demselben Dokument (z. B. Word oder Excel) zu arbeiten. Änderungen werden in Echtzeit synchronisiert, was die kollaborative Zusammenarbeit beschleunigt. [57]

3.2.3. Rollen- und Berechtigungssysteme

Dieses Unterkapitel gibt einen Überblick über die Grundlagen von Rollen- und Berechtigungssystemen in IT-Anwendungen. Dabei werden der Aufbau einzelner Berechtigungen, zentrale Berechtigungsmodelle sowie typische Berechtigungsstufen beschrieben, die für den kontrollierten Zugriff auf Dokumente und Funktionen relevant sind.

3.2.3.1. Aufbau einer Berechtigung

Eine Berechtigung setzt sich grundsätzlich aus zwei zentralen Komponenten zusammen, die gemeinsam definieren, welcher Zugriff in einem System erlaubt oder untersagt ist.

Komponente	**Beschreibung**
Zu schützende Ressource	Beschreibt das Objekt, auf das sich eine Berechtigung bezieht. Dabei kann zwischen funktionalen und inhaltsbezogenen Ressourcen unterschieden werden. Die Definition der Ressource kann auf unterschiedlichen Granularitätsebenen erfolgen, beispielsweise global für ein gesamtes System oder atomar für einzelne Systembestandteile.
Zugehörige Operation auf der Ressource	Legt fest, welche konkrete Handlung in Bezug auf die definierte Ressource durch eine Berechtigung erlaubt oder eingeschränkt wird. Die Operation bestimmt, welche Aktionen eine Identität auf der jeweiligen Ressource ausführen darf.

Tabelle 3.9.: Komponenten einer Berechtigung [58]

3.2.3.2. Arten von Berechtigungen

Berechtigungskonzepte bilden eine wesentliche Grundlage für die Zugriffskontrolle in IT-Systemen. Die elementarste Form stellt dabei die binäre Berechtigung dar. Sie beschreibt eine eindeutig definierte Zugriffserlaubnis, die sich jeweils auf ein konkretes Objekt und eine einzelne Operation bezieht. Der Zugriff ist entweder erlaubt oder verweigert, wodurch diese Berechtigungsform ausschließlich zwei mögliche Zustände kennt. [58]

Aufgrund dieser klaren Struktur eignen sich binäre Berechtigungen besonders für die technische Überprüfung von Zugriffsrechten. Umfangreichere Berechtigungsmodelle, die mehrere Objekte oder Operationen umfassen, werden vor der eigentlichen Zugriffskontrolle in einzelne binäre Berechtigungen zerlegt. Die endgültige Zugriffsentscheidung erfolgt anschließend ausschließlich auf Basis dieser elementaren Berechtigungseinheiten. [58, 59]

Neben binären Berechtigungen existieren sogenannte Wertberechtigungen, bei denen mehrere Einzelrechte in einem numerischen Wert zusammengefasst werden. Ein verbreitetes Anwendungsbeispiel hierfür ist das Berechtigungssystem von UNIX-Dateisystemen. Dort repräsentiert ein Zahlenwert die Kombination einzelner Zugriffsrechte wie Lesen, Schreiben und Ausführen. Der Wert 7 steht für den vollständigen Zugriff und ergibt sich aus der Addition der einzelnen Rechtewerte. [58]

3.2.3.3. Berechtigungsmodelle

In der Praxis werden Berechtigungen nicht als Einzelrechte verwaltet, sondern in strukturierten Modellen organisiert. Die wichtigsten Ansätze sind das rollenbasierte sowie das attributba-

sierte Zugriffsmodell, die sich hinsichtlich ihrer Flexibilität und ihres Verwaltungsaufwands grundlegend unterscheiden. [60, 61]

Role-Based Access Control (RBAC)

Das Modell der rollenbasierten Zugriffskontrolle (RBAC) wurde 1992 von Ferraiolo und Kuhn als Alternative zu den bis dahin dominierenden Modellen der Mandatory Access Control (MAC) und Discretionary Access Control (DAC) eingeführt. Das Grundprinzip von RBAC besteht darin, Berechtigungen nicht direkt an einzelne Benutzer zu vergeben, sondern an Rollen – abstrakte Funktionseinheiten, die typischen Aufgabenprofilen in einer Organisation entsprechen. Benutzer werden diesen Rollen zugewiesen und erben damit automatisch alle zugehörigen Zugriffsrechte. [59, 60]

Sandhu et al. greifen dieses Konzept im RBAC96-Modell auf und strukturieren es in vier Referenzmodelle mit steigender Komplexität: von einem einfachen Flat-RBAC ohne Hierarchien bis hin zu einem Modell mit Rollenhierarchien und Aufgabentrennung (Separation of Duty). Diese Hierarchisierung ermöglicht es, übergeordneten Rollen automatisch die Rechte untergeordneter Rollen zu vererben, was den Verwaltungsaufwand in großen Organisationen erheblich reduziert. [60, 59]

Für Dokumentensysteme wie SharePoint ist RBAC das zentrale Steuerungsmodell: Vordefinierte Rollen wie Besitzer, Mitglied oder Besucher bündeln jeweils spezifische Berechtigungsstufen für Dokumentbibliotheken und Websites. Entra ID-Gruppen können diesen Rollen direkt zugewiesen werden, sodass die Verwaltung organisationsweit an zentraler Stelle erfolgt. [58, 60]

Attribute-Based Access Control (ABAC)

Während RBAC auf feste Rollenzuweisungen setzt, ermöglicht das Modell der attributbasierten Zugriffskontrolle (ABAC) eine deutlich feinkörnigere Steuerung. Laut NIST Special Publication 800-162 wird bei ABAC die Zugriffserlaubnis anhand von Attributen des Subjekts, des Objekts sowie der Umgebungsbedingungen bewertet – nicht allein anhand der Rollenmitgliedschaft. [61]

Typische Attribute sind etwa die Abteilungszugehörigkeit eines Nutzers, die Vertraulichkeitsstufe eines Dokuments oder der Standort, von dem der Zugriff erfolgt. ABAC eignet sich damit besonders für Szenarien, in denen kontextabhängige Zugriffsregeln notwendig sind, die mit starren Rollen nicht abbildbar wären – beispielsweise der Zugriff auf ein Dokument nur dann, wenn der Nutzer der zuständigen Abteilung angehört und das Dokument noch nicht freigegeben ist. [61, 60]

In der Praxis werden RBAC und ABAC häufig kombiniert: Rollen liefern die Grundstruktur der Zugriffsverwaltung, während Attribute zusätzliche Einschränkungen oder Erweiterungen ermöglichen. Diese Kombination bietet sowohl Verwaltbarkeit als auch Flexibilität. [61, 59]

3.2.3.4. Scope von Berechtigungen

Ein zentrales Kriterium bei der Konzeption und Platzierung von Berechtigungen ist die Festlegung, welche Zugriffsrechte durch eine Berechtigung gewährt werden sollen. In diesem Zusammenhang kommt dem Grad der Differenzierung von Berechtigungen, der als Granularität bezeichnet wird, eine besondere Bedeutung zu. [58]

Die Grundlage für die Ausgestaltung von Berechtigungen bildet der Schutzbedarf der jeweiligen Ressource. Dieser wird üblicherweise im Rahmen einer Schutzbedarfsanalyse ermittelt, bei der unterschiedliche Einflussfaktoren berücksichtigt werden. Dazu zählen unter anderem die Relevanz der Ressource für Geschäftsprozesse sowie die Kritikalität ihrer Funktionen und Inhalte. [58, 61]

Die ermittelten Schutzanforderungen fließen anschließend in die Berechtigungsplanung ein. Dabei wird festgelegt, auf welcher Detailebene der Zugriffsschutz umgesetzt werden muss und welche Funktionen durch entsprechende Berechtigungen abzusichern sind. Darüber hinaus wird bestimmt, wie stark der Zugriffsschutz ausgeprägt sein soll, beispielsweise ob der Einsatz erweiterter Authentifizierungsmechanismen erforderlich ist. [58]

3.2.3.5. Berechtigungsstufen

Anstatt jedem Nutzer individuelle Einzelrechte zuzuweisen, werden Berechtigungen in der Praxis häufig in abgestuften Niveaus zusammengefasst. Dies vereinfacht die Verwaltung erheblich, da einem Nutzer lediglich eine Stufe zugeordnet werden muss, die ein definiertes Bündel von Zugriffsrechten repräsentiert. Die folgende Einteilung orientiert sich an dem in der Literatur beschriebenen Prinzip der gestuften Zugriffsrechte und findet sich in vergleichbarer Form in vielen unternehmensweiten Systemen wieder: [58, 60]

- **Stufe 0: Kein Zugriff**
  (Gar keine Rechte)
- **Stufe 1: Nur lesen**
  (Informationen ansehen)
- **Stufe 2: Lesen und Hinzufügen**
  (Daten ansehen und neue erstellen)
- **Stufe 3: Ändern und Löschen**
  (Bestehende Daten bearbeiten)
- **Stufe 4: Vollzugriff**
  (Alle Rechte, einschließlich Verwaltung)

In SharePoint entspricht dieses Stufenmodell den vordefinierten Berechtigungsstufen Lesen, Beitragen, Bearbeiten und Vollzugriff, die direkt auf RBAC-Rollen abgebildet werden. Damit schließt das Berechtigungsmodell von SharePoint unmittelbar an die theoretischen Grundlagen von RBAC an und macht deutlich, warum die Wahl eines strukturierten Rollenmodells gegenüber einer individuellen Rechtevergabe in der Unternehmenspraxis klar bevorzugt wird. [58, 59]

3.2.4. Microsoft Entra ID

Dieses Unterkapitel gibt einen Überblick über Microsoft Entra ID als zentrales System für Identitäts- und Zugriffsverwaltung. Dabei werden die Einordnung, zentrale Verwaltungsfunktionen sowie Authentifizierungs- und Integrationsmöglichkeiten beschrieben.

3.2.4.1. Begriff und Einordnung

Microsoft Entra beschreibt eine umfassende Produktfamilie für Identitätsmanagement und Netzwerkzugriff, die darauf ausgelegt ist, eine moderne Zero Trust-Sicherheitsstrategie in Organisationen zu etablieren. Das Ziel dieser Architektur ist die Schaffung einer Vertrauensstruktur, die Identitäten sowie Zugriffsbedingungen und Berechtigungen konsequent überprüft, Verbindungskanäle verschlüsselt und eine kontinuierliche Überwachung auf Kompromittierungen ermöglicht. Die hierarchische Struktur und die verschiedenen Säulen dieser Architektur sind in Abbildung 3.4 dargestellt. [62, 63]

Das zentrale Kernprodukt dieser Familie ist Microsoft Entra ID. Dabei handelt es sich um einen cloudbasierten Dienst zur Identitäts- und Zugriffsverwaltung, der die grundlegenden Funktionen für Authentifizierung und den Schutz von Benutzern, Geräten sowie Anwendungen bereitstellt. Eine wesentliche Besonderheit für Unternehmen im Microsoft-Umfeld besteht darin, dass Abonnenten von Diensten wie Microsoft 365 oder Azure automatisch einen Microsoft Entra-Mandanten nutzen und somit direkt mit der Verwaltung ihrer Cloud-Anwendungen beginnen können. Über das webbasierte Microsoft Entra Admin Center lassen sich diese Produkte zentral über eine einzige Benutzeroberfläche konfigurieren und verwalten. [62]

Abbildung 3.4.: Übersicht der Microsoft Entra Produktfamilie [62]

3.2.4.2. Zentrale Identitätsverwaltung

Die zentrale Verwaltung innerhalb von Microsoft Entra sorgt dafür, dass Identitäten für unterschiedliche Nutzergruppen und Ressourcen an einem einzigen Ort kontrolliert werden. Dies umfasst nicht nur menschliche Identitäten wie Mitarbeitende, Partner oder Kundschaft, sondern schließt auch Geräte sowie sogenannte Workload-Identitäten (z. B. Anwendungen und Dienste) mit ein. [62, 58]

Ein wesentlicher Bestandteil dieser Verwaltung ist die Microsoft Entra ID Governance. Diese ermöglicht es, den gesamten Lebenszyklus einer Identität zu automatisieren. So können

beispielsweise Zugriffsanfragen, Zuweisungen von Lizenzen und regelmäßige Überprüfungen von Berechtigungen systemgesteuert erfolgen. Dies stellt sicher, dass neuen Mitarbeitern automatisch die benötigten Ressourcen zugewiesen werden und diese Zugänge beim Verlassen des Unternehmens ebenso zuverlässig wieder entfernt werden. [62, 58]

Für die Verwaltung externer Nutzer bietet Microsoft Entra External ID eine sichere Methode zur Zusammenarbeit. Hiermit können Geschäftspartner oder Gäste gezielt Zugriff auf interne Ressourcen erhalten, während Kunden über Self-Service-Registrierungen (z. B. via Google- oder Facebook-Konten) angebunden werden können. [62]

3.2.4.3. Authentifizierung und Autorisierung

Innerhalb von Microsoft Entra ID bilden Authentifizierung und Autorisierung die Grundlage für den sicheren Zugriff auf Ressourcen, folgen jedoch unterschiedlichen Prinzipien:

- **Authentifizierung (Identity):** Dies ist der Prozess der Identitätsprüfung. Hierbei stellt das System sicher, dass ein Benutzer tatsächlich die Person ist, für die er sich ausgibt. Dies geschieht in der Regel durch die Abfrage von Anmeldedaten oder sicheren Methoden wie der Multi-Faktor-Authentifizierung (MFA). Microsoft Entra ID Protection unterstützt diesen Vorgang, indem es risikobasierte Richtlinien nutzt, um verdächtige Anmeldeversuche automatisch zu erkennen und zusätzliche Bestätigungen einzufordern. [62, 64]
- **Autorisierung (Permissions):** Nach einer erfolgreichen Anmeldung bestimmt die Autorisierung, welche spezifischen Zugriffsrechte der Benutzer besitzt. Es wird festgelegt, auf welche Anwendungen, Dateien oder Funktionen (z. B. innerhalb einer SharePoint-Website) zugegriffen werden darf. [62]

3.2.4.4. Single Sign-On (SSO)

Ein zentrales Leistungsmerkmal von Microsoft Entra ID ist das sogenannte Single Sign-On. Diese Funktion ermöglicht es Benutzern, sich mit nur einer einzigen Identität – bestehend aus einem Benutzernamen und einem Kennwort – einmalig anzumelden, um danach automatisch Zugriff auf alle für sie freigegebenen Cloud-Anwendungen sowie lokalen Ressourcen zu erhalten. [62, 64]

Aus technischer Sicht bietet SSO zwei wesentliche Vorteile für das Unternehmen:

- **Benutzerfreundlichkeit:** Da sich die Anwender nicht mehr für jede einzelne Anwendung (wie SharePoint, Teams oder externe Drittanbieter-Tools) unterschiedliche Zugangsdaten merken müssen, wird die tägliche Arbeit effizienter. Zudem sinkt die Anzahl an Support-Anfragen aufgrund vergessener Passwörter erheblich. [62]
- **Sicherheit:** Da die Identität zentral an einer Stelle geprüft wird, können Sicherheitsrichtlinien, wie beispielsweise die Mehrfaktor-Authentifizierung (MFA), global für alle angebundenen Programme erzwungen werden. Zudem wird das Risiko minimiert, dass Nutzer unsichere Passwörter für mehrere Dienste gleichzeitig verwenden oder diese ungesichert notieren. [62, 58]

3.2.4.5. Integration externer Anwendungen

Ein wesentlicher Vorteil von Microsoft Entra ID ist die Fähigkeit, über die Grenzen der Microsoft-Welt hinaus als zentrale Identitätsinstanz zu fungieren. Dies wird durch die Integration externer Anwendungen (Third-Party-Apps) realisiert:

- **SaaS-Anwendungen:** Über vorkonfigurierte Schnittstellen lassen sich gängige Cloud-Dienste (wie Salesforce, Slack oder Dropbox) nahtlos anbinden. Dadurch profitieren auch diese externen Dienste von Sicherheitsfeatures wie Single Sign-On (SSO) und der Mehrfaktor-Authentifizierung. [62, 58]
- **Zentrale App-Verwaltung:** Administratoren können im Entra Admin Center exakt steuern, welche Benutzer oder Gruppen Zugriff auf bestimmte externe Software erhalten. Dies vereinfacht das Onboarding neuer Mitarbeiter, da Zugänge für verschiedene Plattformen an einer zentralen Stelle zugewiesen werden können. [62]
- **Sicherer Internet- und Privatzugriff:** Moderne Komponenten wie der Microsoft Entra Internet Access und Private Access erweitern diesen Schutz. Sie ermöglichen einen sicheren Zugriff auf Internet-Ressourcen sowie interne Unternehmensnetzwerke, ohne dass klassische, oft unsicherere VPN-Verbindungen notwendig sind. [62]

Die zentrale Rolle von Microsoft Entra ID als Vermittler zwischen lokalen Infrastrukturen, Cloud-Applikationen und externen Identitäten wird in Abbildung 3.5 verdeutlicht. Durch diese weitreichende Integrationsfähigkeit wird Microsoft Entra ID zum zentralen Kontrollpunkt für die gesamte IT-Infrastruktur einer Organisation, was die Sicherheit erhöht und die Komplexität für die Endanwender reduziert. [62]

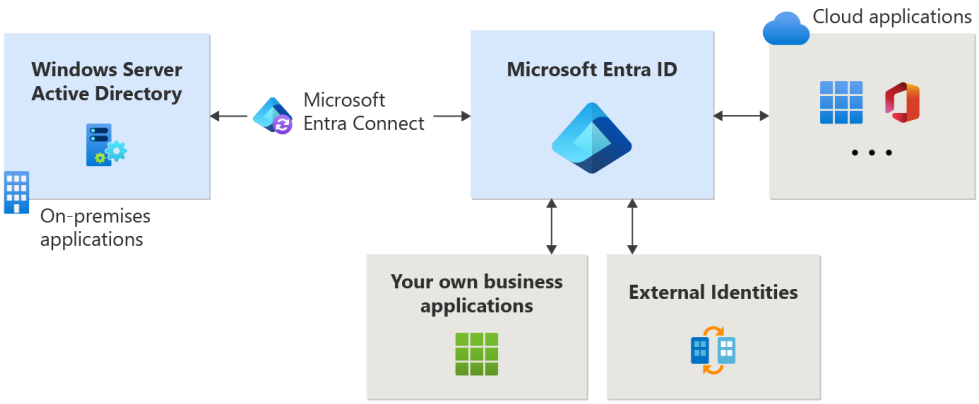


Abbildung 3.5.: Integration von On-Premises- und Cloud-Anwendungen über Microsoft Entra ID [62]

3.2.5. Suche und Filter-Architekturen

Dieses Unterkapitel gibt einen Überblick über zentrale Such- und Filterarchitekturen in modernen Dokumentensystemen. Dabei werden lexikalische, semantische und hybride Suchverfahren sowie die Rolle von Filtern und Facetten für eine präzise und skalierbare Informationssuche beschrieben.

3.2.5.1. Suchalgorithmen im Information Retrieval

Suche und Filter bilden in Dokumentensystemen eine gemeinsame Architekturaufgabe: Die Suche identifiziert relevante Kandidaten, während Filter die Ergebnisliste auf fachlich gültige Teilmengen einschränken. Im Information Retrieval (IR) hat sich dafür eine mehrstufige Verarbeitung etabliert, die Abfrageinterpretation, Kandidatenfindung, Ranking und nachgelagerte Einschränkungen kombiniert. Für unternehmensweite Dokumentplattformen ist diese Trennung besonders wichtig, weil dieselbe Anfrage häufig sowohl unstrukturierte Textfelder als auch strukturierte Metadaten (z. B. Dokumenttyp, Zeitraum, Abteilung) adressiert. Eine belastbare Sucharchitektur muss daher Ranking- und Filterlogik technisch klar trennen, aber in einer gemeinsamen Query-Pipeline ausführbar machen. [65, 1]

3.2.5.2. Lexikalische Suchverfahren (Keyword Search)

Die lexikalische Suche ist der klassische IR-Ansatz und basiert auf dem Abgleich von Termen zwischen Query und Dokumentbestand. Sie ist besonders stark bei exakten Treffern, etwa bei IDs, normierten Begriffen, Produktcodes oder eindeutig benannten Entitäten. [1]

Inverted Index und TF-IDF

Die technische Basis lexikalischer Systeme ist der Inverted Index: Für jeden Term wird eine Postings-Liste der zugehörigen Dokumente geführt, sodass Anfragen nicht über alle Dokumente sequentiell laufen müssen. Diese Datenstruktur ist in IR-Systemen der Standard, weil sie Suchanfragen in großen Korpora effizient auswertbar macht. Auf Ranking-Ebene wird häufig mit termbasierten Gewichtungen gearbeitet. Ein klassisches Schema ist TF-IDF, bei dem ein Term höher gewichtet wird, wenn er in einem Dokument häufig vorkommt, aber im Gesamtkorpus vergleichsweise selten ist. Dadurch erhalten diskriminierende Begriffe mehr Einfluss auf die Relevanzbewertung als sehr häufige, wenig informative Terme. [21, 1]

BM25

In praktischen Suchsystemen ist BM25 der verbreitete Standard für lexikalisches Ranking. Das Modell erweitert den reinen TF-IDF-Gedanken um Termfrequenz-Sättigung und Längennormalisierung und reduziert damit typische Verzerrungen zugunsten sehr langer Dokumente. Dass BM25 nicht nur theoretisch, sondern auch produktiv relevant ist, zeigt sich in verbreiteten Suchbibliotheken: Lucene führt BM25 als zentrale Similarity-Variante und erlaubt zusätzlich die Parametrisierung über k_1 und b für unterschiedliche Korpuscharakteristika. [66, 67]

3.2.5.3. Semantische Suchverfahren (Vector Search)

Semantische Suchverfahren adressieren eine zentrale Schwäche rein lexikalischer Verfahren: relevante Treffer enthalten häufig nicht exakt dieselben Terme wie die Suchanfrage, sind aber inhaltlich nahe verwandt. Daher werden Query und Dokumente in einen gemeinsamen Vektorraum überführt und über Distanz- bzw. Ähnlichkeitsmaße verglichen. Embeddings werden in dieser Arbeit bereits in Abschnitt 3.1.3 eingeführt und dort detailliert erklärt. Für die Sucharchitektur ist hier nur relevant: Die Qualität der semantischen Suche hängt

davon ab, dass Query- und Dokumentvektoren mit kompatiblen Modellen und identischen Dimensionsannahmen erzeugt werden. [68]

Suche mittels k-NN und HNSW

Das grundlegende Suchproblem in Vektorräumen ist k-NN (k-nearest neighbors): Gesucht werden die k nächsten Dokumentvektoren zur Query. Da exakte k-NN-Suche bei großen Datenmengen teuer wird, setzen produktive Systeme meist auf ANN-Methoden (Approximate Nearest Neighbor), die Rechenzeit gegen eine kontrollierte Recall-Abweichung tauschen.Ein verbreiteter ANN-Ansatz ist HNSW (Hierarchical Navigable Small World). Der Algorithmus organisiert Vektoren in einer hierarchischen Navigationsstruktur und erreicht dadurch hohe Suchgeschwindigkeit bei gleichzeitig guter Trefferqualität, was ihn für große Dokumentindizes praktikabel macht. [69, 68]

3.2.5.4. Hybride Suchansätze (Hybrid Search)

In realen Dokumentanwendungen deckt weder ein rein lexikalischer noch ein rein semantischer Retriever alle Abfragetypen optimal ab. Hybrid Search kombiniert deshalb beide Signale in einer gemeinsamen Retrieval-Pipeline und erhöht so die Robustheit gegenüber unterschiedlichen Query-Formen. Eine verbreitete Fusionsstrategie ist Reciprocal Rank Fusion (RRF). RRF kombiniert mehrere Ergebnislisten rein rangbasiert und benötigt keine aufwändige Score-Normalisierung zwischen unterschiedlichen Retrievalmethoden, was die praktische Integration deutlich vereinfacht. [70, 68]

3.2.5.5. Filter- und Facettenarchitekturen

Filter sind kein Ersatz für Ranking, sondern eine komplementäre Architekturkomponente. In Elasticsearch wird diese Trennung explizit als Query Context (scored) versus Filter Context (binär, nicht-scorebasiert) modelliert; insbesondere `bool.filter`-Klauseln werden ohne Relevanzbewertung ausgeführt und für wiederholte Nutzung gecacht. [65, 71]

Für fachliche Navigation in großen Trefferlisten werden zusätzlich Facetten bzw. Aggregationen eingesetzt. Typische Muster sind Terms-Buckets (z. B. Dokumenttyp, Abteilung) und zeitliche Buckets (Date Histogram), die eine schrittweise Eingrenzung der Suchergebnisse ermöglichen. [72, 73, 74]

Dass diese Logik plattformübergreifend etabliert ist, zeigt auch der Solr-Ansatz: Faceting wird dort als Kategorisierung von Suchresultaten über indizierte Terme beschrieben und als zentrales Interaktionsmuster für explorative Suche eingesetzt. [75]

Für die Systemarchitektur ergibt sich daraus ein stabiler Aufbau mit klaren Verantwortlichkeiten:

1. **Retriever-Schicht** für lexikalische und/oder semantische Kandidatensuche, [68]
2. **Ranking-/Fusion-Schicht** für die Ergebniszusammenführung (z. B. RRF), [70]
3. **Filter-/Facet-Schicht** für fachliche Constraints und UI-gestützte Eingrenzung. [65, 73]

Diese Schichtung verbessert Wartbarkeit und Skalierbarkeit, weil Relevanzmodelle, Filterregeln und Navigationslogik unabhängig weiterentwickelt werden können. Gerade in Dokumentensystemen mit häufig wechselnden Metadatenanforderungen ist diese Entkopplung ein zentraler Qualitätsfaktor. [65]

3.2.5.6. Vergleich der Verfahren

Die folgende Tabelle 3.10 stellt die in der Praxis wichtigsten Unterschiede zwischen lexikalischer, semantischer und hybrider Suche gegenüber.

Kriterium	**Lexikalische Suche**	**Semantische Suche**	**Hybride Suche**
Primäres Signal	Termübereinstimmung (z. B. BM25)	Vektorähnlichkeit (k-NN/ANN)	Kombination beider Signale
Stärken	Exakte Treffer bei IDs und Fachbegriffen	Robust bei Synonymen und paraphrasierten Anfragen	Höhere Robustheit über gemischte Query-Typen
Grenzen	Schwach bei semantischer Varianz	Abhängig von Embedding-Qualität und Indexparametern	Höherer Integrations- und Betriebsaufwand

Tabelle 3.10.: Vergleich von Suchansätzen in Dokumentsystemen [67, 68, 70]

4. Implementierung

4.1. Grundlagen

Dieses Kapitel beschreibt die grundlegenden technischen Rahmenbedingungen der umgesetzten Lösung. Dabei werden die Systemarchitektur sowie die verwendeten Technologien als Basis für die nachfolgenden Implementierungs- und Funktionsbeschreibungen vorgestellt.

4.1.1. Systemarchitektur

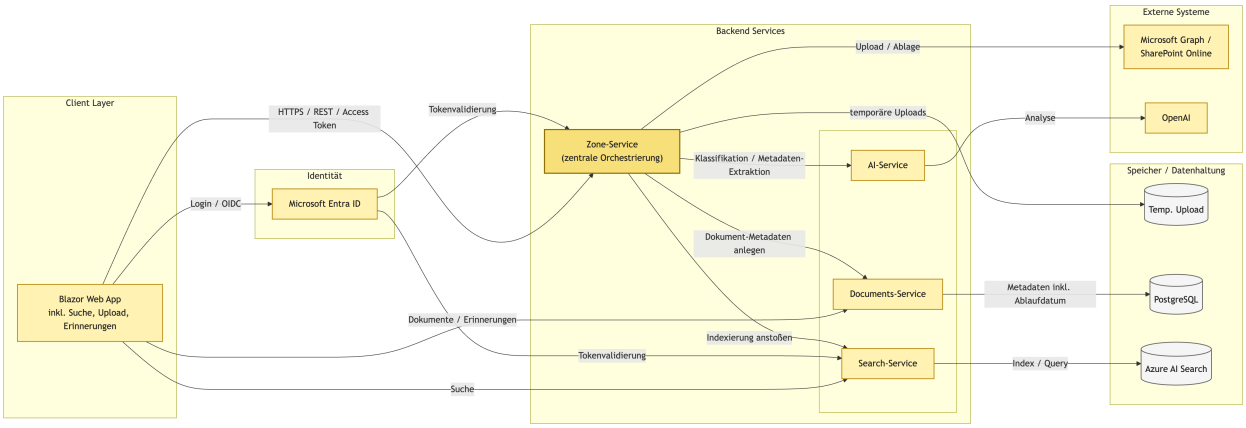


Abbildung 4.1.: Übersicht der Systemarchitektur

Die dargestellte Systemarchitektur in Abbildung 4.1 ist serviceorientiert aufgebaut und trennt Benutzeroberfläche, Identitätsverwaltung, Fachlogik und externe Systeme voneinander. Die Blazor Web App dient als Client und kommuniziert über REST-Schnittstellen mit den Backend-Services, wobei die Authentifizierung und Tokenbereitstellung über Microsoft Entra ID erfolgt.

Der Zone-Service übernimmt die zentrale Orchestrierung der Geschäftslogik. Er steuert den Upload-Prozess, die KI-gestützte Klassifikation über den AI-Service, die Speicherung von Metadaten im Documents-Service sowie die Indexierung im Search-Service. Die eigentliche Dokumentablage erfolgt in SharePoint Online, während PostgreSQL für strukturierte Metadaten und Azure AI Search für Suchanfragen verwendet wird. Dadurch entsteht eine klar getrennte und erweiterbare Systemarchitektur.

4.1.2. Technologien

Dieses Unterkapitel gibt einen Überblick über die zentralen Technologien, die für die Umsetzung des Systems verwendet wurden. Dabei werden insbesondere Backend, Datenhaltung, Suchtechnologie sowie Frontend-nahe Technologien kurz eingeordnet.

Backend und Kommunikation

Die Backend-Implementierung erfolgte in der Programmiersprache **C#**. Zur Kommunikation zwischen Client und Server wird eine **REST-basierte Schnittstelle (REST-API)** verwendet. Diese ermöglicht eine standardisierte Datenübertragung über **HTTP** und sorgt für eine klare Trennung zwischen Präsentationsschicht und Anwendungslogik.

Datenhaltung

Zur persistenten Speicherung der Daten kommt das relationale Datenbankmanagementsystem **PostgreSQL** zum Einsatz. PostgreSQL bietet eine hohe Stabilität, umfangreiche SQL-Funktionalitäten sowie eine gute Skalierbarkeit und eignet sich daher optimal für datenbankgestützte Webanwendungen.

Suchtechnologie

Für die Dokumentensuche wird **Azure AI Search** eingesetzt. Dieser Dienst ermöglicht die Indizierung großer Dokumentbestände sowie eine performante Volltextsuche mit Filter- und Facettenfunktionen. Dadurch können Dokumente anhand von Suchbegriffen und strukturierten Metadaten effizient aufgefunden werden.

Microsoft Entra ID

Für die Benutzeranmeldung und Zugriffsverwaltung wird **Microsoft Entra ID** verwendet. Der Dienst übernimmt die Authentifizierung der Benutzer und stellt den Benutzerkontext für geschützte Zugriffe auf die Anwendung und angebundene Microsoft-Dienste wie SharePoint bereit.

Die registrierte Anwendung **DropIT.Core** fungiert dabei als zentrale Client-App der Webanwendung und fordert Zugriffstoken für die getrennt registrierten Dienste **DropIT.Zones.Api**, **DropIT.Documents.Api** und **DropIT.Search.Api** an. **DropIT.Zones.Api** wird für die Zonenverwaltung sowie für Zugriffe auf SharePoint verwendet, **DropIT.Documents.Api** für die Verwaltung der Dokument-Metadaten und **DropIT.Search.Api** für die abgesicherte Suchfunktion. Die Aufteilung auf mehrere App-Registrierungen ermöglicht es, Berechtigungen pro Dienst getrennt zu vergeben und die einzelnen Systemteile klar voneinander abzugrenzen.

Die in Microsoft Entra registrierten Anwendungen des Systems sind in Abbildung 4.2 dargestellt.

Frontend und Design

Das Frontend wurde mit einem **Blazor WebAssembly** realisiert. Dieses Framework ermöglicht die Entwicklung interaktiver Webanwendungen unter Verwendung von C#, wobei der

Anzeigename	Anwendungs-ID (Client)	Erstellt am	Zertifikate & Geheim...
DropIT.Core	cb2832c8-ca97-4e37-a12a-dffe166c9e25	31.12.2025	Aktuell
DropIT.Documents.Api	0d1c54e7-41ab-4428-a708-cf7563965670	31.12.2025	Aktuell
DropIt.Search.Api	a29f9a8e-7ba1-4403-ba57-1b7a7d76e1fc	2.1.2026	Aktuell
DropIT.Zones.Api	e3eeadea-af53-444f-bbd6-5eadfa59be9f	31.12.2025	Aktuell

Abbildung 4.2.: In Microsoft Entra registrierte Anwendungen von DropIT

Anwendungscode direkt im Browser des Clients ausgeführt wird. Dadurch kann ein Teil der Verarbeitungslogik clientseitig erfolgen, was zu einer verbesserten Benutzerinteraktion führt.

- **CSS:** Das Layout und Design der Benutzeroberfläche wird mithilfe von Cascading Style Sheets (CSS) umgesetzt. Dadurch können Struktur, Darstellung und Responsivität flexibel gestaltet werden.
- **JavaScript:** Zusätzlich wird JavaScript für spezifische browserseitige Funktionen verwendet. Insbesondere kommt es beim Upload von Dokumenten zum Einsatz, um browsernahe Funktionen effizient zu nutzen und die Interaktion mit Dateisystemen zu ermöglichen.

4.2. Funktionen

Dieses Kapitel gibt einen Überblick über die zentralen Funktionen des umgesetzten Systems. Dabei werden insbesondere der Dokumenten-Upload, die Dokumenten-Klassifikation sowie die Dokumenten-Suche und deren technischer Ablauf beschrieben.

4.2.1. Dokumenten-Upload

Dieses Kapitel dokumentiert den vollständigen Upload-Flow von der Anlage einer Upload-Zone bis zur finalen Speicherung des Dokuments in SharePoint inklusive Metadatenpersistenz und Suchindex-Aktualisierung.

4.2.1.1. Technischer Gesamtflow

Der Upload ist als mehrstufige Pipeline umgesetzt, damit fachliche Prüfung, Benutzerinteraktion und Persistenz klar getrennt bleiben:

1. Eine Zone wird im Frontend erstellt oder ausgewählt.
2. Die Zone wird im Zone-Service persistiert; der zugehörige SharePoint-Ordner wird best-effort angelegt.
3. Eine Datei wird per Dateiauswahl oder Drag & Drop in die Zone-Upload-Queue gestellt.
4. Der Zone-Service führt eine Preflight-Phase aus und ruft die Dokumenten-Klassifikation auf.

5. Ergebniswerte der Klassifikation werden zusammen mit der Datei temporär abgelegt und mit `uploadId` referenziert.
6. Die Benutzerin bzw. der Benutzer bestätigt oder korrigiert die erkannten Metadaten.
7. Beim `commit` wird die Datei nach SharePoint geschrieben und die temporäre Ablage aufgelöst.
8. Die Dokument-Metadaten werden in der Documents-API in der Datenbank gespeichert.
9. Das Dokument wird für die Suche über die Search-API indiziert.

Damit kann der Uploadprozess robust aufgeteilt werden: Die KI-Extraktion ist vor der finalen Persistenz sichtbar und korrigierbar, bevor irreversible Schreiboperationen durchgeführt werden.

4.2.1.2. Zone-Erstellung und SharePoint-Ordnerinitialisierung

Der Upload beginnt fachlich mit einer Zone. Die Zone-API stellt dafür `POST /zones` bereit. Beim Anlegen wird ein neuer `ZoneDto`-Eintrag in der zonenspezifischen JSON-Struktur gespeichert und parallel versucht, den Zielordner in SharePoint zu erzeugen.

```
g.MapPost(
    "",
    async (
        ZoneDto zone,
        IZonesStore store,
        IUserContext user,
        ITokenAcquisition tokenAcquisition,
        IHttpClientFactory httpFactory,
        IConfiguration cfg,
        CancellationToken ct
    ) =>
{
    var folderName = zone.FolderPath.Trim('/').Split('/').Last();
    var driveId = cfg["SharePoint:DriveId"] ?? zone.DriveId ?? "";

    var doc = await store.UpsertAsync(
        user.TenantId,
        user.ObjectId,
        d =>
        {
            var list = d.Zones.ToList();
            var newZone = zone with
            {
                Id = Guid.NewGuid(),
                OrderIndex = list.Count + 1,
                DriveId = driveId
            };
```

```
28
29              list.Add(newZone);
30              return d with { Zones = list };
31          },
32          null,
33          ct
34      );
35
36      try
37      {
38          var token = await tokenAcquisition.GetAccessTokenForUserAsync(
39              new[] { "https://graph.microsoft.com/Files.ReadWrite.All" });
40
41          var client = httpFactory.CreateClient("graph");
42          client.DefaultRequestHeaders.Authorization =
43              new AuthenticationHeaderValue("Bearer", token);
44
45          var payload = new
46          {
47              name = folderName,
48              folder = new { },
49              microsoft_graph_conflictBehavior = "fail"
50          };
51
52          await client.PostAsJsonAsync($"drives/{driveId}/root/children",
        payload, ct);
53      }
54      catch { }
55
56      return Results.Ok(doc);
57  }
58  );
```

Quellcode 4.1: Zone-Erstellung mit SharePoint-Ordnerinitialisierung

Die Ordneranlage ist absichtlich tolerant implementiert (`try/catch` ohne Abbruch), sodass die fachliche Zone nicht verloren geht, wenn die Graph-Operation nicht unmittelbar erfolgreich ist.

4.2.1.3. Frontend-Uploadsteuerung über UploadQueueService

In der Webanwendung wird der Upload nicht direkt aus der Seite abgesendet, sondern über den `UploadQueueService` sequenziell verarbeitet. Dadurch können mehrere Dateien nacheinander pro Zone verarbeitet werden, inklusive Statusmeldungen und Dialogsteuerung.

```
private async Task ProcessRequestAsync(UploadRequest request,
    CancellationToken ct)
{
    if (request.SizeBytes > MaxUploadBytes) return; // 100 MB Grenze

    using var form = new MultipartFormDataContent();
    form.Add(new StreamContent(await OpenUploadStreamAsync(request, ct)),
             "file", request.FileName);

    var res = await client.PostAsync($"zones/{request.ZoneId}/upload/preflight
    ", form, ct);
    if (!res.IsSuccessStatusCode) return;

    var preflight = await res.Content.ReadFromJsonAsync<UploadPreflight>(
    cancellationToken: ct);
    CurrentPreflight = preflight;
    _decision = new TaskCompletionSource<UploadDecision>(TaskCreationOptions.
    RunContinuationsAsynchronously);

    var decision = await _decision.Task.WaitAsync(ct);
    if (!decision.Confirmed || decision.Payload is null) return;

    await CommitAsync(client, decision.Payload, ct);
}
```

Quellcode 4.2: Preflight- und Commit-Steuerung im UploadQueueService

Wesentliche technische Punkte in dieser Schicht sind:

- zentrale Dateigrößenbegrenzung auf 100 MB,
- einheitlicher Status pro Zone (z. B. `"KI analysiert Dokument..."`),
- ein expliziter Bestätigungszustand zwischen Preflight und Commit,
- sowie identischer Ablauf für Dateiauswahl und Drag-&-Drop.

4.2.1.4. Preflight-Endpunkt und Klassifikationsaufruf

Die eigentliche serverseitige Vorverarbeitung startet über den Preflight-Endpunkt der Zone-API. Der Endpunkt nimmt das Multipart-File entgegen, sendet es an den AI-Service und legt danach Datei plus Metadaten temporär ab.

```
g.MapPost(
    "{zoneId:guid}/upload/preflight",
    async (
        Guid zoneId,
        HttpRequest req,
```

```
6           FileZonesStore store,
7           IUserContext user,
8           IHttpClientFactory httpFactory,
9           CancellationToken ct
10      ) =>
11  {
12      var form = await req.ReadFormAsync(ct);
13      var file = form.Files["file"];
14      if (file == null) return Results.BadRequest("No file");
15
16      var ai = httpFactory.CreateClient("ai");
17      using var mp = new MultipartFormDataContent();
18      mp.Add(new StreamContent(file.OpenReadStream()), "file", file.FileName);
19
20      var aiResp = await ai.PostAsync("/ai/classify", mp, ct);
21      if (!aiResp.IsSuccessStatusCode) return Results.Problem("KI-Dienst Fehler"
        );
22
23      var extraction =
24          await aiResp.Content.ReadFromJsonAsync<DocumentClassificationResultDto
        >(
25              new JsonSerializerOptions
26              {
27                  PropertyNamingPolicy = JsonNamingPolicy.CamelCase
28              },
29              ct
30          );
31
32      var (uploadId, docType) = await store.SaveTempAsync(
33          user.TenantId,
34          user.ObjectId,
35          zoneId,
36          file,
37          extraction!.DocumentType,
38          extraction.DocumentYear,
39          extraction.ContractType,
40          extraction.Partner,
41          extraction.ExpireDate,
42          ct
43      );
44
45      return Results.Ok(new
46      {
47          uploadId,
48          documentType = docType.ToString(),
49          year = extraction.DocumentYear,
50          partner = extraction.Partner,
51          contractType = extraction.ContractType,
```

```
52          expireDate = extraction.ExpireDate
53      });
54  }
55  ).DisableAntiforgery();
```

Quellcode 4.3: Preflight-Endpunkt im Zone-Service

Die Klassifikation selbst (Prompting, Schema-Bindung, Nachvalidierung) ist in Kapitel 4.2.2 detailliert beschrieben und wird hier nur als aufgerufener Baustein verwendet.

Die Methode `SaveTempAsync(...)` persistiert in dieser Phase:

- die hochgeladene Binärdatei im temporären Bereich,
- ein Metadaten-JSON mit `uploadId`, Zone, Dateieigenschaften und Klassifikation,
- sowie den Zeitstempel der KI-Auswertung (`ClassifiedAtUtc`).

4.2.1.5. Temporäre Ablage und Benutzerkorrektur

Die temporäre Ablage entkoppelt KI-Ergebnis und finale Speicherung. Das Frontend zeigt die zurückgegebenen Werte in einem Dialog (`UploadPreflight`) an. Dort können Dokumenttyp, Jahr, Partner, Vertragsart und Ablaufdatum vor dem Commit angepasst werden.

Die finale Commit-Nachricht enthält damit nicht nur die `uploadId`, sondern den bestätigten Metadatenstand aus der Benutzeroberfläche.

4.2.1.6. Commit in SharePoint und technische Verarbeitung

Der Commit wird über `POST /zones/{zoneId}/upload/{uploadId}/commit` gestartet. Die Hauptarbeit liegt in `CommitTempToSharePointAsync(...)`:

- Laden der temporären Datei und KI-Metadaten,
- Merge mit benutzerbestätigten Änderungen,
- SHA-256-Bildung zur Integritätsreferenz,
- Zielpfad-Aufbau aus Zone-Ordner und optionalem Partner-Unterordner,
- Upload der Datei in SharePoint via Microsoft Graph,
- optionale Textextraktion bei PDF für spätere Suche,
- Löschung der temporären Artefakte.

```
1  var finalMeta = aiMeta with
2  {
3      ExpireDate = confirmed.ExpireDate,
4      Partner = confirmed.Partner ?? aiMeta.Partner,
5      ContractType = confirmed.ContractType ?? aiMeta.ContractType,
6      DocumentYear = confirmed.DocumentYear ?? aiMeta.DocumentYear,
```

```
7      DocumentType = confirmed.DocumentType ?? aiMeta.DocumentType
8  };
9
10 var partnerFolder = SanitizeFolderName(finalMeta.Partner);
11 if (!string.IsNullOrWhiteSpace(partnerFolder))
12 {
13     await EnsureFolderExistsAsync(client, driveId, zoneFolder, partnerFolder,
       ct);
14     targetFolder = string.IsNullOrWhiteSpace(zoneFolder) ? partnerFolder : $"{
       zoneFolder}/{partnerFolder}";
15 }
16
17 var uploadUrl = string.IsNullOrEmpty(targetFolder)
18     ? $"drives/{encodedDriveId}/root:/{encodedFileName}:/content"
19     : $"drives/{encodedDriveId}/root:/" +
20       $"{EncodePathForGraph(targetFolder)}/{encodedFileName}:/content";
21
22 using (var stream = File.OpenRead(binPath))
23 {
24     var resp = await client.PutAsync(uploadUrl, new StreamContent(stream), ct)
       ;
25     resp.EnsureSuccessStatusCode();
26     var item = await resp.Content.ReadFromJsonAsync<JsonElement>(ct);
27     sharePointItemId = item.GetProperty("id").GetString()!;
28 }
```

Quellcode 4.4: Commit-Logik mit SharePoint-Upload

Die Partner-Unterordner werden vor dem Datei-Upload gezielt erstellt. Ungültige Zeichen werden über `SanitizeFolderName(...)` ersetzt, damit SharePoint-konforme Pfade entstehen.

4.2.1.7. Persistenz in Documents-API und Suchindex

Nach erfolgreichem SharePoint-Write registriert der Zone-Service das Dokument in der Documents-API und übergibt dabei auch `sharePointItemId` und `ownerId`. Danach wird ein interner Indexierungsaufruf an die Search-API ausgelöst.

```
1 var (meta, storedPath, sha256, contentText, sharePointItemId) =
2     await store.CommitTempToSharePointAsync(
3         user.TenantId,
4         user.ObjectId,
5         user.DisplayName ?? "",
6         uploadId,
7         confirmed,
8         ct
9     );
```

```
10
11 var dto = new CreateDocumentDto(
12     zoneId,
13     meta.OriginalFilename,
14     storedPath,
15     meta.MimeType,
16     meta.SizeBytes,
17     sha256,
18     meta.DocumentType,
19     meta.DocumentYear,
20     meta.ContractType,
21     meta.Partner,
22     meta.ExpireDate,
23     sharePointItemId,
24     user.ObjectId
25 );
26
27 var regResp = await docApi.PostAsJsonAsync("/documents", dto, ct);
28 var created = await regResp.Content.ReadFromJsonAsync<CreatedDocumentResponse
       >(ct);
29
30 searchApi.DefaultRequestHeaders.TryAddWithoutValidation(
31     "X-Internal-Key",
32     config["Services:SearchInternalApiKey"]!
33 );
34
35 await searchApi.PostAsJsonAsync(
36     "/internal/index",
37     new IndexDocumentDto(
38         created!.id,
39         zoneId,
40         meta.OriginalFilename,
41         storedPath,
42         meta.DocumentType,
43         meta.DocumentYear,
44         meta.ContractType,
45         meta.Partner,
46         meta.ClassifiedAtUtc,
47         contentText,
48         new[] { user.ObjectId }
49     ),
50     ct
51 );
```

Quellcode 4.5: Commit-Endpunkt mit Metadatenpersistenz und Indexierung

Die Suche wird damit direkt nach dem Upload aktualisiert; es ist kein separater Batchlauf

notwendig, um neue Dokumente auffindbar zu machen.

4.2.1.8. Fehlerbehandlung und Sicherheitsaspekte im Upload-Flow

Der Flow implementiert mehrere Sicherheits- und Robustheitsmechanismen:

- Autorisierung auf allen geschützten Zone-Endpunkten über Benutzerkontext (`TenantId`, `ObjectId`),
- temporäre Entkopplung via `uploadId`, um unbestätigte Uploads nicht sofort final zu persistieren,
- typisierte Persistenzobjekte für API-Grenzen (DTOs für Dokumenterfassung und Suchindexierung),
- interne Indexierung nur mit `X-Internal-Key`,
- ACL-Übernahme in den Index über `AllowedUsers`.

Zusätzlich führen Fehler in Preflight, Commit oder Metadatenpersistenz zu klaren Benutzer-Rückmeldungen im Frontend (Toasts) und unterbrechen den Vorgang kontrolliert, statt inkonsistente Zwischenzustände zu erzeugen.

4.2.1.9. Zusammenfassung der Upload-Implementierung

Die Upload-Implementierung verbindet Benutzerführung, KI-Voranalyse und Persistenz in einem durchgängigen Prozess. Durch die Trennung in Preflight und Commit bleiben KI-Ergebnisse vor dem endgültigen Speichern überprüfbar. Datenhaltung und Suchindexierung werden danach konsistent aktualisiert.

4.2.2. Dokumenten-Klassifikation

Dieses Kapitel dokumentiert die implementierte Dokumenten-Klassifikation und beschreibt den technischen Ablauf vom API-Endpunkt bis zur Nachvalidierung der Ergebnisse.

4.2.2.1. Zweck und Ergebnisobjekt

Die Klassifikation extrahiert aus einem PDF die für die weitere Verarbeitung relevanten Metadaten und liefert diese in einem einheitlichen DTO zurück. Das Ergebnis besteht aus Dokumenttyp, Jahr, Vertragstyp, Partner und optionalem Ablaufdatum.

```
1 public record DocumentClassificationResultDto(
2     DocumentType DocumentType,
3     int? DocumentYear,
4     string? ContractType,
5     string? Partner,
6     DateTime? ExpireDate
```

```
);
```

Quellcode 4.6: Rückgabeobjekt der Klassifikation

Damit ist das Ergebnisformat für alle aufrufenden Komponenten stabil und typisiert.

4.2.2.2. Klassifikations-Endpunkt

Die AI-API stellt den Endpunkt `POST /ai/classify` bereit. Der Controller validiert die Eingabe (nicht leer, MIME-Type PDF) und delegiert die eigentliche Klassifikationslogik an den Service `OpenAiClassifier`.

```
[HttpPost("classify")]
public async Task<ActionResult<DocumentClassificationResultDto>> Classify(
    [FromForm] IFormFile file, CancellationToken ct)
{
    if (file.Length == 0) return BadRequest("Empty file.");
    if (!string.Equals(file.ContentType, "application/pdf",
        StringComparison.OrdinalIgnoreCase))
        return BadRequest("Only PDF supported for now.");

    await using var s = file.OpenReadStream();
    var result = await _classifier.ClassifyPdfAsync(s, file.FileName, ct);
    return Ok(result);
}
```

Quellcode 4.7: API-Endpunkt der Dokumenten-Klassifikation

Die Verantwortung des Controllers bleibt damit bewusst schlank: Validierung, Delegation und Response.

4.2.2.3. Kernlogik im OpenAiClassifier

Der Service `OpenAiClassifier` enthält die eigentliche KI-Interaktion. Die Pipeline besteht aus vier Schritten:

1. Upload der PDF-Datei an die OpenAI Files API,
2. Aufruf der Responses API mit Modell, Prompt und strikt vorgegebenem JSON-Schema,
3. Parsen der strukturierten Antwort,
4. technische und fachliche Nachvalidierung der Felder.

Der wichtigste Baustein ist das Schema-gebundene Prompting, weil dadurch die Antwort in ein verlässliches, maschinenlesbares Format gezwungen wird.

```
var payload = new
{
    model = _model,
    input = new[]
    {
        new {
            role = "user",
            content = new object[]
            {
                new { type = "input_file", file_id = fileId },
                new { type = "input_text", text = prompt }
            }
        }
    },
    text = new
    {
        format = new
        {
            type = "json_schema",
            name = "doc_extraction",
            strict = true,
            schema
        }
    }
};

using var resp = await _http.PostAsync(
    "https://api.openai.com/v1/responses",
    new StringContent(
        JsonSerializer.Serialize(payload),
        Encoding.UTF8,
        "application/json"
    ),
    ct
);
```

Quellcode 4.8: Schema-gebundener Responses-Aufruf

Durch `strict = true` und das definierte Schema wird die Ausgabe stark normalisiert. Das reduziert nachgelagerte Parsingfehler.

4.2.2.4. Parsing und fachliche Nachvalidierung

Nach dem API-Aufruf wird der JSON-Text extrahiert, in die internen Typen gemappt und anschließend fachlich bereinigt. Dabei gelten unter anderem folgende Regeln:

- Jahr nur im Bereich 1900 bis 2100,

- `ContractType` und `ExpireDate` nur für `DocumentType.Vertrag`,
- leere Strings werden auf `null` normalisiert.

```
var typeStr = root.GetProperty("document_type").GetString()!;
var docType = Enum.Parse<DocumentType>(typeStr, ignoreCase: true);

int? year = null;
var y = root.GetProperty("year");
if (y.ValueKind == JsonValueKind.Number && y.TryGetInt32(out var yi))
    year = yi;

if (year is < 1900 or > 2100) year = null;
if (docType != DocumentType.Vertrag)
{
    contractType = null;
    expireDate = null;
}
```

Quellcode 4.9: Nachvalidierung der Klassifikationsergebnisse

Diese Nachvalidierung ist zentral, damit inkonsistente Modellantworten nicht ungeprüft in den weiteren Prozess übernommen werden.

4.2.2.5. Fehlerbehandlung und Robustheit

Die Service-Methoden prüfen HTTP-Antworten zentral über `EnsureOk(...)`. Bei nicht erfolgreichem Status wird die API-Antwort inklusive Statuscode in eine Exception übernommen. Dadurch sind Fehler in Files-Upload oder Responses-Aufruf klar lokalisierbar und beobachtbar.

Zusätzlich unterstützt `ExtractFirstOutputText(...)` mehrere Antwortvarianten, insbesondere ein direktes `output_text`-Feld sowie textuelle Inhalte in der `output`-Struktur. Das macht die Verarbeitung robuster gegenüber Response-Formvarianten.

4.2.2.6. Zusammenfassung der Klassifikationsimplementierung

Die Dokumenten-Klassifikation ist als klar gekapselter Service umgesetzt:

- schlanker API-Endpunkt für PDF-Eingaben,
- schema-basierter KI-Aufruf für strukturierte Extraktion,
- typisierte Ergebnisrückgabe über `DocumentClassificationResultDto`,
- sowie explizite Nachvalidierung der KI-Ergebnisse.

Damit ist die Klassifikationslogik technisch isoliert dokumentiert und als wiederverwendeter Baustein beschrieben.

4.2.3. Dokumenten-Suche

Dieses Kapitel dokumentiert die implementierte Suchfunktion über den gesamten Request-Flow hinweg: von der Benutzerinteraktion in der Weboberfläche bis zur Ausführung der Abfrage in Azure AI Search und der Rückgabe von Treffern inklusive Facetten.

4.2.3.1. Technischer Such-Flow

Die Suche ist als mehrstufiger Ablauf umgesetzt:

1. Die Benutzerin bzw. der Benutzer gibt Suchtext und optionale Filter (Dokumenttyp, Vertragstyp, Jahr, Partner) in der Seite `Search.razor` ein.
2. Das Web-Frontend baut daraus ein `SearchRequestDto` und sendet einen `POST` an den Search-Service (`/search`).
3. Der Search-Service liest die Benutzeridentität (`oid`) aus dem Access Token und erzwingt damit eine benutzerbezogene Zugriffskontrolle.
4. Der `AzureSearchQueryService` erstellt `SearchOptions` (Paging, Facets, Filter) und führt die Abfrage gegen den Azure-Search-Index aus.
5. Die Treffer und Facetten werden in ein `SearchResponseDto` gemappt und an das Frontend zurückgegeben.
6. Das Frontend aktualisiert Trefferliste und Filteroptionen dynamisch anhand der zurückgegebenen Facetten.

4.2.3.2. Frontend: Aufbau und Versand des Such-Requests

Die zentrale Suchmethode im Frontend befindet sich in `Search.razor`. Vor dem eigentlichen Suchaufruf wird eine Synchronisation mit der Zone-API ausgelöst. Anschließend wird das DTO aufgebaut und an den Suchendpunkt gesendet.

```
private async Task RunSearch()
{
    await ZoneApi.PostAsync("/zones/sync-all", null);

    var req = new SearchRequestDto(
        Query: string.IsNullOrWhiteSpace(_query) ? null : _query.Trim(),
        DocumentYear: int.TryParse(_documentYear, out var y) ? y : null,
        DocumentType:
            Enum.TryParse<DocumentType>(_documentType, true, out var dt)
                ? dt
                : null,
        ContractType: string.IsNullOrWhiteSpace(_contractType) ? null :
    _contractType,
        Partner: string.IsNullOrWhiteSpace(_partner) ? null : _partner,
        Skip: 0,
```

```
15          Size: 50
16      );
17
18      var resp = await SearchApi.PostAsJsonAsync("/search", req);
19      resp.EnsureSuccessStatusCode();
20  }
```

Quellcode 4.10: Suchen im Frontend mit DTO-Aufbau und API-Aufruf

Die Implementierung ist bewusst defensiv: Leere Eingaben werden als `null` übertragen, numerische und Enum-Filter werden nur bei erfolgreicher Konvertierung gesetzt. Dadurch bleiben Request und Filterlogik stabil, auch wenn einzelne UI-Felder leer bleiben.

4.2.3.3. Suchmodi und Filterkombinationen

Die Suche unterstützt mehrere praktische Nutzungsarten:

- Volltextsuche über `Query` (z. B. Begriff aus Dateiname oder Dokumentinhalt),
- reine Filterabfrage ohne Suchtext (z. B. nur Jahr, Partner oder Dokumenttyp),
- kombinierte Suche aus Volltext und Filtern (z. B. Suchwort plus Jahr und Partner),
- gezielte Eingrenzung über Vertragsart (`ContractType`) bei Vertragsdokumenten.

Technisch wird bei leerem Suchtext automatisch `"*"` als Suchausdruck verwendet. Dadurch kann das System auch dann Ergebnisse liefern, wenn ausschließlich nach Metadaten wie Jahr, Partner oder Dokumenttyp gefiltert wird.

4.2.3.4. Search API: Authentifizierter Endpunkt

Der Search-Service stellt die fachliche Suchoperation über `POST /search` bereit. Der Endpunkt ist geschützt (`RequireAuthorization()`) und extrahiert die Benutzerkennung aus dem JWT-Token. Ohne gültige `oid` wird der Request abgelehnt.

```
1  app.MapPost(
2      "/search",
3      async (
4          SearchRequestDto req,
5          HttpContext ctx,
6          AzureSearchQueryService svc,
7          CancellationToken ct
8      ) =>
9  {
10     var oid = ctx.User.FindFirst("oid")?.Value
11                ?? ctx.User.FindFirst(
12                    "http://schemas.microsoft.com/identity/claims/
       objectidentifier"
```

```
                    )?.Value;

    if (string.IsNullOrWhiteSpace(oid)) return Results.Unauthorized();

    var result = await svc.SearchAsync(req, oid, ct);
    return Results.Ok(result);
}
).RequireAuthorization();
```

Quellcode 4.11: Search-Endpunkt mit Benutzerkontext

Damit ist sichergestellt, dass jeder Suchaufruf mit einem konkreten Benutzerkontext verknüpft ist und die nachgelagerte Filterlogik diesen Kontext verwenden kann.

4.2.3.5. Query-Service: Paging, Facets und Filter

Die eigentliche Suchlogik liegt im `AzureSearchQueryService`. Dort werden Suchtext, Paging und Facetten konfiguriert. Für leere Suchanfragen wird ein Wildcard-Searchtext (`"*"`) verwendet, damit auch reine Filterabfragen möglich sind.

```
var searchText = string.IsNullOrWhiteSpace(req.Query) ? "*" : req.Query;

var opt = new SearchOptions
{
    IncludeTotalCount = true,
    Skip = Math.Max(0, req.Skip),
    Size = Math.Clamp(req.Size, 1, 100)
};

opt.Facets.Add("documentYear,count:50");
opt.Facets.Add("documentType,count:50");
opt.Facets.Add("partner,count:50");

opt.Filter = BuildFilter(
    req,
    callerOid,
    includeDocumentType: true,
    includeDocumentYear: true,
    includeContractType: true,
    includePartner: true
);

var resp = await _client.SearchAsync<SearchDocument>(searchText, opt, ct);
```

Quellcode 4.12: SearchOptions und Filteraufbau im Query-Service

Die Limits (`Size` zwischen 1 und 100) schützen den Service vor übergroßen Antworten. Gleichzeitig liefern Facetten die Datenbasis für dynamische Filteroptionen im UI.

Die wichtigste Sicherheitsregel ist im Filteraufbau implementiert: Ergebnisse werden immer auf Dokumente eingeschränkt, die für den aktuellen Benutzer freigegeben sind.

```
private static string BuildFilter(SearchRequestDto req, string callerOid, ...)
{
    var parts = new List<string>();

    parts.Add($"allowedUsers/any(u: u eq '{EscapeOData(callerOid)}')");

    if (req.DocumentYear is not null)
        parts.Add($"documentYear eq {req.DocumentYear.Value}");
    if (req.DocumentType is not null)
        parts.Add($"documentType eq '{EscapeOData(req.DocumentType.Value.
    ToString())}'");
    if (!string.IsNullOrWhiteSpace(req.Partner))
        parts.Add($"partner eq '{EscapeOData(req.Partner)}'");

    return string.Join(" and ", parts);
}
```

Quellcode 4.13: Benutzerbezogener OData-Filter

Durch die Kombination aus ACL-Bedingung und fachlichen Filtern entsteht ein Security-First-Ansatz: Ein Dokument wird nur dann zurückgegeben, wenn sowohl die Berechtigung als auch die Suchkriterien erfüllt sind.

4.2.3.6. Rückgabe und UI-Aktualisierung

Nach der Suchausführung werden die Index-Dokumente auf `SearchHitDto`-Objekte gemappt. Zusätzlich werden Facettenwerte in Bucket-Strukturen überführt. Das Frontend nutzt diese Buckets, um Filterlisten (z. B. Jahre, Vertragstypen, Partner) mit Trefferanzahl zu aktualisieren.

Dadurch entsteht eine adaptive Suchoberfläche: Filter zeigen nur Werte an, die im aktuellen Ergebnisraum tatsächlich vorkommen. Das reduziert ungültige Kombinationen und verbessert die Bedienbarkeit bei großen Dokumentmengen.

4.2.3.7. Zusammenfassung der Implementierung

Die implementierte Dokumentensuche kombiniert:

- einen authentifizierten API-Endpunkt,
- benutzerbezogene Zugriffskontrolle direkt in der Query,

- kombinierbare Volltext- und Metadatenfilter,
- serverseitige Facettenaggregation,
- sowie eine dynamische UI-Nachführung der Filteroptionen.

Damit ist die Suche sowohl funktional als auch sicherheitstechnisch in die Gesamtarchitektur integriert und für den produktiven Einsatz mit mandanten-/benutzerbezogenen Daten geeignet.

4.2.4. Benutzer- und Rollenverwaltung

Die Benutzer- und Rollenverwaltung basiert auf Microsoft Entra ID. Eine lokale Benutzerverwaltung mit eigenen Passwörtern ist in der aktuellen Lösung nicht vorgesehen. Stattdessen übernimmt Entra ID die Anmeldung, stellt die Identitäts-Claims bereit und liefert damit die Grundlage für geschützte UI-Bereiche sowie autorisierte API-Aufrufe.

4.2.4.1. Authentifizierter Zugriff auf Webanwendung und Dienste

Die Webanwendung ist so konfiguriert, dass die Anmeldung über Microsoft Entra ID erfolgt und nach erfolgreicher Authentifizierung Zugriffstoken für nachgelagerte APIs angefordert werden können. Gleichzeitig werden die zentralen UI-Komponenten nur für angemeldete Benutzer gerendert. Auf Serverseite validieren insbesondere Zone- und Search-Service die mitgesendeten JWT-Tokens und schützen ihre Endpunkte über `RequireAuthorization()`.

```
builder.Services
    .AddAuthentication(OpenIdConnectDefaults.AuthenticationScheme)
    .AddMicrosoftIdentityWebApp(builder.Configuration.GetSection("AzureAd"))
    .EnableTokenAcquisitionToCallDownstreamApi()
    .AddInMemoryTokenCaches();

app.MapRazorComponents<App>()
    .AddInteractiveServerRenderMode()
    .RequireAuthorization();
```

Quellcode 4.14: Authentifizierte Webanwendung mit geschütztem UI-Zugriff

Durch diese Kopplung von Anmeldung und Tokenbeschaffung fungiert die Webanwendung als zentraler Einstiegspunkt für den Benutzer, während die Fachdienste getrennt und zielgerichtet angesprochen werden können.

4.2.4.2. Benutzerkontext aus Entra-Claims

Für die fachliche Verarbeitung wird aus den Entra-Claims ein technischer Benutzerkontext gebildet. Relevant sind vor allem Tenant-ID und Object-ID, da diese Identifikatoren im

weiteren Verlauf für benutzerbezogene Datenzugriffe verwendet werden. Zusätzlich werden Anzeigename und E-Mail aus den Claims übernommen.

```
public string TenantId =>
    U.FindFirstValue("tid") ??
    U.FindFirst("http://schemas.microsoft.com/identity/claims/tenantid")?.
    Value ??
    "";

public string ObjectId =>
    U.FindFirstValue("oid") ??
    U.FindFirst("http://schemas.microsoft.com/identity/claims/objectidentifier
    ")?.Value ??
    U.FindFirstValue(ClaimTypes.NameIdentifier) ??
    "";
```

Quellcode 4.15: Ermittlung des Benutzerkontexts aus Entra-Claims

In der Zone-API wird dieser Kontext beispielsweise verwendet, um Zonen pro Mandant und Benutzer zu laden oder Dateien einem konkreten Eigentümer zuzuordnen. Auch die Suche greift auf die übermittelte `oid` zurück, um nur jene Dokumente zurückzugeben, die für den aktuellen Benutzer freigegeben sind.

4.2.4.3. Rollenauflösung im Frontend

Neben der reinen Benutzeridentität werden auch Rollen aus dem Anmeldekontext ausgewertet. Dazu liest der `CurrentUserService` die Claims `roles` und `ClaimTypes.Role` aus. Falls im Token keine explizite Rolle enthalten ist, wird standardmäßig die Rolle `Mitarbeiter` verwendet.

```
public async Task<IReadOnlyList<string>> GetRolesAsync()
{
    var user = await GetUserAsync();
    var roles = new List<string>();

    if (user.Identity?.IsAuthenticated == true)
    {
        roles.AddRange(user.FindAll("roles").Select(c => c.Value));
        roles.AddRange(user.FindAll(ClaimTypes.Role).Select(c => c.Value));
    }

    if (roles.Count == 0)
    {
        roles.Add(DefaultRole);
    }
```

```
16
17      return roles.Distinct().ToList();
18  }
```

Quellcode 4.16: Rollenauflösung mit Standardrolle

Die ermittelten Rollen werden im Frontend derzeit vor allem für die Benutzeranzeige verwendet, etwa im Benutzer-Popover der Navigationsleiste. Eine umfangreiche fachliche Differenzierung mehrerer Rollen ist in der aktuellen Ausbaustufe jedoch nur vorbereitet und noch nicht als durchgängige Berechtigungslogik umgesetzt.

4.2.4.4. Benutzerbezogene Zugriffskontrolle in den Diensten

Die wirksame Zugriffskontrolle erfolgt in der aktuellen Implementierung primär benutzerbezogen. Geschützte Endpunkte in der Zone- und Search-API verlangen eine gültige Authentifizierung, während die konkrete Datenfreigabe über die Entra-Identität des aufrufenden Benutzers erfolgt. So werden Suchtreffer in der Search-API anhand der `oid` eingeschränkt, und in der Documents-API wird die Eigentümerzuordnung über die übergebene Benutzer-ID (`X-User-Oid`) abgesichert.

Diese Kombination aus geschütztem Zugriff, Claim-basierter Benutzeridentifikation und vorbereiteter Rollenauflösung schafft eine solide Grundlage für ein erweiterbares Berechtigungssystem. Gleichzeitig bleibt die aktuelle Lösung technisch überschaubar, da die operative Freigabelogik noch nicht auf komplexe Rollenmatrizen angewiesen ist.

4.2.4.5. Zusammenfassung der Implementierung

Die implementierte Benutzer- und Rollenverwaltung kombiniert:

- zentrale Anmeldung über Microsoft Entra ID,
- Token-basierte Absicherung von Webanwendung und APIs,
- Extraktion von Benutzeridentitäten aus Claims,
- eine einfache Rollenauflösung mit Standardrolle,
- sowie benutzerbezogene Freigaben in Suche, Zonen- und Dokumentdiensten.

Damit ist die Benutzer- und Rollenverwaltung bereits in die technische Gesamtarchitektur integriert und bietet eine belastbare Basis für spätere, feingranularere Rollen- und Berechtigungserweiterungen.

4.2.5. API-Endpunkte

Dieses Unterkapitel gibt einen kompakten Überblick über die wichtigsten API-Endpunkte der einzelnen Services. Im Fokus stehen dabei jene Endpunkte, die im tatsächlichen Ablauf von Upload, Dokumentverwaltung, Suche und Benutzerzugriff verwendet werden.

4.2.5.1. Zone-Service

Der Zone-Service stellt die zentrale fachliche API des Systems bereit. Er verwaltet Upload-Zonen, steuert den Upload-Prozess und bildet die Brücke zu SharePoint sowie zu den nachgelagerten Dokument- und Suchdiensten.

- `GET /zones`: Lädt alle Zonen des aktuell angemeldeten Benutzers.
- `POST /zones`: Legt eine neue Zone an und initialisiert den zugehörigen SharePoint-Ordner.
- `PUT /zones/{id}`: Aktualisiert Eigenschaften einer bestehenden Zone, z. B. Name oder Aktiv-Status.
- `DELETE /zones/{id}`: Entfernt eine Zone aus der benutzerbezogenen Zonenkonfiguration.
- `GET /zones/folders`: Liest verfügbare Ordner aus SharePoint und unterstützt damit die Auswahl eines Zielpfads.
- `POST /zones/{zoneId}/upload/preflight`: Übergibt eine Datei an den KI-Dienst und erzeugt einen temporären Upload mit erkannten Metadaten.
- `POST /zones/{zoneId}/upload/{uploadId}/commit`: Schreibt die bestätigte Datei nach SharePoint, persistiert Metadaten und stößt die Suchindex-Aktualisierung an.
- `POST /zones/sync-all`: Prüft SharePoint und Datenbank auf Konsistenz und entfernt verwaiste Dokumenteinträge.
- `GET /zones/documents/{id}/download`: Lädt eine Datei über die SharePoint-Referenz des Dokuments herunter.
- `GET /zones/documents/{id}/folder-link`: Liefert den Web-Link auf den übergeordneten SharePoint-Ordner des Dokuments.
- `GET /zones/documents/{id}/file-link`: Liefert den direkten Web-Link zur Datei in SharePoint.

Damit bündelt der Zone-Service die operative Hauptlogik der Anwendung in einer zentralen API.

4.2.5.2. Documents-Service

Der Documents-Service verwaltet die persistenten Metadaten der Dokumente in der PostgreSQL-Datenbank. Während die eigentlichen Dateien in SharePoint liegen, werden hier strukturierte Informationen wie Dokumenttyp, Jahr, Partner oder Eigentümer gespeichert.

- `POST /documents`: Registriert ein neues Dokument nach dem Commit in SharePoint und speichert dessen Metadaten.
- `GET /documents/by-zone/{zoneId}`: Liefert alle Dokumentreferenzen einer Zone für einen bestimmten Benutzer.
- `DELETE /documents/{id}`: Löscht einen Dokumenteintrag aus der Datenbank.

- `GET /api/documents/{id}`: Gibt die Detaildaten eines einzelnen Dokuments zurück.
- `GET /api/documents/recent`: Liefert zuletzt erfasste Dokumente eines Benutzers, etwa für Dashboard und Erinnerungsansicht.

Der Service ist damit für die fachliche Verwaltung der Dokumentdaten zuständig, während Dateiinhalt und Dateispeicherung bewusst ausgelagert bleiben.

4.2.5.3. Search-Service

Der Search-Service kapselt die Anbindung an Azure AI Search. Er stellt einerseits die eigentliche Suchfunktion für das Frontend bereit und andererseits interne Endpunkte zur Pflege des Suchindexes.

- `POST /search`: Führt eine benutzerbezogene Suche mit Suchtext, Filtern und Facetten aus.
- `POST /internal/index`: Übernimmt ein Dokument in den Suchindex; dieser Endpunkt wird intern vom Zone-Service verwendet.
- `DELETE /internal/index/{id}`: Entfernt ein Dokument aus dem Suchindex, etwa wenn es aus dem System gelöscht oder als verwaist erkannt wurde.

Dadurch ist die Suche klar vom übrigen System getrennt und kann unabhängig indiziert sowie abgefragt werden.

4.2.5.4. AI-Service

Der AI-Service übernimmt die automatische Dokumentklassifikation vor dem finalen Upload. Er ist bewusst schlank gehalten und stellt derzeit im Wesentlichen einen zentralen Fachendpunkt bereit.

- `POST /ai/classify`: Nimmt eine PDF-Datei entgegen, analysiert sie mit dem KI-Modell und liefert erkannte Metadaten wie Dokumenttyp, Jahr, Partner oder Ablaufdatum zurück.

Dieser Endpunkt wird im Upload-Preflight verwendet, um dem Benutzer vor dem finalen Commit bereits einen strukturierten Vorschlag anzuzeigen.

4.2.5.5. Zusammenfassung der Implementierung

Die API-Landschaft von DropIT ist damit serviceorientiert aufgeteilt:

- Der Zone-Service steuert die fachlichen Hauptabläufe rund um Zonen, Upload und SharePoint.
- Der Documents-Service verwaltet die strukturierten Dokument-Metadaten.
- Der Search-Service stellt Suche und Indexpflege bereit.
- Der AI-Service übernimmt die automatische Voranalyse hochgeladener Dokumente.

5. Beurteilung

5.1. Bewertung der Implementierung

Die Implementierung des Systems war technisch anspruchsvoll. Wir haben mehrere Technologien eingesetzt, die wir in der Schule zuvor nicht verwendet hatten, darunter Blazor WebAssembly als Frontend-Framework, Azure AI Search als Suchdienst, die Microsoft Graph API für die SharePoint-Integration, die OpenAI Responses API mit schema-gebundenen Prompts für die Dokumentenklassifikation sowie die Infrastruktur rund um Microsoft Entra ID. Die Umsetzung ist insgesamt gelungen. Die einzelnen Services sind voneinander getrennt, die Kommunikationswege über REST-Schnittstellen sind klar definiert und sicherheitsrelevante Aspekte wurden berücksichtigt. Die benutzerbezogene Zugriffskontrolle in der Suche und die Trennung zwischen Preflight und Commit im Upload-Flow sind Beispiele dafür. Das Ergebnis ist funktional und entspricht den Anforderungen einer Diplomarbeit. An zwei Stellen haben wir mehr Aufwand betrieben als notwendig. Alle Services wurden in Microsoft Entra ID registriert und entsprechend konfiguriert, obwohl ein vereinfachter Ansatz ausgereicht hätte. Zusätzlich haben wir Azure AI Search als dedizierten Suchdienst eingesetzt, obwohl die Suchfunktion auch mit der bereits vorhandenen PostgreSQL-Datenbank umsetzbar gewesen wäre. Beide Ansätze erschienen uns zum Zeitpunkt der Planung plausibel, weil sie der Vorgehensweise in produktiven Umgebungen entsprechen. Da alle Services intern betrieben werden und nur von einem einzelnen Client aufgerufen werden, hätte eine einfachere Authentifizierung über API-Keys oder eine zentrale Token-Validierung ausgereicht. Für die Suche hätte PostgreSQL mit integrierter Volltextsuche den Anforderungen genügt, da die Dokumentenbasis überschaubar ist und keine komplexen Ranking-Algorithmen oder semantische Suche benötigt wurden.

5.2. Erfüllung der Forschungsfragen

Die erste Forschungsfrage nach einem flexiblen Rollen- und Berechtigungssystem kann nur teilweise als erfüllt bezeichnet werden. Die benutzerbezogene Zugriffskontrolle über Microsoft Entra ID wurde technisch solide umgesetzt, jedoch ist eine feingranulare rollenbasierte Differenzierung mit mehreren fachlichen Rollen, wie sie ursprünglich im Pflichtenheft geplant war, lediglich vorbereitet und noch nicht vollständig realisiert. Die theoretischen Grundlagen zu RBAC und ABAC hätten eine weitergehende Implementierung durchaus ermöglicht.

Die zweite Forschungsfrage nach einer skalierbaren und plattformunabhängigen Architektur ist als weitgehend beantwortet zu bewerten. Durch die serviceorientierte Aufteilung in vier klar getrennte Dienste, die Nutzung cloud-nativer Komponenten und die Docker-Containerisierung konnten die gestellten Anforderungen in der Praxis erfüllt werden. Die im theoretischen Kapitel erarbeiteten Skalierbarkeitsstrategien spiegeln sich direkt in der Wahl der Azure-Dienste wider.

Die dritte Forschungsfrage zur Erkennungsgenauigkeit von KI-Modellen bei der Dokumenttypenerkennung kann als beantwortet betrachtet werden. Die Entscheidung für GPT-4o wurde durch die theoretische Auseinandersetzung mit den Modellfamilien gut begründet, insbesondere hinsichtlich der Eignung für Zero-Shot-Szenarien und der einfachen Erweiterbarkeit

auf neue Dokumenttypen. Die implementierte Nachvalidierung der KI-Ergebnisse sowie die Möglichkeit zur Benutzerkorrektur vor dem finalen Commit zeigen ein bewusstes Bewusstsein für die natürlichen Grenzen automatisierter Klassifikation und wurden gezielt als Absicherung in den Upload-Prozess integriert.

Die vierte Forschungsfrage nach der optimalen Such- und Filterarchitektur ist von allen Forschungsfragen am vollständigsten beantwortet. Die Implementierung folgt direkt dem theoretisch erarbeiteten dreischichtigen Aufbau aus Retrieval, Filterung und Facettenaggregation. Für den vorliegenden Anwendungsfall, bei dem Dokumente primär anhand strukturierter Metadaten wie Dokumenttyp, Jahr und Partner gesucht werden, stellt die lexikalische Volltextsuche über Azure AI Search eine vollständig ausreichende und praxisgerechte Lösung dar.

5.3. Kritische Reflexion und Grenzen

Ein zentrales Problem war, dass die verfügbaren Ressourcen nicht zu Beginn des Projekts geklärt wurden. Während der Implementierung sind Einschränkungen aufgetreten, die bei sorgfältiger Vorplanung erkennbar gewesen wären. Eine Architektur auf Dienste oder Zugänge auszulegen, die später nicht oder nur eingeschränkt nutzbar sind, führt zu vermeidbarem Mehraufwand.

Ein solches Problem tritt auch in der Praxis häufig auf, da bei der Planung oft stärker berücksichtigt wird, was technisch grundsätzlich möglich wäre, als was mit den tatsächlich verfügbaren Mitteln realistisch umgesetzt werden kann. Gerade bei Schulprojekten, in denen Zugänge zu Cloud-Diensten oder Lizenzen nicht selbstverständlich vorhanden sind, hätte diese Klärung früher erfolgen müssen.

Positiv war die Zusammenarbeit im Team. Auftretende Probleme wurden gemeinsam bearbeitet und gelöst, nicht ignoriert oder aufgeschoben. Die Zusammenarbeit funktionierte auch bei unvollständiger Vorplanung. Probleme wurden dadurch nicht verhindert, aber zügig behoben.

Die Empfehlung für künftige Projekte: Ressourcenplanung und technische Planung sollten gemeinsam am Anfang stehen. Die Architektur sollte erst konkretisiert werden, wenn die verfügbaren Mittel bekannt sind.

6. Installation

Dieses Kapitel beschreibt die vollständige Einrichtung der DropIT-Umgebung. Die Einrichtung gliedert sich in drei Bereiche: die Registrierung der Anwendungen im Microsoft Entra Admin Center, die Bereitstellung der erforderlichen Azure-Dienste sowie den abschließenden Start aller Services über Docker Compose.

6.1. Registrierung im Microsoft Entra Admin Center

Damit sich Benutzer über Microsoft Single Sign-On anmelden können und die Backend-Services sicher mit der Microsoft-Infrastruktur kommunizieren, müssen im Entra Admin Center insgesamt vier App-Registrierungen angelegt werden: eine für den Client (`DropIT.Core`) sowie jeweils eine für die drei Backend-APIs.

6.1.1. DropIT.Core

Im Entra Admin Center zu **Identität** → **Anwendungen** → **App-Registrierungen** navigieren und auf **+ Neue Registrierung** klicken. Als Name `DropIT.Core` eintragen, als unterstützte Kontotypen **Nur Konten in diesem Organisationsverzeichnis (Nur ein Mandant – DropIT)** wählen und mit **Registrieren** bestätigen.

Anschließend wird ein Client Secret angelegt. Dazu unter **Zertifikate & Geheimnisse** → **Clientgeheimnisse** auf **+ Neuer geheimer Clientschlüssel** klicken, eine Beschreibung (z. B. `Entwicklung-Key`) sowie ein Ablaufdatum vergeben und mit **Hinzufügen** bestätigen. **Der angezeigte Wert muss sofort kopiert werden, da er nur einmalig angezeigt wird und später nicht mehr abrufbar ist.**

Zuletzt werden die API-Berechtigungen konfiguriert. Unter **API-Berechtigungen** → **+ Berechtigung hinzufügen** → **Von meiner Organisation verwendete APIs** werden nacheinander folgende Scopes hinzugefügt:

- `DropIT.Documents.Api` – Scope: `Documents`
- `DropIT.Search.Api` – Scope: `Search`
- `DropIT.Zones.Api` – Scope: `Zones`

6.1.2. Backend-APIs

Für die drei Backend-Services `DropIT.Documents.API`, `DropIT.Zones.API` und `DropIT.Search.API` wird das folgende Vorgehen jeweils identisch wiederholt.

Unter **App-Registrierungen** auf **+ Neue Registrierung** klicken. Als Name das Schema `DropIT.<name>.API` verwenden, als Kontotyp **Nur ein Mandant (DropIT)** wählen und die Umleitungs-URI leer lassen. Mit **Registrieren** bestätigen.

Anschließend unter **Eine API verfügbar machen** → **+ Bereich hinzufügen** den Bereichsnamen `access_as_user` eintragen, die Einwilligung für **Administratoren und Benutzer** freigeben und den Status auf **Aktiviert** setzen.

Zusätzlich benötigen zwei der APIs spezifische Graph-Berechtigungen, die unter **API-Berechtigungen** hinzugefügt werden:

- `DropIT.Zones.API`: `Files.ReadWrite`, `Sites.Read.All`
- `DropIT.Documents.API`: `Sites.Read.All`, `Sites.ReadWrite`

6.2. Azure-Ressourcen einrichten

6.2.1. Ressourcengruppe

Im Azure Portal das Menü **Ressourcengruppen** öffnen und auf **+ Erstellen** klicken. Als Ressourcengruppenname empfiehlt sich ein einheitliches Schema wie `rg-dropit-core-dev`. Als Region einen geeigneten Standort wählen (z. B. `Germany West Central`) und mit **Überprüfen + erstellen** und **Erstellen** abschließen.

6.2.2. Azure AI Search

Über **Ressource erstellen** nach **Azure AI Search** suchen und den Dienst auswählen. Die zuvor angelegte Ressourcengruppe auswählen und dieselbe Region wie die übrigen Services verwenden, um Latenzen zu minimieren. **Der Dienstname muss global eindeutig sein**, da er Teil der öffentlichen URL wird. Den Tarif entsprechend des Bedarfs wählen und mit **Überprüfen + erstellen** und **Erstellen** bestätigen. Falls die gewählte Region keine freien Kapazitäten meldet, kann auf eine alternative Region ausgewichen werden.

6.3. Start mit Docker Compose

Für den vollständigen Betrieb aller DropIT-Services steht eine fertige `docker-compose.yml` bereit, die das Frontend, die Zones-API, die Documents-API, die Search-API, den AI-Service sowie die PostgreSQL-Datenbank gemeinsam startet.

Vor dem ersten Start müssen alle Zugangsdaten in einer `.env`-Datei im Wurzelverzeichnis des Projekts eingetragen werden. Tabelle 6.1 listet alle erforderlichen Variablen auf.

Sind alle Werte eingetragen, genügt folgender Befehl, um die gesamte Anwendung zu starten:

```
docker compose up --build
```

Quellcode 6.1: Start aller Services via Docker Compose

Alle Services starten dann automatisch in isolierten Containern, die Datenbank wird initialisiert, und das Frontend ist anschließend im Browser erreichbar.

Variable	**Beschreibung**
`ENTRA_TENANT_ID`	Tenant-ID des Microsoft Entra Mandanten
`ENTRA_CLIENT_ID`	Client-ID der `DropIT.Core`-Registrierung
`ENTRA_CLIENT_SECRET`	Das zuvor erstellte Client Secret
`DOCUMENTS_API_CLIENT_ID`	Client-ID der `DropIT.Documents.API`-Registrierung
`ZONES_API_CLIENT_ID`	Client-ID der `DropIT.Zones.API`-Registrierung
`SEARCH_API_CLIENT_ID`	Client-ID der `DropIT.Search.API`-Registrierung
`AZURE_SEARCH_ENDPOINT`	URL des Azure AI Search Dienstes
`AZURE_SEARCH_KEY`	Admin-Schlüssel des Azure AI Search Dienstes
`SHAREPOINT_DRIVE_ID`	Drive-ID der SharePoint-Dokumentbibliothek
`OPENAI_API_KEY`	API-Schlüssel für die Dokumentklassifikation
`POSTGRES_PASSWORD`	Passwort für die PostgreSQL-Datenbank
`SEARCH_INTERNAL_API_KEY`	Interner Schlüssel für die serviceübergreifende Indexierung

Tabelle 6.1.: Erforderliche Umgebungsvariablen in der `.env`-Datei

7. Zusammenfassung und Ausblick

7.1. Zusammenfassung

Zusammenfassend war diese Diplomarbeit ein lehrreiches Projekt, bei dem viele neue Erfahrungen gesammelt wurden. Gleichzeitig befasst sich diese Diplomarbeit mit der Konzeption, Implementierung und Dokumentation des Systems **DropIT**, einer webbasierten Lösung zur strukturierten, sicheren und effizienten Verwaltung digitaler Dokumente. Ziel der Arbeit war es, die Ablage, Klassifikation, Suche und Organisation von Dokumenten in einer einheitlichen Anwendung zu bündeln und dabei sowohl technische als auch organisatorische Anforderungen moderner Dokumentenverwaltung zu berücksichtigen.

Im Rahmen der Arbeit wurde eine serviceorientierte Systemarchitektur umgesetzt, die Benutzeroberfläche, Fachlogik, Identitätsverwaltung und externe Dienste klar voneinander trennt. Die Webanwendung basiert auf Blazor und wird über Microsoft Entra ID authentifiziert. Für die eigentliche Dokumentablage wird SharePoint Online verwendet, während strukturierte Metadaten in PostgreSQL gespeichert und Suchanfragen über Azure AI Search verarbeitet werden. Die zentrale Orchestrierung der Geschäftslogik erfolgt über den Zone-Service, der den Upload-Prozess, die Metadatenverarbeitung sowie die Anbindung der weiteren Backend-Dienste steuert.

Ein wesentlicher Funktionsbaustein von DropIT ist die KI-gestützte Dokumentenklassifikation. Über den AI-Service werden hochgeladene PDF-Dokumente analysiert, automatisch einem Dokumenttyp zugeordnet und um relevante Metadaten wie Jahr, Vertragstyp, Partner oder Ablaufdatum ergänzt. Dadurch wird der manuelle Erfassungsaufwand reduziert und eine konsistentere Weiterverarbeitung der Dokumente ermöglicht. Ergänzend dazu wurde eine Sucharchitektur realisiert, die Volltextsuche, Metadatenfilter und benutzerbezogene Zugriffskontrolle kombiniert, um Dokumente schnell und zielgerichtet auffindbar zu machen.

Die Arbeit zeigt damit, wie sich Microsoft-Dienste, moderne Webtechnologien und KI-gestützte Dokumentenanalyse zu einer praxistauglichen Gesamtlösung verbinden lassen. DropIT stellt eine belastbare Grundlage für eine zentrale Dokumentenverwaltung dar und verdeutlicht, wie Automatisierung, Suchbarkeit und Sicherheit in einem gemeinsamen System zusammengeführt werden können.

7.2. Ausblick

Die entwickelte Lösung stellt eine Basis dar, auf der weitere Funktionen und Erweiterungen aufgebaut werden können. Mögliche nächste Schritte wären die Erweiterung der Webanwendung um eine mobile App sowie der direkte Upload von Dokumenten über Smartphones und Tablets. Dabei könnten Unterlagen mit der Handykamera fotografiert und unmittelbar in das System übertragen werden, wodurch die Erfassung papierbasierter Dokumente deutlich vereinfacht würde.

Darüber hinaus wären weitere Verbesserungen im Bereich der KI-gestützten Dokumentenanalyse, der Metadatenextraktion und der Suche denkbar. Die gewählte Architektur ermöglicht es, solche Erweiterungen schrittweise in das bestehende System zu integrieren.

I. Literaturverzeichnis

[1] Manning, Christopher D., Prabhakar Raghavan, and Hinrich Schütze: *Introduction to Information Retrieval*. Cambridge University Press, 2008. Online in Internet: URL: `https://nlp.stanford.edu/IR-book/` (Zugriff: 05.03.2026).

[2] Amazon Web Services: *Training classification models*. Online in Internet: URL: `https://docs.aws.amazon.com/en_us/comprehend/latest/dg/training-classifier-model.html` (Zugriff: 14.03.2026).

[3] Microsoft Learn: *Document intelligence custom classification model*. Online in Internet: URL: `https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/train/custom-classifier?view=doc-intel-4.0.0` (Zugriff: 14.03.2026).

[4] Ding, Yihao, Soyeon Caren Han, Jean Lee, and Eduard Hovy: *Deep learning based visually rich document content understanding: A survey*, 2025. Online in Internet: URL: `https://arxiv.org/abs/2408.01287` (Zugriff: 14.03.2026).

[5] Microsoft Learn: *What is azure document intelligence in foundry tools?* Online in Internet: URL: `https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/overview?view=doc-intel-4.0.0` (Zugriff: 14.03.2026).

[6] Amazon Web Services: *Document processing*. Online in Internet: URL: `https://docs.aws.amazon.com/comprehend/latest/dg/idp.html` (Zugriff: 14.03.2026).

[7] Amazon Web Services: *What is amazon textract?* Online in Internet: URL: `https://docs.aws.amazon.com/textract/latest/dg/what-is.html` (Zugriff: 14.03.2026).

[8] Google Cloud: *Create, use, and manage a custom document classifier*. Online in Internet: URL: `https://docs.cloud.google.com/document-ai/docs/custom-classifier` (Zugriff: 14.03.2026).

[9] Xu, Yiheng, Minghao Li, Lei Cui, Shaohan Huang, Furu Wei, and Ming Zhou: *Layoutlm: Pre-training of text and layout for document image understanding*. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1192–1200, 2020. Online in Internet: URL: `https://arxiv.org/abs/1912.13318` (Zugriff: 05.03.2026).

[10] Xu, Yiheng, Tengchao Lv, Lei Cui, Furu Wei, Guoyin Wang, and Yiming Lu: *Layoutlmv2: Multi-modal pre-training for visually-rich document understanding*. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, pages 2579–2591, 2021. Online in Internet: URL: `https://arxiv.org/abs/2012.14740` (Zugriff: 05.03.2026).

[11] Amazon Web Services: *Custom classification*. Online in Internet: URL: `https://docs.aws.amazon.com/comprehend/latest/dg/how-document-classification.html` (Zugriff: 14.03.2026).

[12] Griesch, Leon, Leon Görgen und Kurt Sandkuhl: *Ki-als-service: Vergleich von plattformen zur dokumentenklassifikation*. In: *INFORMATIK 2024*, Seiten 1505–1518. Gesellschaft für Informatik eV, 2024.

[13] Microsoft Learn: *Use azure api management to front azure openai and other language models*. Online in Internet: URL: `https://learn.microsoft.com/en-us/azure/api-management/ai-gateway-capabilities` (Zugriff: 08.03.2026).

[14] Microsoft Learn: *Load balance azure openai endpoints with azure api management*. Online in Internet: URL: `https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/azure-openai-gateway-multi-backend` (Zugriff: 08.03.2026).

[15] Microsoft Learn: *Asynchronous request-reply pattern*. Online in Internet: URL: `https://learn.microsoft.com/en-us/azure/architecture/patterns/async-request-reply` (Zugriff: 08.03.2026).

[16] Microsoft Learn: *Retry pattern*. Online in Internet: URL: `https://learn.microsoft.com/en-us/azure/architecture/patterns/retry` (Zugriff: 08.03.2026).

[17] Microsoft Learn: *Circuit breaker pattern*. Online in Internet: URL: `https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker` (Zugriff: 08.03.2026).

[18] OpenAI: *Latency optimization*. Online in Internet: URL: `https://platform.openai.com/docs/guides/latency-optimization` (Zugriff: 08.03.2026).

[19] OpenAI: *Prompt caching 101*. Online in Internet: URL: `https://cookbook.openai.com/examples/prompt_caching101` (Zugriff: 08.03.2026).

[20] scikit-learn developers: *Feature extraction*. Online in Internet: URL: `https://scikit-learn.org/stable/modules/feature_extraction.html` (Zugriff: 14.03.2026).

[21] Stanford NLP Group: *Tf-idf weighting*. Online in Internet: URL: `https://nlp.stanford.edu/IR-book/html/htmledition/tf-idf-weighting-1.html` (Zugriff: 05.03.2026).

[22] Google for Developers: *Machine learning glossary*. Online in Internet: URL: `https://developers.google.com/machine-learning/glossary#embedding` (Zugriff: 14.03.2026).

[23] Devlin, Jacob, Ming Wei Chang, Kenton Lee, and Kristina Toutanova: *Bert: Pre-training of deep bidirectional transformers for language understanding*. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, 2019. Online in Internet: URL: `https://aclanthology.org/N19-1423/` (Zugriff: 14.03.2026).

[24] Mikolov, Tomas, Kai Chen, Greg Corrado, and Jeffrey Dean: *Efficient estimation of word representations in vector space*. arXiv preprint arXiv:1301.3781, 2013. Online in Internet: URL: `https://arxiv.org/abs/1301.3781` (Zugriff: 14.03.2026).

[25] Reimers, Nils and Iryna Gurevych: *Sentence-bert: Sentence embeddings using siamese bert-networks*. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 3982–3992, 2019. Online in Internet: URL: `https://aclanthology.org/D19-1410/` (Zugriff: 14.03.2026).

[26] Sentence Transformers: *Semantic textual similarity.* Online in Internet: URL: https://sbert.net/docs/sentence_transformer/usage/semantic_textual_similarity.html (Zugriff: 14.03.2026).

[27] Dascalescu, Dan and Zain Hasan: *Vector embeddings explained*, January 2023. Online in Internet: URL: https://weaviate.io/blog/vector-embeddings-explained (Zugriff: 03.02.2026).

[28] Elastic: *Text classification.* Online in Internet: URL: https://www.elastic.co/de/what-is/text-classification (Zugriff: 03.02.2026).

[29] Google Cloud: *What are convolutional neural networks (cnns)?* Online in Internet: URL: https://cloud.google.com/discover/what-are-convolutional-neural-networks (Zugriff: 03.02.2026).

[30] Harley, Adam: *Rvl-cdip: A large-scale dataset for document classification.* Online in Internet: URL: https://adamharley.com/rvl-cdip/ (Zugriff: 03.02.2026).

[31] V7 Labs: *Vision transformers (vit): A practical guide.* Online in Internet: URL: https://www.v7labs.com/blog/vision-transformer-guide (Zugriff: 03.02.2026).

[32] Hugging Face: *Layoutlm.* Online in Internet: URL: https://huggingface.co/docs/transformers/en/model_doc/layoutlm (Zugriff: 03.02.2026).

[33] Azure Samples: *Document classification with gpt vision (notebook).* Online in Internet: URL: https://github.com/azure-samples/azure-ai-document-processing-samples/blob/main/samples/python/classification/document-classification-gpt-vision.ipynb (Zugriff: 03.02.2026).

[34] Shaip: *KI-basierte Dokumentenklassifizierung – Vorteile, Prozesse und Anwendungsfälle*, Juli 2025. Online in Internet: URL: https://de.shaip.com/blog/ai-based-document-classification/.

[35] Tesseract OCR Contributors: *Improvequality.* Online in Internet: URL: https://tesseract-ocr.github.io/tessdoc/ImproveQuality.html (Zugriff: 05.03.2026).

[36] scikit-learn developers: *train_test_split.* Online in Internet: URL: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html (Zugriff: 05.03.2026).

[37] scikit-learn developers: *Cross-validation: evaluating estimator performance.* Online in Internet: URL: https://scikit-learn.org/stable/modules/cross_validation.html (Zugriff: 05.03.2026).

[38] Bass, Len, Paul Clements und Rick Kazman: *Software Architecture in Practice.* Addison-Wesley, 3. Auflage, 2012.

[39] Fielding, Roy Thomas: *Architectural Styles and the Design of Network-based Software Architectures.* Dissertation, University of California, Irvine, 2000.

[40] Vyshyvaniuk, Kseniia: *Client-Server Architecture. Advantages and Disadvantages of the Network Computing Model.* https://kitrum.com/blog/client-server-architecture-advantages-and-disadvantages/, November 2025. abgerufen am 21.03.2026.

[41] Richardson, Chris: *Pattern: Monolithic Architecture*, 2018. https://microservices.io/patterns/monolithic.html.

[42] ScienceDirect: *Monolithic Architecture*, 2023. https://www.sciencedirect.com/topics/computer-science/monolithic-architecture.

[43] TechTarget: *What is monolithic architecture in software?*, 2023. https://www.techtarget.com/whatis/definition/monolithic-architecture.

[44] Lewis, James und Martin Fowler: *Microservices – a definition of this new architectural term*, 2014. https://martinfowler.com/articles/microservices.html.

[45] Newman, Sam: *Building Microservices*. O'Reilly Media, 2015.

[46] Dragoni, Nicola, Saverio Giallorenzo, Alberto Lluch Lafuente, Manuel Mazzara, Fabrizio Montesi, Ruslan Mustafin, and Larisa Safina: *Microservices: Yesterday, today, and tomorrow*. In *Present and Ulterior Software Engineering*, pages 195–216. Springer, Cham, 2017. Online in Internet: URL: https://arxiv.org/abs/1606.04036 (Zugriff: 18.03.2026).

[47] Mell, Peter und Timothy Grance: *The NIST Definition of Cloud Computing*. Technischer Bericht, National Institute of Standards and Technology, 2011. https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-145.pdf.

[48] Liu, Fang, Jin Tong, Jian Mao, Robert Bohn, John Messina, Lee Badger und Dawn Leaf: *NIST Cloud Computing Reference Architecture*, 2011. https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication500-292.pdf.

[49] Microsoft Support: *SharePoint – Hilfe und Lernen*, 2024. https://support.microsoft.com/de-de/sharepoint, Zugegriffen am: 02.02.2026.

[50] StudySmarter: *Skalierbare Architekturen: Definition, Arten & Beispiele*, 2024. Online im Internet: URL: https://www.studysmarter.de/schule/informatik/technische-informatik/skalierbare-architekturen/, abgerufen am 27. Januar 2024.

[51] Boehm, Barry W., John R. Brown, and Myron Lipow: *Quantitative evaluation of software quality*. In *Proceedings of the 2nd International Conference on Software Engineering*, pages 592–605, San Francisco, CA, 1976. IEEE Computer Society Press. Online in Internet: URL: https://dl.acm.org/doi/10.5555/800253.807736 (Zugriff: 24.03.2026).

[52] ISO/IEC: *Systems and software engineering – systems and software quality requirements and evaluation (SQuaRE) – system and software quality models*. International Standard ISO/IEC 25010:2011, 2011. Online in Internet: URL: https://www.iso.org/standard/35733.html (Zugriff: 24.03.2026).

[53] Wikipedia-Autoren: *Plattformunabhängigkeit — Wikipedia, Die freie Enzyklopädie*, Dezember 2024. Online im Internet: URL: https://de.wikipedia.org/wiki/Plattformunabh%C3%A4ngigkeit, abgerufen am 27. Januar 2026.

[54] Mooney, James D.: *Bringing portability to the software process*. Technical Report TR-97-1, West Virginia University, Department of Statistics and Computer Science, 1997. Online in Internet: URL: https://www.researchgate.net/publication/2801786_Bringing_Portability_to_the_Software_Process (Zugriff: 24.03.2026).

[55] Kampffmeyer, Ulrich: *ECM Enterprise Content Management*. PROJECT CONSULT Unternehmensberatung GmbH, Hamburg, 2006, ISBN 3-936534-09-8.

[56] Microsoft Learn: *Übersicht über Websites und Websitesammlungen in SharePoint Server*, 2023. `https://learn.microsoft.com/de-de/sharepoint/sites/sites-and-site-collections-overview`, Zugegriffen am: 02.02.2026.

[57] Microsoft Corporation: *SharePoint – Hilfe und Lernen*, 2026. `https://support.microsoft.com/de-de/sharepoint`, Zugriff am 30. März 2026.

[58] Tsolkas, Alexander und Klaus Schmidt: *Rollen und Berechtigungskonzepte: Identity-und Access-Management im Unternehmen*. Springer-Verlag, 2017.

[59] Ferraiolo, David F. und D. Richard Kuhn: *Role-Based Access Controls*. In: *Proceedings of the 15th NIST-NCSC National Computer Security Conference*, Seiten 554–563, Baltimore, MD, 1992.

[60] Sandhu, Ravi S., Edward J. Coyne, Hal L. Feinstein und Charles E. Youman: *Role-Based Access Control Models*. Computer, 29(2):38–47, 1996.

[61] Hu, Vincent C., David Ferraiolo, Rick Kuhn, Adam Schnitzer, Kenneth Sandlin, Robert Miller und Karen Scarfone: *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*. Technischer Bericht NIST SP 800-162, National Institute of Standards and Technology, Gaithersburg, MD, 2014.

[62] Microsoft: *Was ist Microsoft Entra?*, 2025. `https://learn.microsoft.com/de-de/entra/fundamentals/what-is-entra`, Abgerufen am 2. Februar 2026.

[63] Rose, Scott, Oliver Borchert, Stu Mitchell, and Sean Connelly: *Zero trust architecture*. Technical Report NIST SP 800-207, National Institute of Standards and Technology, Gaithersburg, MD, 2020. Online in Internet: URL: `https://doi.org/10.6028/NIST.SP.800-207` (Zugriff: 25.03.2026).

[64] Grassi, Paul, Michael Garcia, and James Fenton: *Digital identity guidelines*. Technical Report NIST SP 800-63-3, National Institute of Standards and Technology, Gaithersburg, MD, 2017. Online in Internet: URL: `https://doi.org/10.6028/NIST.SP.800-63-3` (Zugriff: 25.03.2026).

[65] Elastic: *Query and filter context*. Online in Internet: URL: `https://www.elastic.co/guide/en/elasticsearch/reference/current/query-filter-context.html` (Zugriff: 05.03.2026).

[66] Apache Lucene: *Package org.apache.lucene.search.similarities*. Online in Internet: URL: `https://lucene.apache.org/core/9_9_1/core/org/apache/lucene/search/similarities/package-summary.html` (Zugriff: 05.03.2026).

[67] Robertson, Stephen and Hugo Zaragoza: *The probabilistic relevance framework: Bm25 and beyond*. Foundations and Trends in Information Retrieval, 3(4):333–389, 2009. Online in Internet: URL: `https://www.nowpublishers.com/article/Details/INR-019` (Zugriff: 05.03.2026).

[68] Elastic: *knn search*. Online in Internet: URL: `https://www.elastic.co/guide/en/elasticsearch/reference/current/knn-search.html` (Zugriff: 05.03.2026).

[69] Malkov, Yu A. and Dmitry A. Yashunin: *Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 42(4):824–836, 2020. Online in Internet: URL: `https://arxiv.org/abs/1603.09320` (Zugriff: 05.03.2026).

[70] Elastic: *Reciprocal rank fusion*. Online in Internet: URL: `https://www.elastic.co/guide/en/elasticsearch/reference/current/rrf.html` (Zugriff: 05.03.2026).

[71] Elastic: *Boolean query*. Online in Internet: URL: `https://www.elastic.co/guide/en/elasticsearch/reference/current/query-dsl-bool-query.html` (Zugriff: 05.03.2026).

[72] Elastic: *Bucket aggregations*. Online in Internet: URL: `https://www.elastic.co/guide/en/elasticsearch/reference/current/search-aggregations-bucket.html` (Zugriff: 05.03.2026).

[73] Elastic: *Terms aggregation*. Online in Internet: URL: `https://www.elastic.co/guide/en/elasticsearch/reference/current/search-aggregations-bucket-terms-aggregation.html` (Zugriff: 05.03.2026).

[74] Elastic: *Date histogram aggregation*. Online in Internet: URL: `https://www.elastic.co/guide/en/elasticsearch/reference/current/search-aggregations-bucket-datehistogram-aggregation.html` (Zugriff: 05.03.2026).

[75] Apache Solr: *Faceting*. Online in Internet: URL: `https://solr.apache.org/guide/solr/latest/query-guide/faceting.html` (Zugriff: 05.03.2026).

II. Abbildungsverzeichnis

2.1. Schematische Darstellung der Ausgangslage 12

3.1. Übersicht der Vektordarstellung von Embeddings [27] 22
3.2. Schematischer Aufbau einer monolithischen Architektur [40] 29
3.3. Microservices-Architektur [40] 30
3.4. Übersicht der Microsoft Entra Produktfamilie [62] 38
3.5. Integration von On-Premises- und Cloud-Anwendungen über Microsoft Entra ID [62] 40

4.1. Übersicht der Systemarchitektur 44
4.2. In Microsoft Entra registrierte Anwendungen von DropIT 46

III. Tabellenverzeichnis

3.1. Textklassifizierung vs. Dokumentenklassifizierung [1, 2, 3] 15
3.2. Wichtige Begriffe der Merkmalsextraktion [20, 22] 20
3.3. Relevante Verfahren zur Merkmalsextraktion [20, 21, 23] 20
3.4. Relevante Objekte für Embeddings in Dokumentensystemen [24, 25, 9] . . . 22
3.5. Qualitätsziele in der Dokumentaufbereitung [35] [9] 25
3.6. Merkmale von Softwaresystemen [50] 31
3.7. Techniken zur Leistungsoptimierung [50] 31
3.8. Überblick über SharePoint-Produkte [49] 33
3.9. Komponenten einer Berechtigung [58] 35
3.10. Vergleich von Suchansätzen in Dokumentsystemen [67, 68, 70] 43

6.1. Erforderliche Umgebungsvariablen in der `.env`-Datei 71

A.1. Kapitelverzeichnis 82
A.2. Arbeitstagebuch Jacob Toifl 84
A.3. Arbeitstagebuch Michael Schaider 86

IV. Quellcodeverzeichnis

4.1. Zone-Erstellung mit SharePoint-Ordnerinitialisierung 47
4.2. Preflight- und Commit-Steuerung im UploadQueueService 48
4.3. Preflight-Endpunkt im Zone-Service 49
4.4. Commit-Logik mit SharePoint-Upload 51
4.5. Commit-Endpunkt mit Metadatenpersistenz und Indexierung 52
4.6. Rückgabeobjekt der Klassifikation 54
4.7. API-Endpunkt der Dokumenten-Klassifikation 55
4.8. Schema-gebundener Responses-Aufruf 56
4.9. Nachvalidierung der Klassifikationsergebnisse 57
4.10. Suchen im Frontend mit DTO-Aufbau und API-Aufruf 58
4.11. Search-Endpunkt mit Benutzerkontext 59
4.12. SearchOptions und Filteraufbau im Query-Service 60
4.13. Benutzerbezogener OData-Filter 61
4.14. Authentifizierte Webanwendung mit geschütztem UI-Zugriff 62
4.15. Ermittlung des Benutzerkontexts aus Entra-Claims 63
4.16. Rollenauflösung mit Standardrolle 63

6.1. Start aller Services via Docker Compose 70

A. Anhang

A.1. Arbeitsteilung

Jacob Toifl:

- Integration von Microsoft SharePoint und Entra ID
- Entwicklung der Upload-Funktionalität
- Entwicklung des Erinnerungssystem
- Implementierung der Benutzeroberfläche
- Konzeption der Datenbankstruktur

Michael Schaider:

- Implementierung der KI-gestützten Dokumentenerkennung
- Integration von Azure AI Search
- Implementierung der Such- und Filterarchitektur
- Implementierung der Benutzeroberfläche
- Konzeption der Datenbankstruktur

A.2. Kapitelverzeichnis

Kapitel	Name
1	Jacob Toifl
2.1	Michael Schaider
2.2.1	Jacob Toifl
2.2.2	Michael Schaider
3.1	Michael Schaider
3.2	Jacob Toifl
4.1.1	Michael Schaider
4.1.2	Jacob Toifl
4.2.1	Jacob Toifl
4.2.2	Michael Schaider
4.2.4	Jacob Toifl
4.2.5	Michael Schaider
5	Jacob Toifl / Michael Schaider
6	Jacob Toifl
7	Michael Schaider

Tabelle A.1.: Kapitelverzeichnis

A.3. Projekttagebücher

A.3.1. Projekttagebuch Jacob Toifl

Datum	Zeit	kum.	Tätigkeit
14.09.2025	3h	3h	Antrag der Diplomarbeit
01.10.2025	6h	9h	Erstellung des Pflichtenhefts und Mockups
02.10.2025	3h	12h	Ausarbeitung des Pflichtenhefts
06.10.2025	3h	15h	Ausarbeitung des Anforderungsdokuments
09.10.2025	4h	19h	Überarbeitung des Pflichtenhefts, Erstellung einer Teams-Verbindung mit SharePoint sowie Anpassung des Zeitplans
27.10.2025	3h	22h	Projektsetup erstellt, Beginn der Implementierung des Authentifizierungsservices
28.10.2025	4h	26h	Implementierung des Authentifizierungsservices
29.10.2025	2h	28h	Recherche zu Berechtigungen und Rollen (Microsoft Entra)
15.11.2025	2h	30h	Weitere Recherche zu Berechtigungen und Rollen in Microsoft Entra
21.11.2025	4h	34h	Implementierung des Rollensystems
27.11.2025	2h	36h	LaTeX-Projekt aufgesetzt und Grundstruktur erstellt
28.11.2025	4h	40h	Erweiterung des LaTeX-Dokuments und Fehlerbehebungen im Code
29.11.2025	3h	43h	Ausarbeitung des Kapitels „Skalierbare plattformunabhängige Systemarchitekturen“
03.12.2025	4h	47h	Fertigstellung des Kapitels „Skalierbare plattformunabhängige Systemarchitekturen“
05.12.2025	2h	49h	Anpassung und Besprechung des Inhaltsverzeichnisses
15.12.2025	3h	52h	Implementierung der Zones API
17.12.2025	5h	57h	Implementierung des Dokument-Uploads, Analyse von Problemen mit Dateigrößen
23.12.2025	4h	61h	Implementierung der Dokumentenanzeige ohne Vorschau sowie Downloadfunktion
29.12.2025	4h	65h	Projektbereinigung und Code-Refactoring
30.12.2025	4h	69h	Integration von SharePoint
01.01.2026	6h	75h	Weiterführung der SharePoint-Integration
15.01.2026	4h	79h	Implementierung eines manuellen Pop-ups
22.01.2026	4h	83h	Implementierung der manuellen Einstellung des Ablaufdatums über ein Pop-up
24.01.2026	6h	89h	Fehlerbehebung

Datum	Zeit	kum.	Tätigkeit (Fortsetzung)
27.01.2026	3h	92h	Bereinigung des Git-Repositories gemeinsam mit Projektpartner
27.01.2026	4h	96h	Erweiterung des LaTeX-Dokuments und Zusammenfassung der Meeting-Protokolle
28.01.2026	3h	99h	Weitere Fehlerbehebungen
01.02.2026	4h	103h	Anpassung des Inhaltsverzeichnisses und weitere Recherche
02.02.2026	4h	107h	Integration von SharePoint und Microsoft Entra ID
03.02.2026	5h	112h	Recherche und Ausarbeitung der wissenschaftlichen Arbeit
04.02.2026	6h	118h	Recherche und Ausarbeitung der wissenschaftlichen Arbeit
05.02.2026	4h	122h	Recherche und Ausarbeitung der wissenschaftlichen Arbeit
06.02.2026	5h	127h	Recherche und Ausarbeitung der wissenschaftlichen Arbeit
07.02.2026	5h	132h	Recherche und Ausarbeitung der wissenschaftlichen Arbeit
08.02.2026	6h	138h	Recherche und Ausarbeitung der wissenschaftlichen Arbeit
01.03.2026	4h	142h	Weiterarbeit an der wissenschaftlichen Arbeit
05.03.2026	5h	147h	Weiterarbeit an der wissenschaftlichen Arbeit
08.03.2026	4h	151h	Weiterarbeit an der wissenschaftlichen Arbeit und zusätzliche Recherche
09.03.2026	3,5h	154,5h	Wissenschaftliche Arbeit überarbeitet
14.03.2026	3h	157,5h	InDesign Plakat und PowerPoint Präsentation fertiggestellt
14.03.2026	3h	160,5h	Wissenschaftliche Arbeit überarbeitet
16.03.2026	5h	165,5h	Wissenschaftliche Arbeit überarbeitet
18.03.2026	4h	169,5h	Wissenschaftliche Arbeit überarbeitet
18.03.2026	2h	171,5h	Wissenschaftliche Arbeit Layout angepasst
19.03.2026	3h	174,5h	Wissenschaftliche Installation verfasst
24.03.2026	3h	177,5h	Wissenschaftliche Beurteilung verfasst
26.03.2026	4h	181,5h	Wissenschaftliche Arbeit finalisiert

Tabelle A.2.: Arbeitstagebuch Jacob Toifl

A.3.2. Projekttagebuch Michael Schaider

Datum	Zeit	kum.	Tätigkeit
14.09.2025	2h	2h	Diplomarbeit Antrag
02.10.2025	2,5h	4,5h	Pflichtenheft bearbeiten

Datum	Zeit	kum.	Tätigkeit (Fortsetzung)
06.10.2025	1h	5,5h	Pflichtenheft bearbeiten
09.10.2025	1h	6,5h	User-Cases für Pflichtenheft erstellt
09.10.2025	1,5h	8h	Pflichtenheft bearbeiten
27.10.2025	3h	11h	Projekt Setup, begonnen mit dem Layout des Frontends
28.10.2025	2h	13h	Recherche zur Zones-Architektur
29.10.2025	4h	17h	Zones Service fast fertig implementiert, Consent für API fehlt noch
31.10.2025	3h	20h	Am Zones Service weitergearbeitet
07.11.2025	5h	25h	Zones Service weitergearbeitet und funktionsfähig umgesetzt, getestet über Swagger
10.11.2025	4h	29h	Zones Page und CSS bearbeitet
27.11.2025	4h	33h	Zones Page weitergearbeitet und funktionsfähig gemacht, zusätzlich LaTeX-Inhaltsverzeichnis über Jacobs Laptop begonnen
30.11.2025	3,5h	36,5h	SiteId, DriveId und FolderId ermittelt, Problem mit Consent für `Site.Read.All`, zusätzlich LaTeX funktionsfähig gemacht
04.12.2025	4h	40,5h	An der wissenschaftlichen Arbeit weitergearbeitet
09.12.2025	2h	42,5h	An der wissenschaftlichen Arbeit weitergearbeitet
10.12.2025	2h	44,5h	Git für die wissenschaftliche Arbeit bereinigt
16.12.2025	2h	46,5h	Dokumentenklassifikation getestet
17.12.2025	7h	53,5h	AI Service und Document Service implementiert
22.12.2025	2h	55,5h	Bugs behoben
28.12.2025	4h	59,5h	AI Service weiterbearbeitet
30.12.2025	4h	63,5h	OneDrive-Integration umgesetzt
31.12.2025	3h	66,5h	SharePoint-Integration umgesetzt und OneDrive entfernt
02.01.2026	8h	74,5h	Search API implementiert
03.01.2026	4h	78,5h	Search API weiter implementiert
03.01.2026	5h	83,5h	Synchronisation zwischen SharePoint, PostgreSQL und Azure Indexes implementiert
15.01.2026	4h	87,5h	Search Page weiterbearbeitet
22.01.2026	3h	90,5h	Code Cleanup durchgeführt und Bugs behoben
24.01.2026	5h	95,5h	Search bearbeitet
27.01.2026	2h	97,5h	Bug Fixes und Git gemeinsam mit Jacob bereinigt
27.01.2026	4h	101,5h	Search weiterbearbeitet
31.01.2026	1h	102,5h	Inhaltsverzeichnis der wissenschaftlichen Arbeit bearbeitet
31.01.2026	5h	107,5h	Upload-Prozess asynchron implementiert

Datum	Zeit	kum.	Tätigkeit (Fortsetzung)
01.02.2026	4h	111,5h	Bugs behoben
02.02.2026	3h	114,5h	Inhaltsverzeichnis der wissenschaftlichen Arbeit bearbeitet und Recherche durchgeführt
03.02.2026	5h	119,5h	Merkmalsextraktion und Embeddings ausgearbeitet sowie Recherche durchgeführt
04.02.2026	3h	122,5h	Embeddings ausgearbeitet und Recherche durchgeführt
05.02.2026	4h	126,5h	Recherche durchgeführt und Modellfamilien für Dokumenttypenerkennung ausgearbeitet
11.02.2026	4h	130,5h	Docker eingerichtet
12.02.2026	6h	136,5h	Docker finalisiert, Speicherort aktualisiert und Deployment mit der Firma besprochen
25.02.2026	3h	139,5h	Bugs in der wissenschaftlichen Arbeit behoben und Recherche durchgeführt
05.03.2026	5h	144,5h	An der wissenschaftlichen Arbeit weitergearbeitet und Recherche durchgeführt
07.03.2026	5h	149,5h	An der wissenschaftlichen Arbeit weitergearbeitet und Recherche durchgeführt
08.03.2026	7h	156,5h	An der wissenschaftlichen Arbeit weitergearbeitet und Recherche durchgeführt
14.03.2026	3,5h	160h	Wissenschaftliche Arbeit überarbeitet
16.03.2026	4h	164h	InDesign Plakat und wissenschaftliche Arbeit Teil ergänzt
18.03.2026	2h	166h	Wissenschaftliche Arbeit Ausgangslage und Forschungsfragen ergänzt
19.03.2026	3h	169h	Wissenschaftliche Arbeit überarbeitet
24.03.2026	3h	172h	An der wissenschaftlichen Arbeit weitergearbeitet
26.03.2026	3h	175h	An der Implementierung der wissenschaftlichen Arbeit weitergearbeitet
27.03.2026	2h	177h	Wissenschaftliche Arbeit weitergearbeitet
29.03.2026	3h	180h	Wissenschaftliche Arbeit weitergearbeitet und Beurteilung ergänzt
30.03.2026	2h	182h	Wissenschaftliche Arbeit finalisiert

Tabelle A.3.: Arbeitstagebuch Michael Schaider

A.4. Besprechungsprotokolle

...Hier können auch pdf Dateien eingebunden werden!

Betreuungsprotokoll zur Diplomarbeit lfd. Nr.:

Themenstellung:
Kandidaten/Kandidatinnen:

Jahrgang:
Betreuer/in:
Ort:
Datum:
Zeit:

Besprechungsinhalt:

Name	Notiz

Aufgaben:

Name	Notiz	zu erledigen bis

A.5. Pflichtenheft

htlkrems Bautechnik & IT

Informationstechnische Projekte

Höhere Technische Bundeslehranstalt Krems, 3500, Alauntalstraße 29A
mit Expositur Zwettl, 3910, Hammerweg 1

Projekt		**Team**
Art	Diplomarbeit	***Toifl Jacob*** Schaider Michael *Winkler Norbert, MSc*
Name	DropIT	
Dokument		
Nr	G02	
Name	Pflichtenheft	

Versionsgeschichte

Version	Datum	Autor/in	Änderungen
1.0	1.10.2025	Toifl Jacob	Erstellung des Dokuments
2.0	2.10.2025	Toifl Jacob, Schaider Michael	Erweiterung des Dokuments
3.0	9.10.2025	Toifl Jacob, Schaider Michael	Fertigstellung des Dokuments

Inhalt

1. Einleitung 4
1.1. Zweck des Dokuments 4
1.2. Gültigkeit des Dokuments 4
1.3. Begriffsbestimmungen und Abkürzungen 4
1.4. Überblick über das Dokument 4
1.5. Relevante Dokumente 4
2. Entwicklungsteam 4
2.1. Mitglieder 4
2.2. Rollen und Verantwortungen 4
2.3. Projektmanagement 5
3. Detaillierte Beschreibung der geforderten Produktmerkmale 5
3.1. Ausgangslage 5
3.2. Projektidee 5
3.3. Ziele 5
3.3.1. Muss-Ziele 5
3.3.2. Soll-Ziele 6

3.3.3. Kann-Ziele 6
3.4. Geforderte Funktionen des Produkts 6
3.4.1. FA1: Integration mit Microsoft-Diensten 6
3.4.2. FA2: Dokumentenverwaltung und Upload 6
3.4.3. FA3: Automatische Dokumentenerkennung 7
3.4.4. FA4: Suche und Filter 7
3.4.5. FA5: Erinnerungs- und Benachrichtigungssystem 7
3.4.6. FA6: Rollen- und Berechtigungssystem 7
3.5. Nicht-funktionale Anforderungen des Produkts 7
3.5.1. NFA1: Skalierbarkeit 7
3.5.2. NFA2: Benutzerfreundlichkeit 7
3.5.3. NFA3: Sicherheits- und Datenschutzanforderungen 8
3.6. Benutzerszenarios und Interaktionen mit der Umgebung 8
3.6.1. Login 8
3.6.2. File Upload 8
3.6.3. Dokumentenübersicht und -suche 9
3.6.4. Upload-Zone erstellen 9
4. Systemarchitektur 10
4.1. Abbildung 10
4.2. Subsystem 11
4.2.1. Client-Layer: 11
4.2.2. API-Gateway und Routing: 11
4.2.3. Dokument-Prozessor 11
4.2.4. KI-Service: 11
4.2.5. Such- und Filter 11
4.2.6. Erinnerungs-Service 11
5. Datenbank 12
6. User Interface/Mockups 13
6.1. Landing-Page (Dokumente) 13
6.2. Zonen-Page 13
6.3. Zonen-Page (neue Zone erstellen) 14
6.4. Such-Page 14

6.5. Erinnerung-Page 15
6.6. Log-Page 15

1. Einleitung

1.1. Zweck des Dokuments

Auslegung der Pflichten des DropIT – Projektteams in Bezug auf das Projekt

1.2. Gültigkeit des Dokuments

Die Gültigkeit dieses Dokumentes erstreckt sich über die gesamte Dauer des Projektes.

1.3. Begriffsbestimmungen und Abkürzungen

Abkürzung	*Beschreibung*
Microsoft-SSO	Microsoft Single Sign-On
Meta-Data	Metadaten eines Projekts sind beschreibende Informationen, die das Projekt eindeutig kennzeichnen und die Verwaltung erleichtern.

1.4. Überblick über das Dokument

Auslegung der Anforderungen, Projektübersicht, Systemarchitektur und Definition aller Schnittstellen

1.5. Relevante Dokumente

Keine

2. Entwicklungsteam

2.1. Mitglieder

Toifl Jacob – 5AHIT

Schaider Micheal – 5AHIT

2.2. Rollen und Verantwortungen

Rolle	***Name***
Projektleiter	Toifl Jacob
Mitglied	Schaider Michael

2.3. Projektmanagement

- MS-Office Suite
- JIRA
- GIT Version Control System
- GITHub Actions CI/CD

3. Detaillierte Beschreibung der geforderten Produktmerkmale

3.1. Ausgangslage

Die Ausgangslage ist, dass in der Firma keine **einheitliche** und **strukturierte** digitale Dokumentenablage existiert. Verträge und Unterlagen werden unsystematisch gespeichert, wodurch klare Zugriffsrechte fehlen und wichtige Informationen schwer auffindbar sind. Dies führt zu Fristversäumnissen, Suchaufwand, Fehlern und Sicherheitsrisiken.

3.2. Projektidee

Ziel des Projekts ist die Entwicklung einer **zentralen**, **strukturierten** und **benutzerfreundlichen** digitalen Dokumentenablage. Sie soll eine klare Ordnerstruktur, definierte Zugriffsrechte für verschiedene Benutzerrollen und eine einfache, schnelle Suche nach Dokumenten ermöglichen. So wird die Zusammenarbeit verbessert, Fristversäumnisse verringert und der Zugriff beschleunigt.

3.3. Ziele

3.3.1. Muss-Ziele

1. Der Benutzer kann sich ausschließlich mit seinem bestehenden SharePoint-Konto in das System einloggen.
2. Der Benutzer kann frei definierbare Upload-Zonen anlegen, speichern und bearbeiten.
3. Der Benutzer kann Dokumente per Drag & Drop in konfigurierbare Zonen hochladen.
4. Der Benutzer kann Dokumente hochladen, die automatisch durch eine KI klassifiziert und kategorisiert werden.
5. Der Benutzer hat die Möglichkeit mittels einer Schnellsuchfunktion, Dokumente nach Jahr, Vertragstyp und Partner zu filtern
6. Der Benutzer kann über eine Erinnerungsfunktion Fristen, Vertragslaufzeiten eintragen und erhält rechtzeitig Benachrichtigungen.

3.3.2. Soll-Ziele

1. Der Benutzer kann gespeicherte Dokumente über eine Volltextsuche nach Schlagworten im Titel und Inhalt auffinden.
2. Der Benutzer kann mehrere Dokumente gleichzeitig in eine definierte Zone verschieben.
3. Der Benutzer erhält seine Berechtigungen automatisch über die bestehende Rollen- und Rechteverwaltung der SharePoint-Organisation.
4. Der Benutzer wird mittels Fehlermeldung benachrichtig, wenn während des Verschiebungsprozesses Fehler auftreten.
5. Der Benutzer kann alle Aktionen in einem Bewegungsprotokoll mit Zeitstempel nachverfolgen
6. Der Benutzer kann gespeicherte Dokumente im Vorschaumodus ansehen und herunterladen.

3.3.3. Kann-Ziele

1. Der Benutzer kann mittels einer Rückgängig-Funktion zuletzt verschobene Dateien zurückholen / entfernen.
2. Der Benutzer kann über frei definierbare Hotkeys direkt auf bestimmte Upload-Zonen zugreifen.
3. Der Benutzer kann Zielordner definieren, die sich auch auf Netzlaufwerken befinden.

3.4. Geforderte Funktionen des Produkts

Basierend auf den Projektzielen und den Anwendungsszenarien wurden die Anforderungen in funktionale und nicht-funktionale Anforderungen unterteilt, um klare Entwicklungsrichtlinien bereitzustellen.

3.4.1. FA1: Integration mit Microsoft-Diensten

1. Das System soll die Authentifizierung über Microsoft Entra ID (Azure AD) ermöglichen.
2. Das System soll sich über die Microsoft Graph API mit SharePoint verbinden können.
3. Das System soll Dokumente zentral in SharePoint speichern und abrufen.

3.4.2. FA2: Dokumentenverwaltung und Upload

1. Das System soll Benutzern ermöglichen, Dokumente per Drag-and-Drop oder Dateiauswahl hochzuladen.
2. Das System soll hochgeladene Dokumente automatisch versionieren.
3. Das System soll Statusmeldungen (z. B. „Upload erfolgreich", „Fehler") anzeigen.

3.4.3. FA3: Automatische Dokumentenerkennung

1. Das System soll mittels KI-Modellen den Dokumententyp (z. B. Vertrag, Rechnungen) automatisch erkennen.
2. Das System soll relevante Metadaten (z. B. Vertragslaufzeit, Ablaufdatum, Kundennamen) aus dem Dokumentinhalt extrahieren.
3. Das System soll die erkannten Informationen in strukturierter Form speichern.

3.4.4. FA4: Suche und Filter

1. Das System soll eine Volltextsuche über Dokumentnamen, Metadaten und Inhalte ermöglichen.
2. Das System soll Filterfunktionen nach Dokumenttyp, Datum, Benutzer oder Status bereitstellen.

3.4.5. FA5: Erinnerungs- und Benachrichtigungssystem

1. Das System soll automatisch Erinnerungen für Fristen, Vertragsabläufe und Wiedervorlagen erstellen.
2. Das System soll Benachrichtigungen visuell in der Benutzeroberfläche anzeigen.

3.4.6. FA6: Rollen- und Berechtigungssystem

1. Das System soll ein flexibles Rollenmodell (z. B. Administrator, Mitarbeiter, Externer) unterstützen.
2. Das System soll sicherstellen, dass Benutzer nur auf ihre berechtigten Dokumente zugreifen können.

3.5. Nicht-funktionale Anforderungen des Produkts

3.5.1. NFA1: Skalierbarkeit

1. Das System soll in Containern (z. B. Docker) lauffähig sein.
2. Das System soll bei steigendem Datenvolumen horizontal skalierbar sein.
3. KI-Dienste und Backend-Instanzen sollen unabhängig voneinander skalierbar sein.
4. Das System soll sich für den Einsatz in Cloud-Umgebungen eignen.

3.5.2. NFA2: Benutzerfreundlichkeit

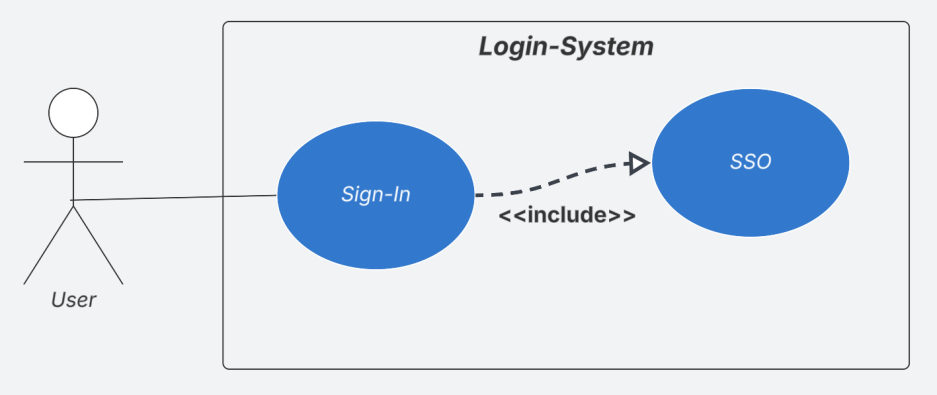
1. Die Benutzeroberfläche soll intuitiv, klar strukturiert und responsiv gestaltet sein.
2. Wichtige Funktionen (Upload, Suche, Filterung) sollen mit maximal zwei Klicks erreichbar sein.
3. Fehlermeldungen sollen verständlich dargestellt werden

3.5.3. NFA3: Sicherheits- und Datenschutzanforderungen

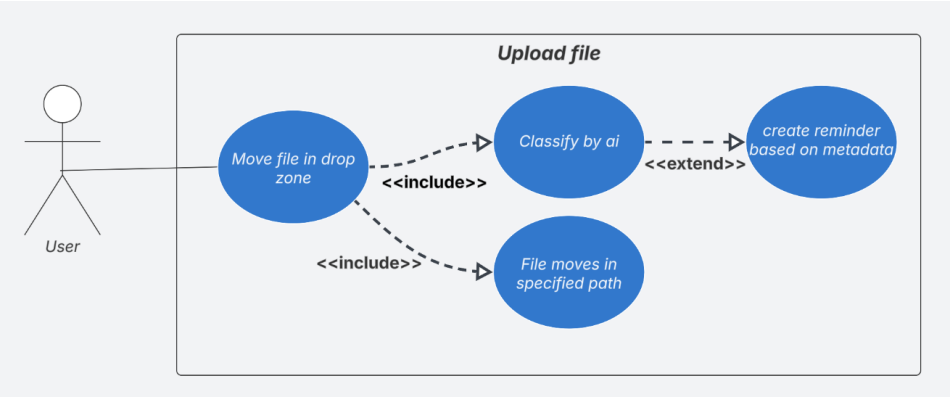
1. Der Zugriff auf das System darf nur über authentifizierte Microsoft-Konten erfolgen.
2. Die gesamte Kommunikation zwischen Client und Server muss verschlüsselt (HTTPS) erfolgen.
3. Das System darf keine vertraulichen Inhalte außerhalb der Unternehmensumgebung speichern.

3.6. Benutzerszenarios und Interaktionen mit der Umgebung

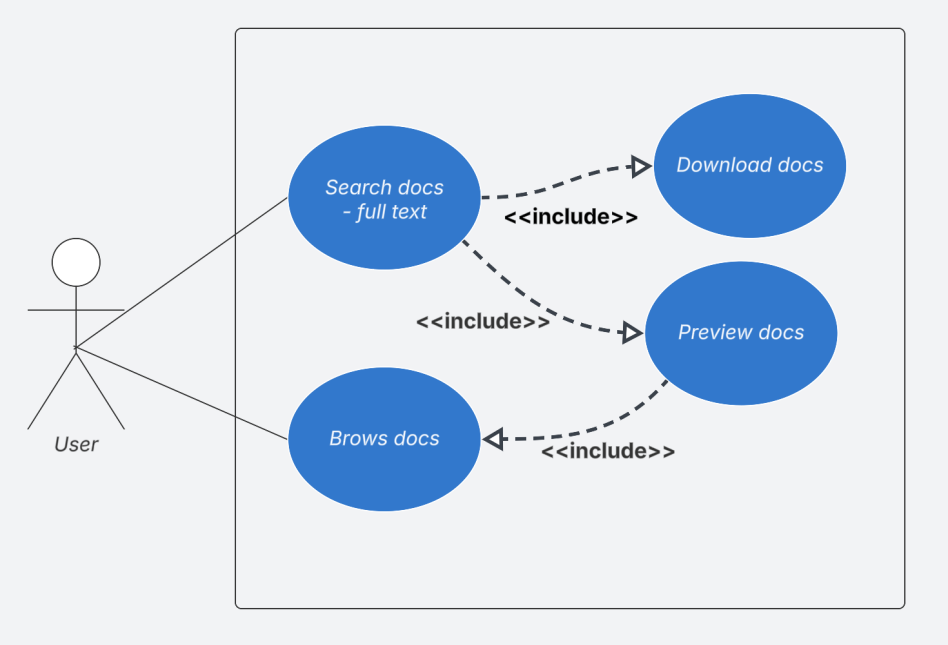
3.6.1. Login



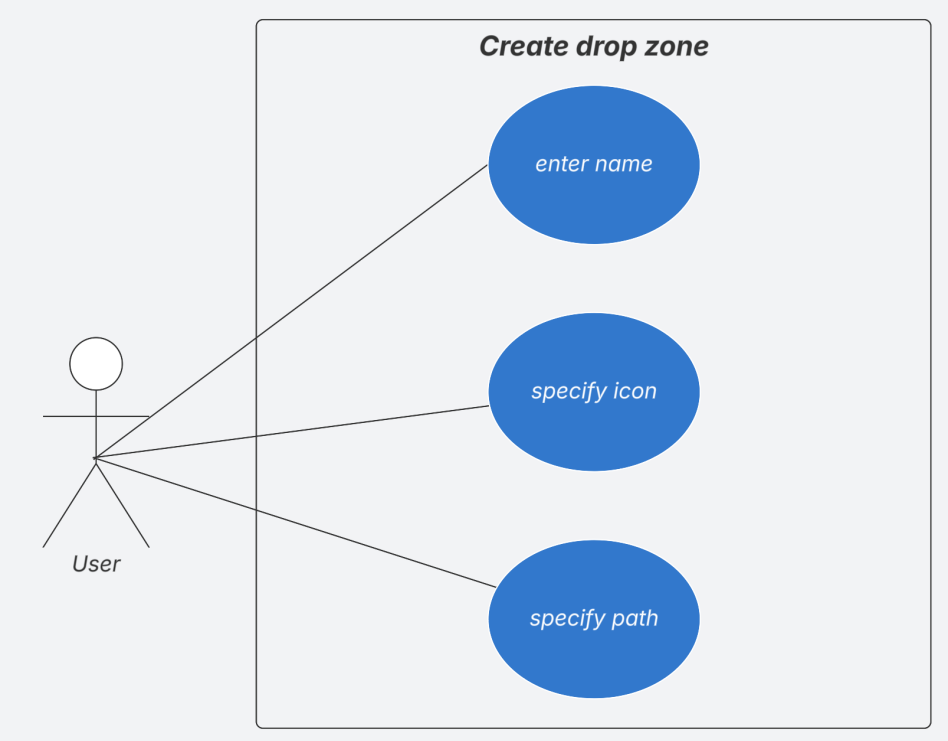
3.6.2. File Upload



3.6.3. Dokumentenübersicht und -suche



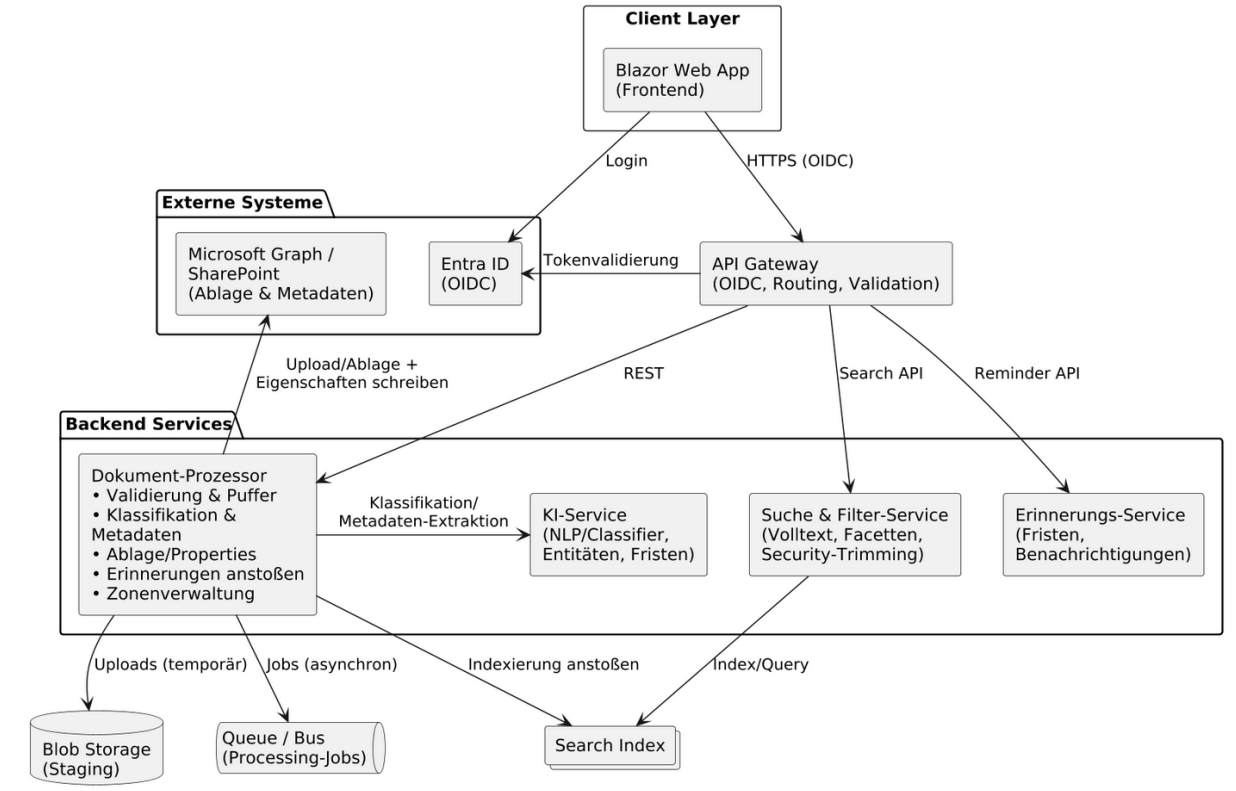
3.6.4. Upload-Zone erstellen



4. Systemarchitektur

Wir nutzen eine verteilte Architektur mit kleinen Diensten (Microservices). Die Blazor-App spricht über ein zentrales API-Gateway mit den Services für Upload/Ingestion, KI-Klassifikation & Metadaten-Extraktion, SharePoint-Anbindung (über Microsoft Graph), Suche/Filter und Erinnerungen. So können Dokumente sicher hochgeladen, automatisch erkannt, mit Metadaten gespeichert und schnell gefunden werden – und das System bleibt gut skalierbar und erweiterbar.

4.1. Abbildung



4.2. Subsystem

4.2.1. Client-Layer:

Das Frontend besteht aus einer .NET-basierten Blazor Web App. Diese Anwendung integriert die bereitgestellten SDKs und APIs, sodass Benutzer Dokumente hochladen, durchsuchen, anzeigen und herunterladen können.

4.2.2. API-Gateway und Routing:

Ein zentrales API-Gateway verarbeitet eingehende Anfragen der Clients und leitet sie an die entsprechenden Backend-Dienste weiter. Diese Komponente übernimmt Authentifizierung (OIDC/Entra ID), Load-Balancing und Request-Validierung, um eine sichere und effiziente Kommunikation zwischen Clients und Backend sicherzustellen.

4.2.3. Dokument-Prozessor

Der Verarbeitungs-Layer übernimmt Uploads aus der Blazor-App, validiert und puffert Dateien, klassifiziert sie mit lokalen KI-Modellen, extrahiert Metadaten und führt Aktionen wie Ablage in SharePoint, Aktualisieren von Eigenschaften und Erstellen von Erinnerungen aus. Zudem verwaltet er Upload-Zonen, erstellt und konfiguriert neue Bereiche, passt bestehende an oder archiviert sie und hinterlegt Regeln sowie Berechtigungen, sodass Upload, Erkennung, Ablage und Organisation konsistent aus einer Schicht erfolgen.

4.2.4. KI-Service:

Dieser Dienst interpretiert die extrahierten Inhalte kontextsensitiv, versteht die Benutzerabsicht (z. B. Ablageort, Dokumententyp, Fristen) und ordnet die Felder konkreten Anwendungsaktionen zu (Ablage, Metadaten schreiben, Erinnerungen anlegen).

4.2.5. Such- und Filter

Dieser Dienst ermöglicht die Volltextsuche und das Filtern von Dokumenten nach bestimmten Kriterien. Er nutzt den Suchindex, um schnelle und präzise Ergebnisse zu liefern und sorgt gleichzeitig für Security-Trimming, sodass Benutzer:innen nur die für sie freigegebenen Dokumente sehen können.

4.2.6. Erinnerungs-Service

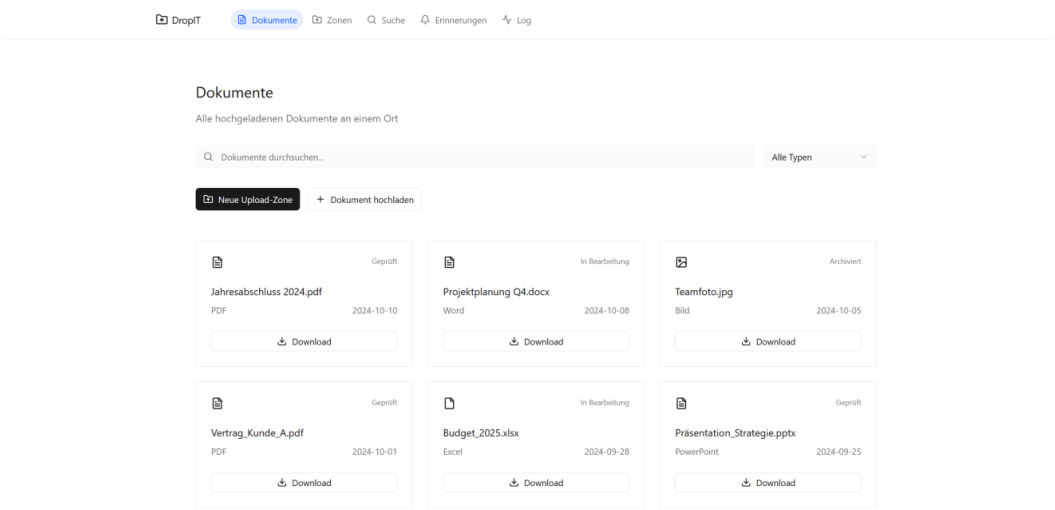
Dieser Dienst verwaltet Fristen und Benachrichtigungen zu Dokumenten. Er analysiert Metadaten, legt Erinnerungen an und benachrichtigt Benutzer:innen automatisch über anstehende Aufgaben oder Fristüberschreitungen

5. Datenbank

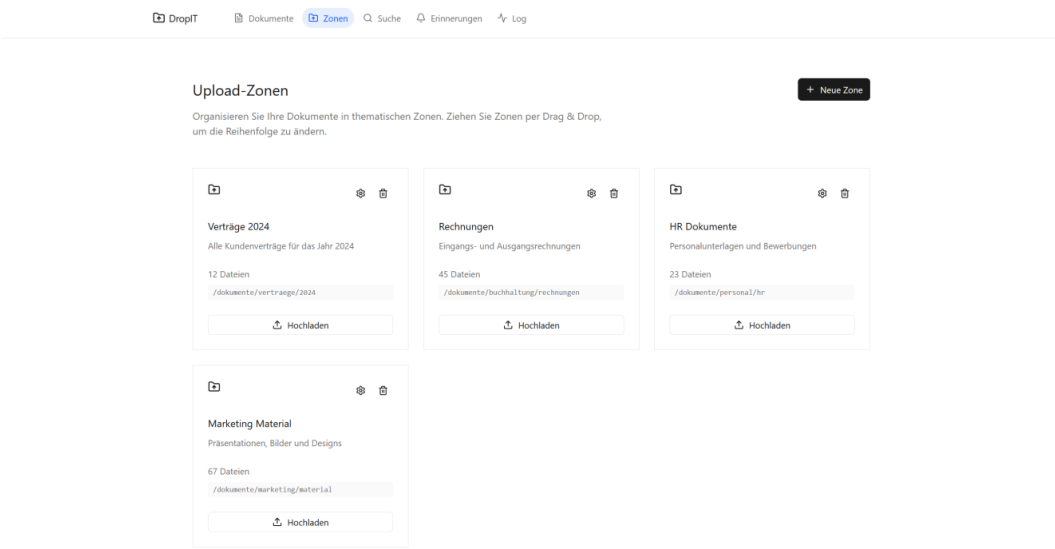
Für die dauerhafte Speicherung der Daten wird PostgreSQL eingesetzt. In der Datenbank werden alle wichtigen Informationen wie Metadaten, Benutzereinstellungen, Erinnerungen und Protokolle der einzelnen Dienste gesammelt. So bleibt das System übersichtlich, zuverlässig und kann bei Bedarf einfach erweitert werden.

6. User Interface/Mockups

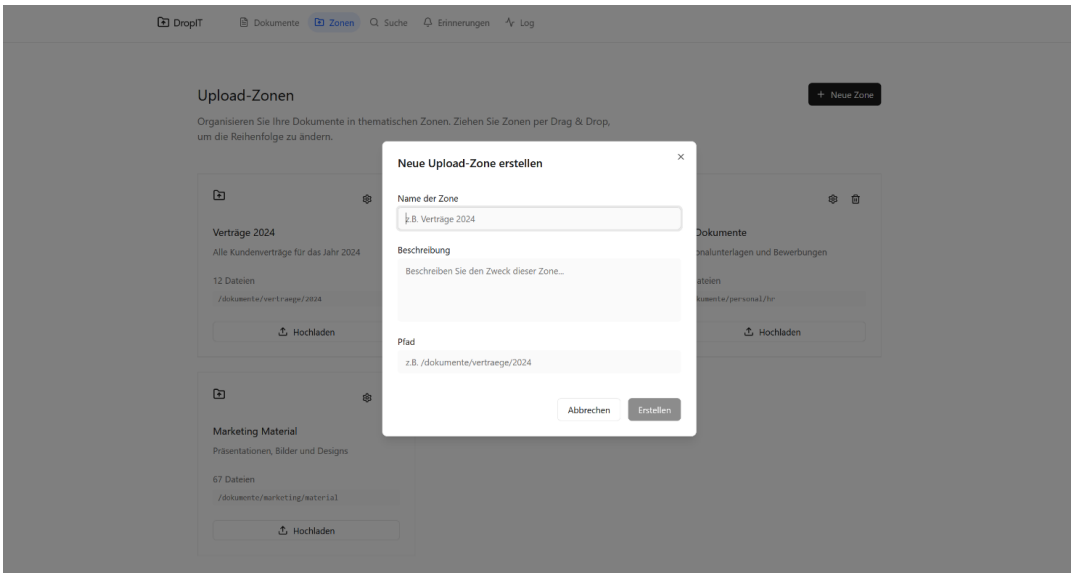
6.1. Landing-Page (Dokumente)



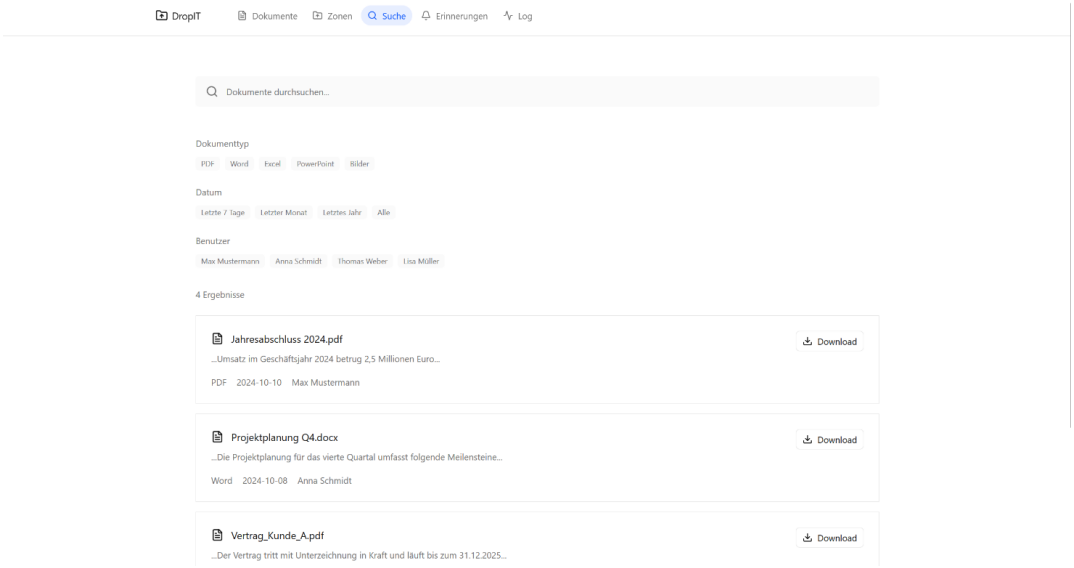
6.2. Zonen-Page



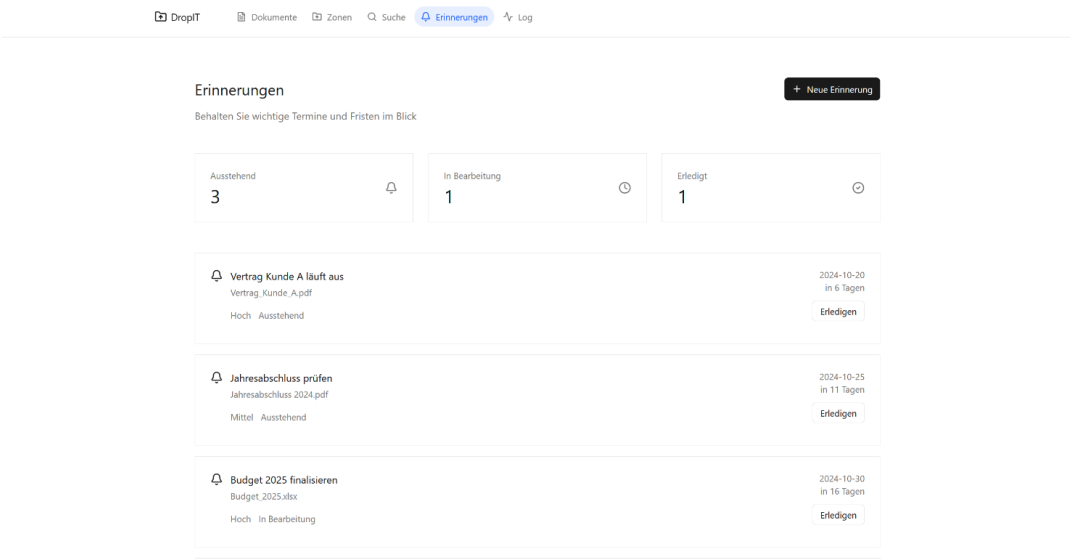
6.3. Zonen-Page (neue Zone erstellen)



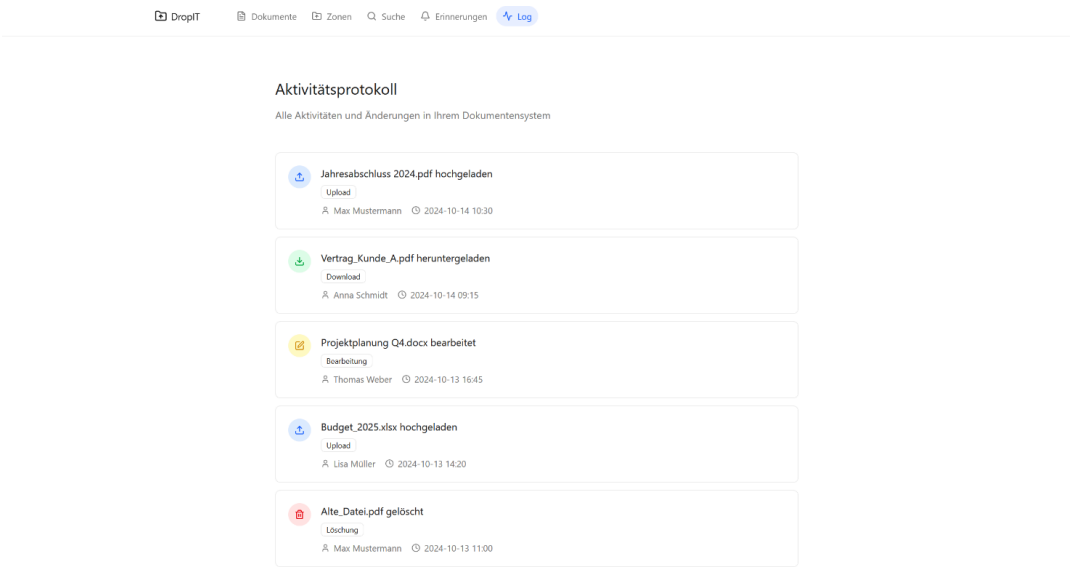
6.4. Such-Page



6.5. Erinnerung-Page



6.6. Log-Page



A.6. Datenträgerbeschreibung

Auf dem beigefügten Datenträger befinden sich die folgenden Dateien und Ordner:

- Im Ordner source befindet sich der Quellcode der entwickelten Anwendung.
- Im Ordner thesis befindet sich der schriftliche Teil der Diplomarbeit.
- Im Ordner video befinden sich die Videodateien der Arbeit.
- Im Ordner documents befinden sich weitere Dokumente, die im Rahmen der Arbeit erstellt wurden (Pflichtenheft).
- Im Ordner dockercompose befindet sich das Docker-Compose-File.